

Foreword

Computing is entering its parallel age, and the GPU is the machine that defines it.

For sixty years, the default path to a faster program was a faster serial processor: a higher clock, a smarter pipeline, a larger cache. That path ran into a wall in the mid-2000s, when clock speeds stopped climbing and silicon stopped cooperating. The industry's answer was not surrender but a change of question - from "how fast can one core go?" to "how many cores can we set free at once?" The GPU is that answer, multiplied a hundred thousand times. A modern accelerator executes tens of trillions of floating-point operations per second, moves memory at terabytes per second, and keeps more threads in flight than there are people on Earth - all directed by code that you, an individual programmer, can write and understand completely.

That last fact is the miracle of the age. The most powerful machine most of us will ever touch is not locked behind corporate walls. Its instruction set is documented. Its programming model is teachable. Its performance is explained by a handful of principles - the warp, the memory hierarchy, the roofline model - that fit on a single page and govern every GPU ever built. Few subjects in computer science offer this much power per hour of study. This book is an invitation to claim it.

Why This Book Exists

Modern software engineering has developed a strange relationship with the GPU. We treat it as a magic box: a library call here, a framework call there, and suddenly our training loop is twenty times faster. The library does the hard part, we tell ourselves, and we never look inside. And that is true - until it is not.

The abstraction leaks at the worst possible moments. Your matrix multiplication runs at 3% of peak because the threads in a warp read columns instead of rows. Your reduction silently drops half of the data because threads in the same warp diverged across a `__syncthreads()`. Your latency doubles because the memory allocation was pageable instead of pinned. None of these failures produce an error message. They produce a benchmark that is embarrassingly slow, or a result that is subtly, catastrophically wrong.

This book builds the bridge between "the library works" and "I understand why it works". The bridge has three lanes: the **hardware** model, the **CUDA C++** programming model, and the modern ecosystem of **C++ idioms**, **Rust** and **CUDA-Oxide** that now surrounds the GPU. Cross it, and the magic box becomes a machine you can reason about - and reason about it is how you make it fast.

What You Will Build

Every chapter builds toward one project: **a complete GPU image-processing pipeline** - read an image, convert it to greyscale, apply a separable Gaussian blur, run a Sobel edge detector, and write the result - implemented three times:

1. in **CUDA C++** with hand-written, fully commented kernels;
2. with the **Thrust/CUB/cuBLAS** library ecosystem;
3. in **pure Rust** with NVIDIA's experimental **CUDA-Oxide** compiler, which turns idiomatic Rust into PTX.

You will not build a toy. You will build the same pipeline a camera vendor would ship, complete with pinned-memory transfers, streamed double buffering, an occupancy-tuned kernel configuration, and reproducible benchmarks. When you have finished, you will be able to look at any CUDA kernel - including the ones inside the libraries you already use - and explain, line by line, what it does and why it is fast.

Who This Book Is For

You should read this book if:

- You can write C++ or Rust, but every GPU program you have written so far was a library call you did not fully understand.
- You have launched a kernel, seen it produce garbage, and had no idea whether the bug was in your index arithmetic, your memory layout, or your synchronisation.
- You suspect that most GPU tutorials skip the hardware model and want to understand the primitives - the warp, the streaming multiprocessor, the memory hierarchy - before touching a single CUDA API.
- You write Rust and want to know what CUDA-Oxide changes, and what it does not.
- You do not own a GPU and want to learn on the free compute that the cloud gives away (see the next section).
- You ship software whose performance budget is measured in microseconds and whose correctness budget is zero.

You do not need prior GPU experience. You need to be willing to sit with the hardware model. This book does not hand-wave the memory hierarchy. Every term is defined when it first appears; every primitive - every type, every built-in variable, every API call - is described before it is used. Where the book refers to a number (register counts, memory bandwidths, transaction sizes), it gives you the reasoning behind it, not just the number.

You Do Not Need to Own a GPU

Every line of code in this book runs on any NVIDIA GPU with CUDA 12.x - including the free ones. If you do not own a GPU, the cloud has you covered, and most providers give away enough free compute to finish this entire book:

- **Google Colab** - free T4 GPUs in browser notebooks, zero setup; the fastest way to run your first kernel.
- **Kaggle Notebooks** - free GPU hours (roughly 30 per week, refreshed weekly), data-science friendly.
- **Google Cloud** - a new-account trial credit (about USD 300 at the time of writing) covers serious L4 and A100 sessions.
- **Microsoft Azure** - a new-account credit (about USD 200) for NC/ND-series GPU virtual machines.
- **AWS** - GPU instances (g4dn, g5, p4); the free tier is CPU-only, but the Activate and Educate programmes grant credits to startups and students.
- **Paperspace / Gradient** - GPU notebooks and cloud workstations with free and low-cost tiers.
- **Lambda, RunPod, Vast.ai** - cheap on-demand GPUs (RTX 4090 up to H100) when you outgrow the free tiers.
- **NVIDIA LaunchPad** - free, time-boxed hands-on labs on real NVIDIA hardware.
- **Modal** - serverless GPU code with recurring free compute credits (about USD 30 per month at the time of writing).

Credit amounts and session limits change frequently; check the current terms before you sign up. The repository README contains a fuller comparison table to help you choose.

The Structure

The book is organised into six parts:

Part I - Foundations of GPU Computing (Chapters 1-3) covers the mathematics of parallelism, the GPU hardware model, and the CUDA programming model. Read this part carefully; every later chapter assumes the primitives defined here.

Part II - Writing CUDA C++ Kernels (Chapters 4-6) covers memory management, synchronisation, atomics, and asynchronous execution with streams and events.

Part III - Optimisation & Advanced Patterns (Chapters 7-9) covers memory optimisation, the canonical parallel algorithms (reduction, scan, histogram), and a complete, step-by-step optimisation of matrix multiplication.

Part IV - Modern C++ & The CUDA Ecosystem (Chapters 10-12) covers RAII wrappers, templates and modern C++ idioms, the Thrust/CUB/cuBLAS libraries, and runtime compilation

with NVRTC.

Part V - Rust, CUDA-Oxide & Safe GPU Programming (Chapters 13-15) covers Rust host code driving CUDA kernels, NVIDIA's experimental CUDA-Oxide compiler for writing kernels in pure Rust, and the image-processing capstone.

Part VI - The Engineering Mindset (Chapter 16) covers profiling with Nsight Compute, debugging with Compute Sanitizer, and reproducible performance engineering.

A Note on the Coding Standards

This project has a constitution. You will find it as `CODING_STANDARDS.md` in the repository root. It is not a suggestion. Every code block in this book follows it:

- Every primitive is explained before it is used.
- No magic numbers - if it is not `0`, `1`, `-1`, or a power of two required by the CUDA API, it gets a named constant.
- Every kernel is commented line by line.
- Every CUDA API call that can fail is checked.
- Every `__syncthreads()` and every atomic carries a comment stating which data it protects and why.

A Note on Hardware and Honesty

The examples in this book target the CUDA 12.x toolkit and are written against the compute capability of modern NVIDIA GPUs (Ada and Hopper architectures, compute capability 8.x and 9.0). You do not need to own this hardware: the section “You Do Not Need to Own a GPU” lists the cloud options, and the free tiers alone are enough for everything in this book. Where a feature is architecture-specific, the book says so explicitly.

This book is also honest about the tooling. NVIDIA's CUDA-Oxide is an experimental, alpha-stage compiler; its API is evolving and its syntax may change. The chapters that cover it describe the project as it exists today, with code written in the style of its documented examples. Treat those chapters as a map of the territory, not a surveyor's certificate.

If you find a bug in the book - in the prose or in the code - open an issue or submit a pull request. This is a living document. The GPU does not stop changing, and neither should the book.

You are one chapter away from understanding the most important machine of our time. Let us build something fast, and understand it.

- *Arpan Pathak*

Chapter 1: The Mathematics of Parallelism

"A fast program is not the same thing as a parallel program. The mathematics below is the difference between the two."

Before we touch a single CUDA API, we must understand what parallelism can and cannot buy us. This chapter establishes the mathematical vocabulary of the entire book: *speedup*, *efficiency*, *Amdahl's law*, *scaling*, *Flynn's taxonomy*, and the *roofline model*. None of these ideas are optional context; they are the instruments you will use to decide, for every kernel in this book, whether a given optimisation is worth the effort.

1.1 Latency, Throughput, and the Meaning of "Faster"

When we say a program is "slow", we usually mean one of two things, and the distinction matters enormously on a GPU.

- **Latency** is the time between the start of an operation and its completion. A network round trip has latency. A single memory access has latency. Latency is measured in time units (nanoseconds, milliseconds).
- **Throughput** is the number of operations completed per unit time. A pipeline that processes 60 frames per second has a throughput of 60 Hz. Throughput is measured in operations per second.

A CPU is designed to **minimise latency**: a small number of very fast cores, each executing one instruction stream with a branch predictor, out-of-order execution, and large caches to hide the latency of DRAM.

A GPU is designed to **maximise throughput**: a very large number of simple execution units, none of which is individually fast, but which together complete millions of operations per clock. The GPU hides latency not by predicting what happens next, but by having so many independent threads in flight that the hardware always has something to do while others wait.

This is the first primitive of the book:

Primitive - latency hiding. If an execution unit must wait for a slow operation (a memory access, a division), the unit is idle. The GPU avoids idleness by switching to another ready thread. The cost of the wait is hidden, not eliminated.

Consequently, a GPU is a poor tool for a single sequential computation and an excellent tool for a computation that can be decomposed into many independent pieces. The mathematics of that decomposition is the subject of this chapter.

1.2 Speedup and Efficiency

Let T_1 be the time a program takes to solve a problem on a single processing unit (a single core, a single thread), and let T_p be the time it takes on p processing units. We define:

Speedup - the ratio of the serial time to the parallel time:

$$S(p) = \frac{T_1}{T_p}$$

A perfect speedup of p means the p -fold work is done in $\frac{1}{p}$ the time. We then define:

Efficiency - the speedup per processing unit:

$$E(p) = \frac{S(p)}{p} = \frac{T_1}{p \cdot T_p}$$

An efficiency of 1.0 (100%) is ideal: every processing unit contributes proportionally. An efficiency of 0.5 means half of the processing units' potential is being wasted. Efficiency is the honest measure; speedup is the flattering one. A vendor will report "10x speedup on 64 cores" and omit that the efficiency is 0.156.

Why efficiency matters on a GPU. A GPU may have tens of thousands of threads in flight. If the achievable efficiency is 20%, you are paying for five times more hardware than you are using. Almost every optimisation in this book is, at heart, an attempt to raise efficiency - by keeping threads busy, by keeping memory transactions full, and by removing serialisation points.

1.3 Amdahl's Law

Gene Amdahl observed in 1967 that any program has a serial fraction: the part that cannot be parallelised (initialisation, I/O, a single reduction step, a dependency chain). Let f be the fraction of the *serial* execution time that is strictly serial. The parallelisable fraction is $(1 - f)$. If the parallel part is perfectly parallelised across p units, the best possible total time is:

$$T_p = f \cdot T_1 + \frac{(1 - f) \cdot T_1}{p}$$

and therefore the maximum speedup is:

$$S(p) = \frac{T_1}{f \cdot T_1 + \frac{(1-f) \cdot T_1}{p}} = \frac{1}{f + \frac{1-f}{p}}$$

The crucial property is the limit as $p \rightarrow \infty$:

$$\lim_{p \rightarrow \infty} S(p) = \frac{1}{f}$$

The serial fraction is a hard ceiling. If 5% of your program is serial, no amount of parallelism can exceed a 20x speedup, because the serial part still takes $0.05 \cdot T_1$ regardless of how many units you add.

An intuition: the manager and the cashiers. Picture a shop with p cashiers and one manager who must personally approve every transaction. The cashiers are the parallel part; the manager is the serial fraction. Hiring more cashiers shortens the queue only up to the point where the manager becomes the bottleneck - and no number of cashiers removes the manager. On a GPU, the “manager” is anything that cannot be parallelised: host launch overhead, a single reduction step, a dependency chain. Amdahl’s law is just the arithmetic of that manager’s unavoidable time.

Worked example. Consider a kernel launch pipeline: 10 microseconds of host overhead (serial) plus a kernel that takes 100 microseconds on one GPU and scales perfectly. Here $f = 10/110 \approx 0.091$. The maximum speedup is $1/0.091 \approx 11$. No matter how many GPUs you buy, the pipeline cannot be faster than 11x. This is why Chapter 6 (streams and asynchronous execution) is dedicated to hiding host overhead: the serial fraction is the enemy.

Why Amdahl’s law is pessimistic. Amdahl assumed the *problem size is fixed*. If the problem grows with the number of processing units, the conclusion changes. That is the subject of the next section.

1.4 Gustafson-Barsis Law

John Gustafson and Edwin Barsis argued in 1988 that in practice the problem size is not fixed: given more hardware, users solve *larger* problems in the same wall-clock time. Let s be the serial fraction of the *parallel* execution time (the time when all p units are busy). The scaled speedup is:

$$S(p) = p + (1 - p) \cdot s$$

Unlike Amdahl’s law, this grows *linearly* with p for fixed s . The two laws answer different questions:

- **Amdahl:** “How much faster does my *fixed* workload run with more units?”
- **Gustafson:** “How much *larger* a workload can I run in the same time with more units?”

Why both matter for GPU programming. When you increase the image resolution or the matrix dimension, you are doing Gustafson scaling: the workload grows, and the GPU's parallel fraction grows with it. When you optimise a fixed-size kernel, you are fighting Amdahl's law. Knowing which regime you are in tells you which optimisation is worthwhile.

1.5 Strong Scaling and Weak Scaling

These two terms name the two regimes above:

- **Strong scaling** - fixed problem size, increasing units. The limit is Amdahl's law. Used for latency-critical workloads where the problem size is dictated by the application (a 1080p frame must be processed at 60 Hz).
- **Weak scaling** - fixed problem size *per unit*, increasing units. The total problem grows with the units. The limit is Gustafson's law. Used for throughput workloads (larger batch, larger grid).

Every kernel configuration decision in this book is a strong-vs-weak scaling decision in miniature: whether to use more threads per element (weak, more parallel work per unit) or fewer threads doing more work each (strong, fixed total work).

1.6 Types of Parallelism

Parallelism is not one idea but several, and each maps to different hardware:

- **Task parallelism** - different *functions* run concurrently on different data (e.g., decode one frame while filtering another). On a GPU, task parallelism is coarse and limited: a GPU has few independent "task" slots, but they correspond to *streams* (Chapter 6).
- **Data parallelism** - the *same* function runs on many data elements. This is the natural mode of the GPU: one kernel, millions of elements.
- **Pipeline parallelism** - a computation is split into stages; each stage processes a different element simultaneously. The classic example is a convolution pipeline: stage one loads, stage two computes, stage three stores. On a GPU, pipeline parallelism appears both at the hardware level (the memory pipeline, the instruction pipeline) and at the application level (double buffering, Chapter 6).

A GPU is a **data-parallel** machine. When you read "massively parallel", the word "parallel" means "data parallel". Task parallelism on a GPU is an afterthought; data parallelism is the design centre.

1.7 Flynn's Taxonomy: SISD, SIMD, SIMT, MIMD

Michael Flynn's 1966 taxonomy classifies computers by whether they operate on one or many *instruction streams* and one or many *data streams*. The four combinations are:

- **SISD** (single instruction, single data) - a conventional scalar CPU core. One instruction stream, one data stream. Your laptop's cores, in scalar mode.
- **SIMD** (single instruction, multiple data) - one instruction operates on a *vector* of data elements simultaneously. Examples: SSE and AVX on x86 CPUs. The programmer (or compiler) explicitly packs data into wide registers; a 256-bit AVX register holds eight 32-bit floats, and one instruction adds all eight at once.
- **MIMD** (multiple instruction, multiple data) - each processing unit runs its own instruction stream on its own data. Examples: multi-core CPUs, GPU streaming multiprocessors as a whole.
- **SIMT** (single instruction, multiple threads) - NVIDIA's execution model, a hybrid of SIMD and MIMD. The hardware fetches *one* instruction per cycle for a *group* of threads (a **warp**, defined in Chapter 2), but each thread has its own registers, its own program counter, and its own data. This combination - one instruction, many independent thread contexts - is the single most important architectural idea in this book.

Why SIMT is not SIMD. In SIMD, the data elements are explicitly packed into a vector register, and divergence is impossible: all lanes execute the same instruction, always. In SIMT, threads *appear* to execute independently; the hardware executes them in lockstep *when their control flow agrees*. If threads in the same warp take different branches, the hardware serialises the branches (Chapter 5). SIMT gives you the programming convenience of MIMD (each thread can follow its own data-dependent path) with the cost of SIMD when paths diverge.

1.8 Arithmetic Intensity and the Roofline Model

The roofline model, introduced by Williams, Waterman and Patterson in 2009, is the most useful performance model in this book. It answers one question: *for a given computation, is the limit set by the arithmetic units or by the memory system?*

1.8.1 Why "per byte"? The question the ratio answers

Before the formula, the intuition - because the formula is only confusing until you can *feel* the ratio.

A GPU has two completely different kinds of resources:

- **The arithmetic units** (FP32 cores): they can do work at a fixed maximum rate, P_{peak} FLOP/s. They are the *workers*.
- **The memory system** (DRAM, L2, the bus): it can deliver data at a fixed maximum rate, B bytes/s. It is the *supply line*.

Here is the catch that defines everything: **the workers cannot work on data they do not have**. Before the FP32 cores can add two numbers, those two numbers must physically travel from DRAM across the bus and into the chip. That travel is not free - it consumes memory bandwidth, and bandwidth is a finite, per-second budget.

So imagine each kernel as a *transaction*: it moves some number of bytes out of memory, and for each byte it does some number of FLOPs. The ratio

$$I = \frac{\text{FLOPs}}{\text{Bytes}}$$

is a *productivity measure*: **how much work do you get out of each byte of data you bother to ship?** It is exactly like fuel efficiency - miles per gallon. “Arithmetic intensity” is **work per byte**: FLOPs per byte moved.

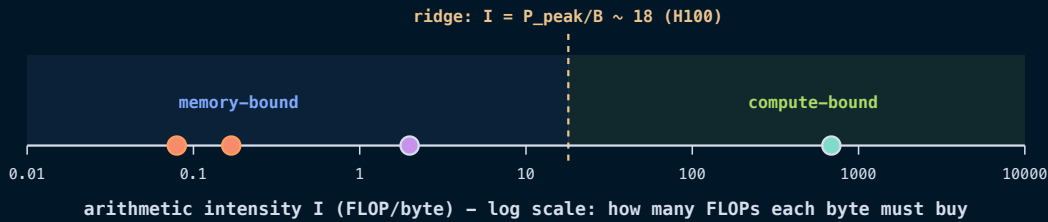
Why is this ratio the single most important number in GPU programming? Because it decides *which one of the two resources runs out first*:

- A kernel with **low intensity** (few FLOPs per byte) uses up the memory system’s byte budget long before the arithmetic units are tired. The arithmetic units then sit idle, waiting for the next byte to arrive. Such a kernel is **memory-bound** - no amount of extra arithmetic horsepower helps, because the bottleneck is the supply line.
- A kernel with **high intensity** (many FLOPs per byte) makes the arithmetic units the bottleneck instead: the supply line could easily deliver more data, but the workers cannot chew through it fast enough. Such a kernel is **compute-bound** - extra bandwidth is wasted, because the bottleneck is the workers.

What raises a kernel’s intensity? **Reusing data**. If a byte is loaded once and used for many operations, it “pays for itself” many times over. If it is loaded, used once, and discarded, it is expensive fuel.

ARITHMETIC INTENSITY: HOW MUCH WORK PER BYTE

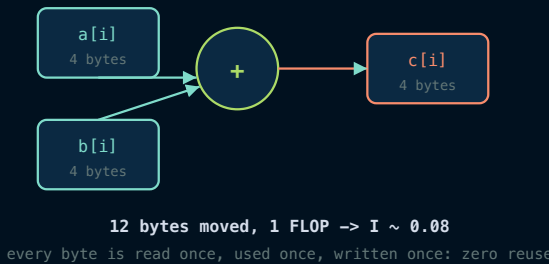
$I = \text{FLOPs} / \text{Bytes moved from memory}$ – a kernel's "fuel efficiency": the work each byte must pay for



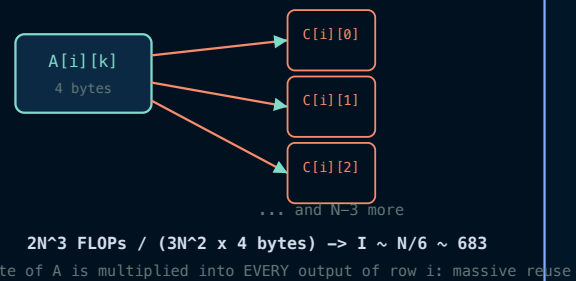
four kernels on the same machine – the only thing that changes is how many times each byte is reused:

- vector add: $I = 0.08$ FLOP/byte – 1 add for every 12 bytes (read $a[i]$, read $b[i]$, write $c[i]$)
- SAXPY: $I = 0.17$ – 2 ops (multiply + add) for the same 12 bytes – still memory-bound
- FFT (typical): $I \sim 2$ – each datum is reused across several butterfly stages
- SGEMM: $I \sim 683$ – every byte of A and B is reused $\sim N/6$ times ($N = 4096$) – compute-bound

LOW INTENSITY: EACH BYTE WORKS ONCE



HIGH INTENSITY: EACH BYTE WORKS $\sim N/6$ TIMES



Read the diagram from left to right: every kernel on the same machine moves the same kinds of bytes, but gets wildly different amounts of work out of them. A vector add ships 12 bytes (two reads, one write) to earn a single FLOP - intensity 0.08, deep in memory-bound territory. A dense matrix multiply reuses each loaded byte for hundreds of operations - intensity ~ 683 , deep in compute-bound territory. *Nothing about the machine changed; only the reuse.* This is why Chapter 9's matrix multiply is the book's crowning optimisation: it is the art of raising intensity.

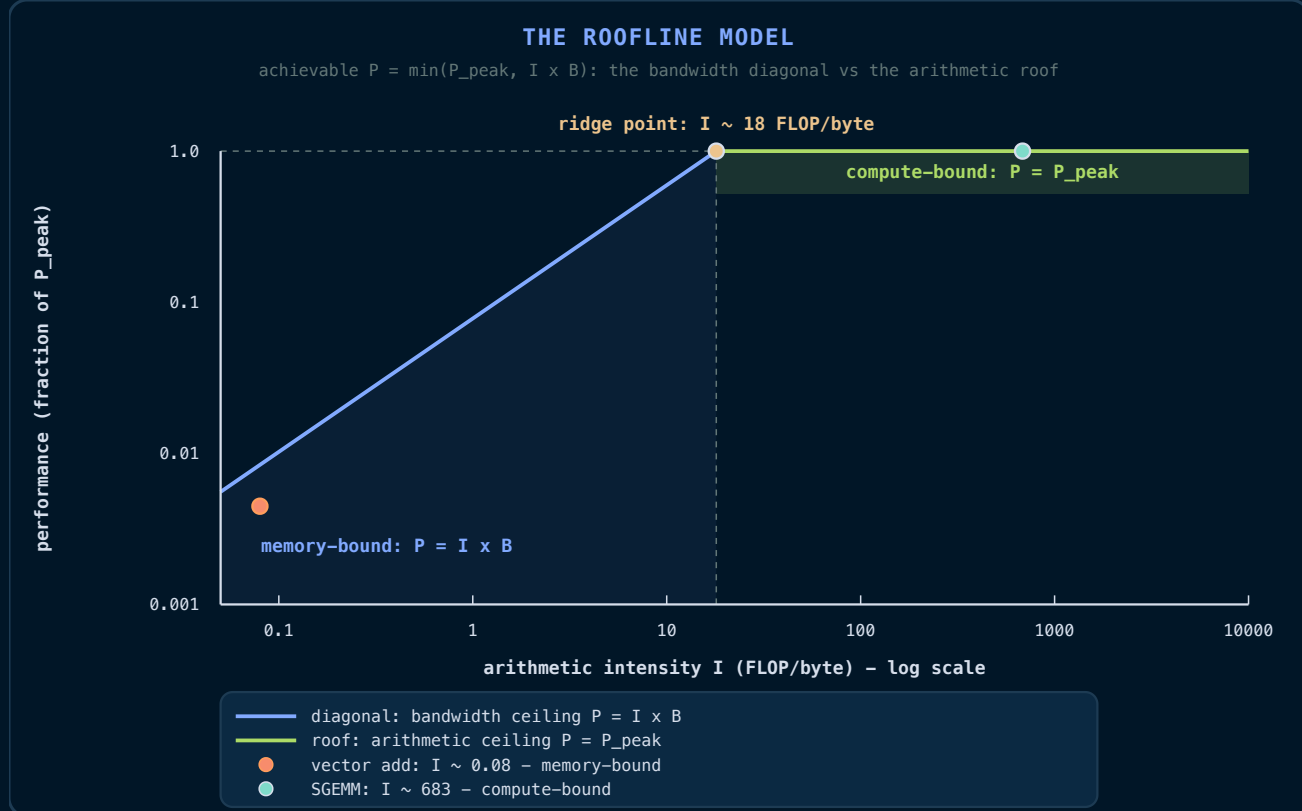
1.8.2 The ridge point: where the two limits meet

Let P_{peak} be the machine's peak floating-point throughput (FLOP/s) and B its peak memory bandwidth (bytes/s). If a kernel has intensity I , then while the memory system delivers bytes, the workers can at most produce:

$$P \leq \min(P_{\text{peak}}, I \cdot B)$$

The two limits meet at the **ridge point** - the intensity at which the supply line and the workers are exactly balanced:

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B}$$



In words: if your intensity is below the ridge point, you are **memory-bound** and performance is capped by bandwidth ($I \cdot B$); if above, you are **compute-bound** and capped by peak FLOP rate. The diagram above *is* the model: the diagonal is the bandwidth ceiling ($P = I \cdot B$), the roof is the arithmetic ceiling (P_{peak}), and the ridge point is where the two meet. Every kernel in this book is a dot on this picture; its distance from the ridge tells you which resource to optimise.

The ridge point as a break-even efficiency. You can read I_{ridge} as: “the minimum work-per-byte a kernel must achieve on this machine, or the workers will starve.” It converts the machine’s two raw specs into a single number you can compare any kernel against - which is why every hardware chapter in this book quotes it (e.g., §2.1).

An intuition: the factory and the freight line. Think of the machine as a factory (the FP32 cores, capable of P_{peak} units of work per second) supplied by a freight line (the memory bus, capable of B bytes per second). Every byte that arrives buys you I units of work. If the freight line delivers less work per second than the factory can consume, the factory idles between deliveries - that is *memory-bound*, and no amount of factory (arithmetic) tuning helps. If the freight line delivers more than the factory can consume, the line backs up - that is *compute-bound*, and buying more bandwidth is wasted money. The ridge point is the intensity at which both are exactly busy: the only intensity where adding either resource pays off.

Worked numbers. An RTX-class GPU with $P_{\text{peak}} = 40$ TFLOP/s of FP32 and $B = 1$ TB/s has a ridge point of $I_{\text{ridge}} = 40$ FLOP/byte. Now compute the intensity of a vector add, carefully - this is the calculation that explains the entire field of GPU memory optimisation. Each output element $c[i] = a[i] + b[i]$ does exactly **1 FLOP** (one addition), but it must first **read two 4-byte floats and write one 4-byte float** - 12 bytes moved:

$$I = \frac{1 \text{ FLOP}}{(2 \text{ reads} + 1 \text{ write}) \times 4 \text{ bytes}} = \frac{1}{12} \approx 0.08 \text{ FLOP/byte}$$

That is 500× below the ridge point - deep in memory-bound territory. No amount of arithmetic optimisation will make a vector add faster; only bandwidth optimisation will (coalesced accesses, §2.7; avoiding redundant reads, Chapter 7). This single observation explains why Chapter 7 is devoted to memory: for most real kernels, *the bytes are the problem, not the arithmetic*.

1.9 The Cost of Synchronisation

Parallel work must occasionally rendezvous: threads must agree on an order, or share a partial result. The primitive operations are introduced in Chapter 5, but the *economics* belong here.

Three costs attend any synchronisation point:

1. **Idle time.** While threads wait at a barrier, their execution units do nothing. The barrier converts available parallelism into a serial stall.
2. **Memory visibility cost.** For one thread to see another thread's write, the write must be flushed and made visible (caches must be coherent or bypassed). On a GPU this is not free.
3. **Load imbalance.** A barrier is only as fast as the *slowest* thread reaching it. If one thread does ten times the work of its neighbours, every barrier in the loop pays for the straggler.

The engineering corollary is: **minimise the number of synchronisation points, and make the work between them as balanced as possible.** The optimised reduction in Chapter 8 exists almost entirely to reduce the number of block-level barriers from $\log_2 N$ to a constant.

1.10 A Vocabulary Summary

The terms defined in this chapter are the book's working vocabulary:

Term	Definition	First used in anger
Latency	Time from start to completion of one operation	§1.1
Throughput	Operations completed per unit time	§1.1
Speedup $S(p)$	T_1/T_p	§1.2
Efficiency $E(p)$	$S(p)/p$	§1.2
Serial fraction f	The non-parallelisable fraction	§1.3
Strong scaling	Fixed problem, more units	§1.5
Weak scaling	Fixed per-unit problem, more units	§1.5
SIMT	Single instruction, multiple threads	§1.7
Arithmetic intensity I	FLOPs per byte moved	§1.8
Ridge point	P_{peak}/B	§1.8
Memory-bound	$I < I_{\text{ridge}}$, limited by bandwidth	§1.8
Compute-bound	$I > I_{\text{ridge}}$, limited by FLOPs	§1.8

Key Takeaways

- Parallelism buys throughput, not latency; the GPU hides latency by keeping many warps in flight.
- Speedup $S(p) = T_1 / T_p$; efficiency $S(p) / p$ is the honest metric.
- Amdahl's law: a serial fraction f caps speedup at $1/f$, no matter how many units you add.
- Gustafson-Barsis: when the problem grows with the hardware, scaled speedup grows linearly.
- Arithmetic intensity $I = \text{FLOPs} / \text{bytes}$, compared with the ridge point P_{peak} / B , decides memory-bound vs compute-bound.
- SIMT executes one instruction per warp; divergent control flow serialises the divergent paths.

1.11 Exercises

1. A kernel has a serial fraction of $f = 0.02$. What is the maximum speedup Amdahl's law permits, regardless of hardware?
2. A vector add moves 4 bytes per element read and 4 bytes per element written, and performs 1 FLOP per element. Compute its arithmetic intensity.
3. On a machine with a ridge point of 40 FLOP/byte, is the vector add memory-bound or compute-bound? What is the maximum utilisation of peak FLOPs achievable?
4. Explain, in your own words, why a GPU designed for throughput would willingly execute a warp of threads in lockstep even though each thread has its own program counter.

Chapter 2: GPU Hardware from First Principles

“The programming model is a fiction that the hardware honours. To write fast kernels you must know where the fiction ends.”

CUDA’s programming model presents a pleasant abstraction: a grid of blocks, a block of threads, a hierarchy of memories. The hardware underneath is messier, and the mess is exactly where performance lives. This chapter strips the programming model away and describes the machine: the *streaming multiprocessor*, the *warp*, the *memory hierarchy*, and the rules of *coalescing* and *occupancy*. Every term defined here is used in every later chapter.

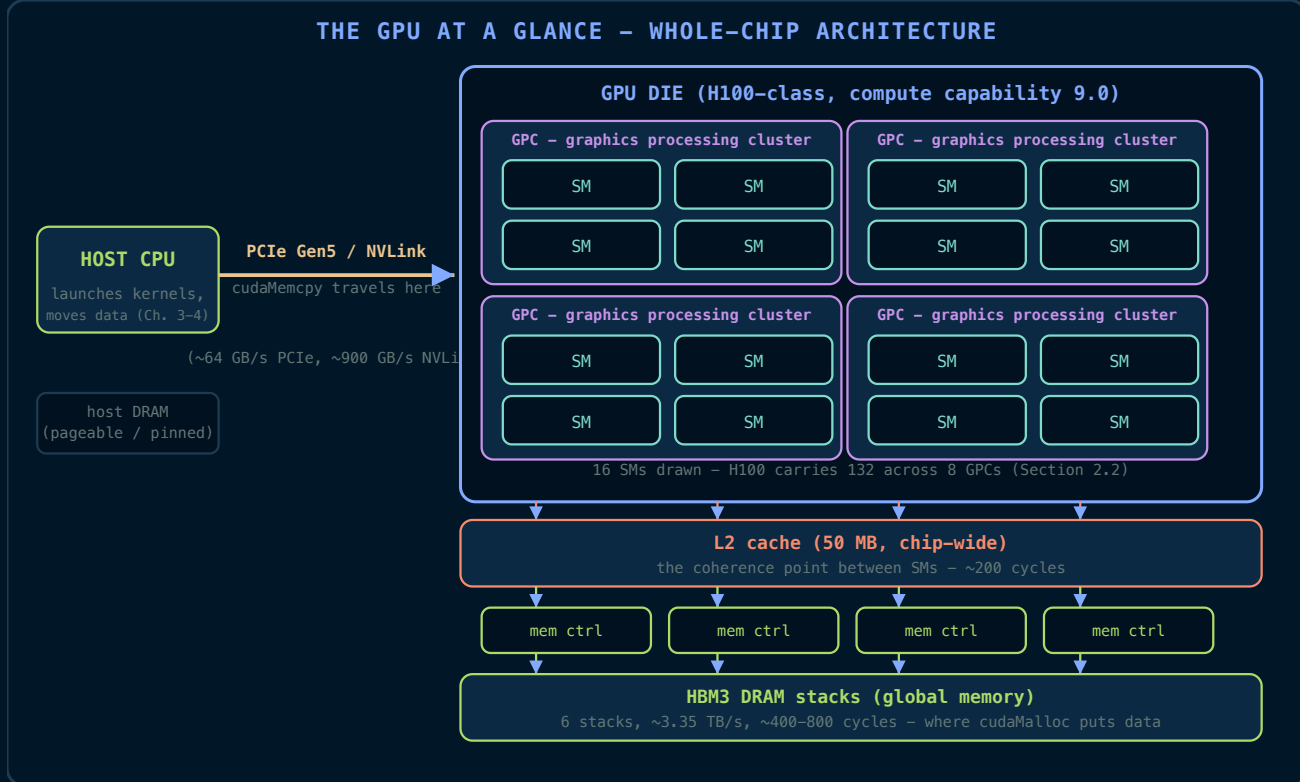
We take as our running example a modern NVIDIA GPU of the Hopper family (compute capability 9.0, such as the H100). Numbers differ between generations, but the structure does not.

2.1 The GPU at a Glance

A GPU is a collection of identical compute clusters plus a memory system. The H100, for example, has:

- **132 streaming multiprocessors (SMs)** - the compute clusters;
- **128 FP32 cores per SM** - the arithmetic units;
- **64 KB to 228 KB of shared memory per SM**, configurable;
- **64 K registers per SM**, partitioned among the threads;
- **50 MB of L2 cache**, shared by all SMs;
- **HBM3 DRAM** with a bandwidth of roughly **3.35 TB/s**.

THE GPU AT A GLANCE – WHOLE-CHIP ARCHITECTURE



Read the diagram as the whole machine, top to bottom: the host CPU lives across a bus; the die is organised into *graphics processing clusters* (GPCs), each holding several SMs; every SM funnels into one chip-wide L2; L2 feeds the memory controllers; the controllers drive the HBM3 stacks. This is the physical map that every later chapter's reasoning walks along.

Why these numbers? The headline figures are design consequences, not arbitrary specifications:

- **Why so many SMs?** A GPU is a throughput machine (Chapter 1, §1.1). Chip area is spent on many small compute clusters rather than a few large cores, because parallelism - not single-thread speed - is the product. 132 SMs is what fits when each SM is deliberately small.
- **Why 128 FP32 cores per SM?** Each SM has four warp schedulers (§2.2), and a scheduler issues one *warp* instruction per clock - 32 lanes at once. $128 = 4 \times 32$: one full warp per scheduler per clock, with no lane sharing. The number is dictated by the warp, the fundamental unit of execution.
- **Why 64 K registers?** Registers are the SM's working storage for resident warps. A deeper register file holds more warps, which hides more latency (§2.9) - at the price of chip area and clock speed. 64 K is the engineered balance.
- **Why is DRAM so fast yet so far?** HBM3 stacks memory vertically beside the die on a silicon interposer, with thousands of narrow channels - that is where 3.35 TB/s comes from. But every access still leaves the chip, which is why latency stays in the hundreds of cycles (§2.6), and why the on-chip memory hierarchy exists at all.

The arithmetic rate of such a chip is on the order of 60-70 TFLOP/s in FP32. The memory bandwidth is 3.35 TB/s. Applying the roofline formula from Chapter 1:

$$I_{\text{ridge}} = \frac{60 \times 10^{12} \text{ FLOP/s}}{3.35 \times 10^{12} \text{ B/s}} \approx 18 \text{ FLOP/byte}$$

Any kernel below roughly 18 FLOP/byte is memory-bound on this machine. Keep this number in your pocket; it will explain most of the optimisation chapters.

2.2 The Streaming Multiprocessor (SM)

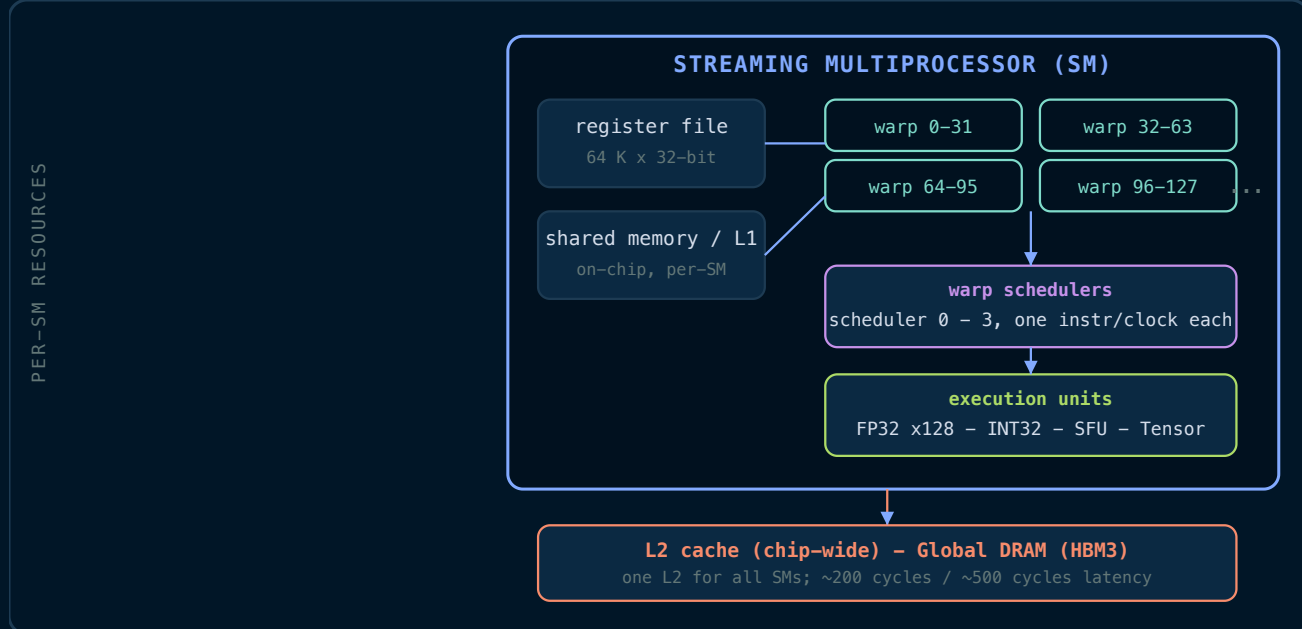
The SM is the GPU's unit of compute. It is best thought of as a **small, heavily multithreaded processor** - closer to a 128-lane vector machine than to a CPU core.

Each SM contains:

- **FP32 cores** (also called CUDA cores): single-precision floating-point units, one FMA (fused multiply-add) per core per clock. An FMA computes $a \cdot b + c$ in one instruction, so counting it as *two* FLOPs is why peak FLOP rates look so large.
- **INT32 cores**: integer units, which in modern architectures share the dispatch but have their own register ports.
- **Tensor cores**: specialised matrix-multiply units for AI workloads. They are a separate pipeline; we note them here and return in Chapter 11.
- **Special function units (SFUs)**: fast approximate transcendental functions (`sin`, `cos`, `exp`, `log`, `1/x`, `rsqrt`). Each SFU serves the whole warp, one result per clock per unit.
- **A register file** of 64 K 32-bit registers.
- **A shared memory / L1 cache** unit.
- **Four warp schedulers** (on modern SMs), each able to issue one instruction per clock to a warp.

The important consequence: **the SM is not a multicore CPU**. It does not have one instruction stream per core. It has a small number of *warp schedulers*, each feeding instructions to a *warp* of threads. The threads are the data parallelism; the scheduler is the control.

Here is the internal anatomy of one SM, drawn to scale in the sense that matters (everything on the left feeds the four schedulers on the right):



Read the diagram as a data-flow picture: warps live in the register file, share memory and arithmetic units through the schedulers, and reach the rest of the chip through the L1/L2 path. The four schedulers are the control plane; the FP32/INT32/SFU/Tensor units are the data plane; shared memory and registers are the on-chip storage.

2.3 The Warp

Primitive - warp. A warp is a group of **32 consecutive threads** that are scheduled and executed together. The warp is the hardware's unit of execution, exactly as the *thread* is the programmer's unit of logic.

A picture first. Forget definitions for a moment and look at the diagram: one instruction is fetched once and broadcast to 32 lanes, and all 32 lanes execute it in the same clock - but each lane applies it to its *own* registers and its *own* data:

THE WARP: 32 THREADS, ONE INSTRUCTION, LOCKSTEP

the hardware fetches ONE instruction; all 32 lanes (threads) execute it together, each on its OWN data

warp scheduler picks one instruction

LD R4, [R2+8]

one instruction, broadcast to all 32 lanes



one instruction, fetched once - executed by 32 lanes in the SAME clock, each on its OWN data
this is SIMT (Chapter 1, 1.7): the programming convenience of many threads, the cost of SIMD
if lanes take different branches, the paths run one after another - that is divergence (Chapter 5)

That is the entire idea of a warp. The scheduler does not manage 32 threads as 32 separate things; it manages them as *one row of 32 seats*. When it issues an instruction, every occupied seat executes it simultaneously, on whatever that seat's thread happens to be holding.

Why 32? The number is an architectural constant of every NVIDIA GPU to date. It is a deliberate engineering balance, not a magic value:

- **It amortises instruction cost.** Fetching and decoding an instruction costs the same whether it serves one thread or thirty-two. A wider warp means the fixed cost of each instruction is spread over more useful work.
- **It is a power of two.** Warp boundaries fall at 32, 64, 96, ... which makes thread-to-warp arithmetic (integer division and modulo by 32) free on the hardware.
- **It matches the memory system's granularity.** 32 threads $\times$ 4 bytes = 128 bytes - exactly one cache line (§2.7). A warp of consecutive threads can be satisfied by one memory transaction. This is not a coincidence; it is how coalescing became cheap.

The cost of a large warp is that divergence (§ below) is coarser: one thread taking a different branch forces the whole warp to pay. Thirty-two balances instruction amortisation against divergence waste, and every generation has kept it.

How a warp executes - and what "lockstep" means. The warp scheduler picks an instruction for the warp; the instruction is fetched once and issued to all 32 lanes at the same time. Each lane (thread) has its **own registers**, so each lane can hold different data - but all lanes execute the *same* instruction at the *same* time. This is SIMT (Chapter 1, §1.7), and the difference from a CPU is the whole game: a CPU runs one instruction stream per core; a GPU runs one instruction stream per *32 threads*.

Consequence: divergence. If two threads in a warp take different branches of an `if`, the hardware cannot execute both paths simultaneously. It executes the `then` path with the other lanes masked off, then the `else` path, then reconverges. The two paths run *serially*, each using the full warp's instruction slots. A 50/50 branch costs double. Chapter 5 returns to this.

Consequence: one instruction, many data. Because all 32 lanes share one instruction, a single memory load instruction issued to a warp is, in fact, 32 loads. How those 32 loads are serviced by the memory system is the subject of §2.7 (coalescing).

The empty-seat footnote. When you launch a kernel with 1,000 threads, the hardware creates 32 warps: 31 full warps ($32 \times 31 = 992$ threads) plus one partial warp of 8 threads. The remaining 24 lanes of that last warp are disabled but still occupy scheduling slots - dead weight that costs occupancy (§2.9) without doing work. This is why real kernels are launched with block sizes that are multiples of 32 (Chapter 3).

2.4 Blocks, Grids, and the Hardware's View

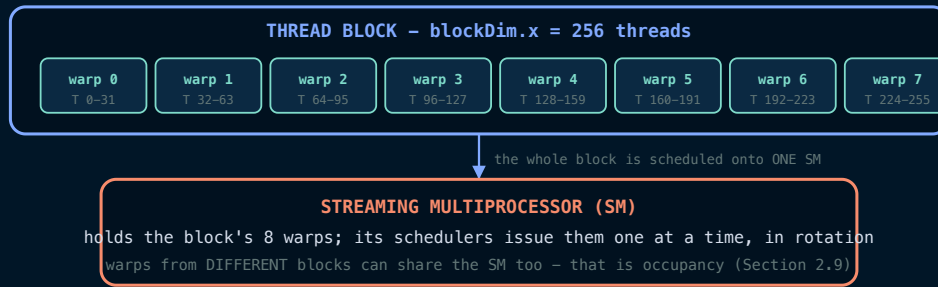
CUDA's programming model (Chapter 3) organises threads as: a **grid of thread blocks**, each block a group of **threads**. The hardware maps this hierarchy as follows:

- A **thread block** is scheduled onto **one SM**, as a unit. All threads of a block run on the same SM, which is what makes block-level shared memory and `__syncthreads()` possible.
- A block is partitioned into **warps** by consecutive thread IDs. Threads 0-31 form warp 0, threads 32-63 form warp 1, and so on. For a 2-D block, the threads are linearised in x-major order (x varies fastest).
- The SM runs **many blocks concurrently**, time-slicing its warps. How many depends on occupancy (§2.9).

Why two levels? An analogy: rooms and rows. Think of a block as a *room* and a warp as a *row of seats* inside it. The programmer says: "here is a room of 256 people who must be able to talk to each other." The hardware answers: "I cannot track 256 individuals cheaply, so I will seat them in 8 rows of 32 and march each row as one unit." The room (block) is the *unit of cooperation* - everyone in it can share memory and synchronise. The row (warp) is the *unit of execution* - the hardware only ever moves whole rows at a time.

A BLOCK OF 256 THREADS = 8 WARPS OF 32

the block is the programmer's unit of cooperation; the hardware slices it into warps – its own unit of execution



why this two-level design? Because the two levels answer different questions:

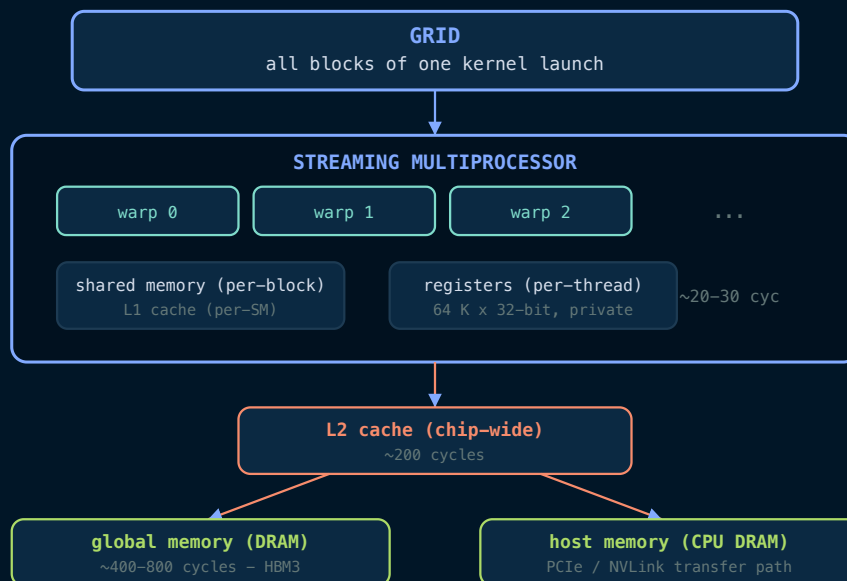
- The BLOCK (programmer's level): threads that share shared memory and `__syncthreads()` must be co-located on one SM, so the hardware places a whole block on one SM – it cannot be split.
- The WARP (hardware's level): the SM does not track 256 threads separately – it tracks 8 warps. One scheduler instruction covers 32 threads at once, which is far cheaper to manage.
- Threads are numbered x-fastest: in a 2-D block (`blockDim.x`, `blockDim.y`), thread (`x`, `y`) has linear ID `y * blockDim.x + x`, so warp 0 is always the first 32 threads in x-major order.
(The warp is the hardware's unit of execution; never confuse the two levels. A block of 32 threads = exactly 1 warp.)

The block is the programmer's unit of *cooperation*; the warp is the hardware's unit of *execution*. Never confuse the two levels:

- You, the programmer, choose the **block** size (Chapter 3's `blockDim`) - and you choose it in multiples of 32 so that no warp is partially empty.
- The hardware, invisibly, slices your blocks into **warps** - you never create a warp, and you rarely address one directly. It exists purely so the SM can schedule 32 threads with the cost of one.

2.5 The Memory Hierarchy

The GPU memory hierarchy is a hierarchy of *distance and size*:



From top to bottom, each level is larger and slower:

1. Registers. Private to a single thread; 32-bit wide; up to 255 per thread. There is no address for a register - it is named by the instruction (`R0` , `R1` , ...). Access is free, but there are only 64 K per SM, shared by all threads. Register pressure directly limits occupancy (§2.9).

2. Shared memory. Private to a *block*; on-chip; configurable as part of the SM's 228 KB (H100) unified L1/shared resource. Access latency is ~20-30 cycles, versus ~400+ cycles for global memory. Shared memory is the programmer's explicitly managed cache - the workhorse of Chapter 7.

3. L1 cache. On-chip, per-SM, unified with shared memory. Global loads that hit L1 avoid the trip to DRAM. L1 lines are 128 bytes.

4. L2 cache. On-chip, shared by *all* SMs, 50 MB on H100. It caches global, constant, and texture accesses. L2 is the coherence point between SMs: two blocks on different SMs communicate through L2 (or explicitly through atomics, Chapter 5).

5. Global memory. The GPU's DRAM (HBM3), the largest and slowest level. This is where `cudaMalloc` puts data (Chapter 4). Bandwidth is enormous (3.35 TB/s), latency is enormous (hundreds of cycles). The entire optimisation enterprise is, mostly, keeping global traffic low.

6. Constant and texture memory. Two specialised read-only paths. Constant memory is a small (64 KB) cache that broadcasts a single value to all threads in a warp *for free* when they read the same address - ideal for kernel parameters. Texture memory is a cached read-only path with hardware support for 2-D spatial locality and interpolation - used for images. Both are discussed in Chapter 7.

7. Local memory. A misnomer: "local" memory is actually global memory allocated per-thread, used when a thread's register demand exceeds the register file (a *register spill*). Local memory is slow; spills are to be avoided. The compiler reports spills with `--ptxas-options=-v` .

2.6 The Latency Table

The numbers below are typical orders of magnitude for a modern GPU; treat them as teaching figures, not datasheet values:

Resource	Approximate latency	Notes
Register	~0 cycles	Operand to instruction
Shared memory	~20-30 cycles	On-chip, banked
L1 hit	~30 cycles	Per-SM
L2 hit	~200 cycles	Chip-wide
Global DRAM	~400-800 cycles	HBM3
Host memory (PCIe)	~1,000+ cycles + transfer time	Off-chip, CPU side

The table is a map of *physical distance*, not a marketing sheet. Registers sit on the SM, a few millimetres from the arithmetic units; shared memory and L1 are on the same die; L2 spans the whole chip; DRAM is a separate package beside the die on an interposer; host memory is across a bus and an OS boundary. Every step off the SM adds distance *and* arbitration - more circuits competing for the same wires. The 20× gap between shared memory and DRAM is not a tuning detail; it is the difference between an on-chip wire and an off-chip trip, and it is the entire reason the optimisation chapters exist.

The lesson: **one global memory access costs roughly 30 shared-memory accesses**. Any algorithm that can restructure itself to reuse data in shared memory is buying speed with engineering effort - and Chapter 7 will show the accounting in detail.

2.7 Coalescing: How a Warp Reads Memory

Here is the single most consequential performance rule in CUDA programming:

Primitive - coalescing. When the threads of a warp issue a load, the memory system groups their requests and services them in **128-byte cache lines** (or 32-byte sectors) - *if* the addresses are contiguous. If the 32 threads read 32 consecutive 4-byte words, one or two 128-byte transactions satisfy the entire warp. If the threads read a stride-32 pattern, the same data is fetched in 32 separate transactions.

Concretely: a warp of 32 threads loading `float` values at consecutive addresses accesses $32 \times 4 = 128$ bytes. The memory system fetches exactly one 128-byte line. A warp loading the same 128 bytes in a scrambled order may trigger several times that traffic. The hardware cannot detect the access pattern in general; it only knows which sectors the warp touched.

Why coalescing matters. Global memory bandwidth is the scarcest resource on a memory-bound kernel (Chapter 1's roofline). Coalescing is the difference between using 100% of that bandwidth and using 6% of it. A stride of 32 floats between consecutive threads wastes 31/32 of every fetched byte.

The rule of thumb. Arrange your data so that **consecutive threads access consecutive addresses**. This single sentence explains the layout choices made throughout this book: the row-major matrix layout in Chapter 9, the thread-to-pixel mapping in the capstone (Chapter 15), and the padded shared-memory arrays of Chapter 7.

2.8 Shared Memory Banks

Shared memory is fast because it is **banked**: it is physically organised into 32 banks, each 4 bytes wide, that can be accessed *simultaneously*. The address of a shared-memory word maps to a bank by:

$$\text{bank} = \left\lfloor \frac{\text{address in bytes}}{4} \right\rfloor \bmod 32$$

When a warp accesses shared memory, the hardware services **one access per bank per cycle**. If two threads in the warp hit the same bank, the hardware serialises them: that is a **bank conflict**, and it costs extra cycles.

- Threads 0-31 reading consecutive words: all 32 banks busy, one access, no conflict.
- Threads 0-31 reading a stride of 32 words: all 32 threads hit bank 0, thirty-two-way conflict - 32 cycles of pain.
- Threads 0-31 reading the *same* word (a broadcast): hardware broadcasts, one access, no conflict.

Bank conflicts are a shared-memory phenomenon (Chapter 7 shows the classic fix: padding). Global memory has no banks; it has lines and sectors.

2.9 Occupancy

Primitive - occupancy. The ratio of active warps on an SM to the maximum number of warps the SM can hold. An SM with 64 warp slots at 100% occupancy has 64 warps resident.

Why does occupancy matter? **Latency hiding** (Chapter 1, §1.1). When a warp stalls on a global load (~500 cycles), the scheduler switches to another resident warp. If occupancy is high, there is always another warp to switch to. If it is low, the SM idles.

The limits on occupancy are the SM's finite resources:

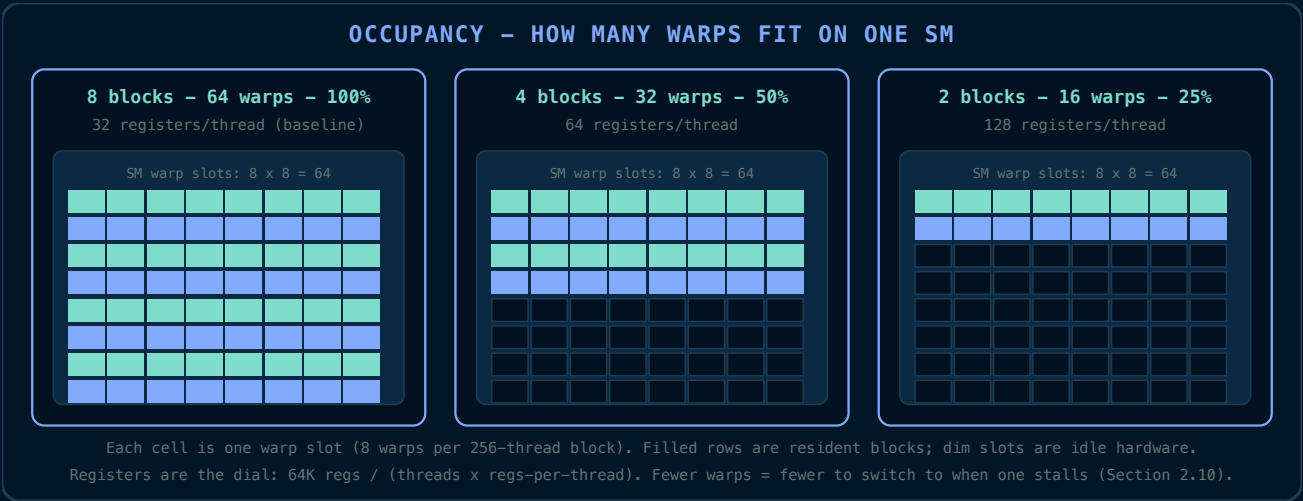
- **Registers:** 64 K per SM. If each thread uses 32 registers, the SM can host 2,048 threads (64 K / 32). If each uses 128 registers, only 512 threads.
- **Threads per SM:** a hardware maximum (2,048 on most modern SMs).
- **Threads per block and blocks per SM:** limits of 1,024 threads per block and 32 blocks per SM (both architecture-specific).
- **Shared memory:** 228 KB per SM (H100); a block declaring 100 KB of shared memory leaves room for only two such blocks.

The occupancy of a given launch configuration is the *minimum* over all these limits. The famous **occupancy calculator** spreadsheet (and `cudaOccupancyMaxActiveBlocksPerMultiprocessor`, Chapter 16) computes it for you.

A worked occupancy calculation. Suppose the SM limits are the ones used throughout this chapter (64 K registers, 2,048 threads per SM, 32 blocks per SM, 228 KB shared memory), and the kernel is launched with blocks of 256 threads (8 warps). We take the minimum of the four constraints:

Constraint	Equation	Blocks allowed
Registers (32/thread)	$64\text{ K} / (256 \times 32)$	8 blocks
Threads per SM	$2,048 / 256$	8 blocks
Blocks per SM	hardware limit	32 blocks
Shared memory (0 bytes used)	no demand on the 228 KB budget	not a constraint

The minimum is **8 blocks**, i.e., 64 warps resident - and since the SM holds at most 64 warps, this is 100% occupancy. Now repeat the register column with a register-heavy kernel using 64 registers per thread: $64\text{ K} / (256 \times 64) = 4\text{ blocks}$ - occupancy drops to 50%. The same kernel with 128 registers per thread: 2 blocks, 25% occupancy. This is why Chapter 9's `__launch_bounds__` matters: *register count is an occupancy dial*.



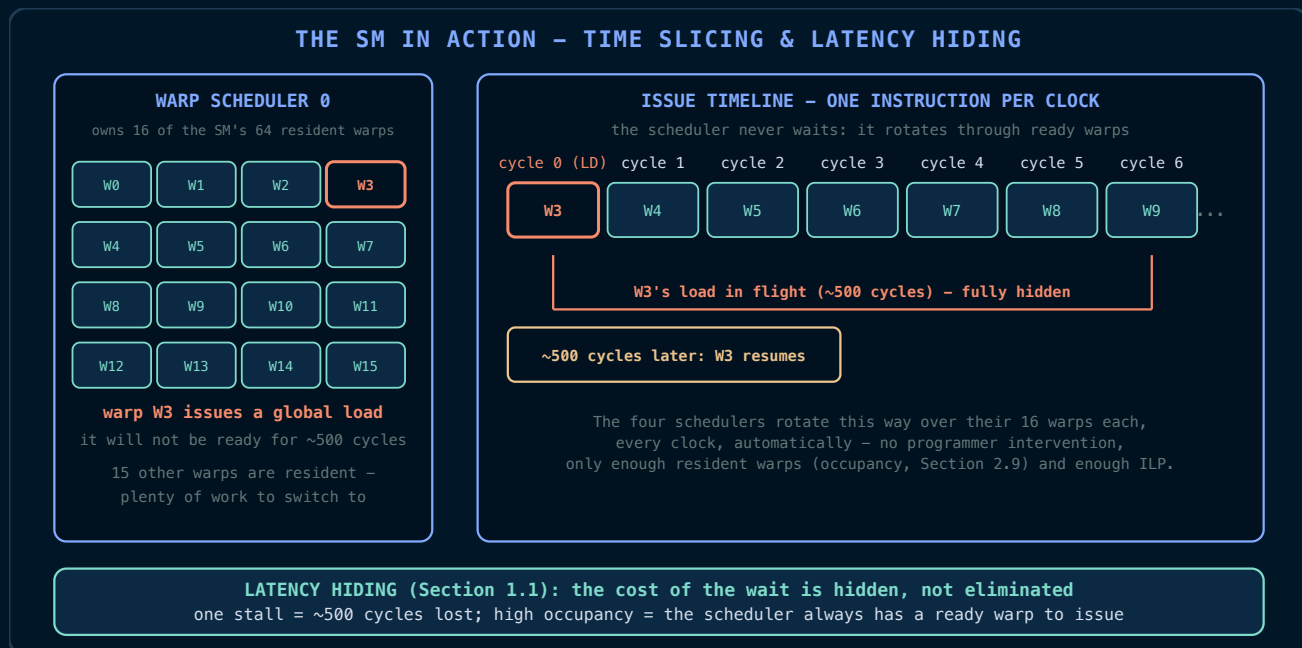
The diagram above draws the same arithmetic as the three columns: each cell is one warp slot, each row is one block's eight warps, and the dim cells are slots the scheduler *could* have switched to but cannot - the register file ran out. The difference between 100% and 25%

occupancy is the difference between always having a ready warp when one stalls and frequently having none. That is why §2.10's time-slicing only works when occupancy is high.

The trade. High occupancy is not always good. A kernel that uses shared memory heavily may want fewer blocks to fit more shared memory per block. A kernel whose working set fits in registers may want low occupancy to avoid spills. Occupancy is a knob, not a goal; Chapter 9 demonstrates tuning it.

2.10 The SM in Action: A Time Slicing Example

Suppose an SM has 64 warp slots and your kernel is configured with blocks of 256 threads (8 warps per block), and the occupancy calculation permits 8 blocks per SM. The SM hosts 8 blocks = 64 warps = 100% occupancy.



At any instant, each of the four warp schedulers owns 16 warps. The scheduler issues an instruction from one of its warps each clock. When warp 3 issues a global load, it will not be ready for ~500 cycles; the scheduler simply issues from warps 4, 5, ... meanwhile - the orange-to-blue handoff in the diagram above. When warp 3's load returns, the scheduler resumes issuing for it.

The elegance is that **no one explicitly scheduled anything**. The hardware rotates among resident warps automatically. Your job as a programmer is to give the hardware enough warps (occupancy) and enough independent work per warp (instruction-level parallelism and coalesced accesses) to keep the rotation from ever stalling.

2.11 Architecture Generations: A Caution

The structure in this chapter is stable across NVIDIA GPUs, but the *numbers* are not. Compute capability (CC) encodes the generation: CC 7.x is Volta, CC 8.x is Ampere, CC 9.0 is Hopper, CC 10.x is Blackwell. Each generation changes SM size, register file size, warp scheduling, tensor core capabilities, and shared memory amounts. When you read a performance claim in this book or anywhere else, the first question to ask is: *on which compute capability?*

You can query your own hardware with `deviceQuery` (a CUDA sample), which reports the SM count, CC, register file, shared memory per SM, and the limits from §2.9. Chapter 16 shows how to read that output.

Key Takeaways

- The warp (32 threads) is the hardware unit of execution, not the thread.
- The whole chip: GPCs of SMs above a chip-wide L2 above HBM3 DRAM, with the host across PCIe/NVLink - a physical map, not an abstraction.
- Blocks map to SMs; warps are consecutive thread IDs within a block.
- Memory hierarchy: registers, shared memory, L1, L2, global DRAM - each level larger and slower (roughly 20-30 cycles for shared, 400-800 for DRAM).
- Coalescing: consecutive threads should read consecutive addresses; the hardware fetches 128-byte lines.
- Shared memory has 32 banks of 4 bytes; a 32-way bank conflict costs 32 cycles - padding fixes it.
- Occupancy is the ratio of resident warps to the SM's maximum; registers, threads and shared memory each cap it.

2.12 Exercises

1. A kernel uses 64 registers per thread. How many threads can one SM host before the register file is exhausted (64 K registers per SM)?
2. The same kernel, now using 128 registers per thread. What is the maximum occupancy, given a hardware limit of 2,048 threads per SM?
3. A warp reads 32 consecutive `int`s (4 bytes each). How many 128-byte cache lines does the hardware fetch? How many would it fetch if the threads read every 32nd `int`?
4. Why must all threads of a *block* be resident on the *same* SM? What shared primitive does this enable that would be impossible otherwise?

Chapter 3: The CUDA Programming Model

"A kernel is a function that the hardware multiplies."  **Code companion:** the complete, buildable code for this chapter lives in `code/ch03_vector_add/` in the repository.

This chapter introduces the CUDA programming model: how a function becomes a kernel, how a launch describes a grid of work, and how data moves between the CPU (the *host*) and the GPU (the *device*). Every concept is introduced from first principles, and the first complete program - a vector addition - is presented with line-by-line commentary.

3.1 Host and Device

CUDA programs are divided into two worlds:

- **Host** - the CPU and its memory. The host *launches* kernels and moves data.
- **Device** - the GPU and its memory (global memory, §2.5). The device *executes* kernels.

The two worlds do not share an address space. A pointer obtained from `cudaMalloc` is a *device* pointer: dereferencing it on the host is undefined behaviour and, in practice, a crash. Data must cross the boundary explicitly with `cudaMemcpy`. This separation is the single most common source of confusion for new CUDA programmers, and it is permanent: Chapter 4 introduces the escape hatches (pinned memory, unified memory), but the separation remains the mental model.

Primitive - host. The CPU side of a CUDA program. **Primitive - device.** The GPU side of a CUDA program. **Primitive - kernel.** A function that runs on the device, launched by the host, executed by many threads.

The physical picture (from Chapter 2). The host is a CPU sitting across a bus; the device is the whole GPU die - GPCs of SMs above a chip-wide L2 above DRAM. Every `cudaMemcpy` is a shipment across that bus; every kernel launch is a work order delivered to the SMs. The two address spaces are separate *because the hardware is physically separate*: different DRAM, different caches, different execution units. The programming model is simply refusing to pretend otherwise - and that refusal is the source of most of the API's apparent ceremony (Chapter 4 explains the escape hatches). Keep the die diagram of §2.1 in mind and none of the rules in this chapter will feel arbitrary.

3.2 The Function Qualifiers

CUDA extends C++ with three function qualifiers:

- `__global__` - the kernel qualifier. The function runs **on the device** and is **called from the host** (or from the device in some later CUDA generations, via cooperative launch). A `__global__` function must return `void`. Its arguments are copied from host memory to the device before launch.
- `__device__` - the function runs on the device and is called *only from device code* (from a kernel, or from another `__device__` function).
- `__host__` - the default: a normal host function. It can be combined as `__host__ __device__` to produce one function compiled for both sides - a workhorse of modern CUDA (Chapter 10).

A `__device__` function cannot call a `__host__` function; the device has no host runtime. A `__global__` function cannot be called recursively (on most architectures) and cannot take a variable number of arguments.

3.3 The Launch Configuration: Grids and Blocks

A kernel launch looks like this:

```
myKernel<<<gridDim, blockDim>>>(args...);
```

The double-angle-bracket expression is the **execution configuration**: it describes the *shape* of the work. Both arguments are of type `dim3` - a three-component vector type with fields `x`, `y`, `z`, each an unsigned integer (`unsigned int`).

- **blockDim** - the number of threads per block, one to three dimensions. Total threads per block = `blockDim.x * blockDim.y * blockDim.z`, and must not exceed 1,024 on modern hardware.
- **gridDim** - the number of blocks in the grid, one to three dimensions. Total threads in the kernel = `gridDim * blockDim` across all dimensions.

Two 3-D vectors describe the whole launch. The key to not getting lost in `<<<grid, blockDim>>>` is to see both arguments as what they literally are: vectors. `gridDim` is a 3-D vector that says *how many blocks exist along each axis*; `blockDim` is a 3-D vector that says *how many threads exist along each axis inside every block*. Together they describe a 3-D grid of 3-D blocks - a box of boxes:

THE LAUNCH HIERARCHY IN 3-D: `gridDim` AND `blockDim` ARE 3-D VECTORS

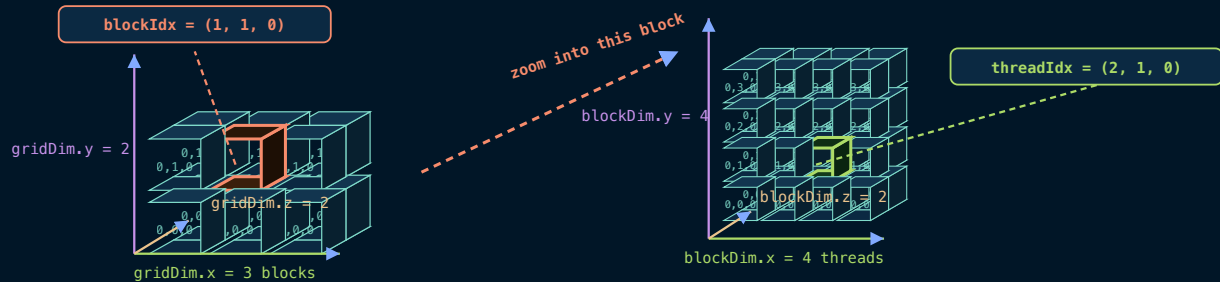
`gridDim.x` = number of blocks per axis | `blockDim` = how many threads per block axis | total threads = `gridDim.x` * `blockDim.x` * `gridDim.y` * `blockDim.y` * `gridDim.z` * `blockDim.z`

THE GRID

a 3-D array of blocks
each cube = one thread block

THE BLOCK

a 3-D array of threads
 $4 \times 4 \times 2 = 32$ threads = one warp



global position = blockIdx * blockDim + threadIdx, per axis (component-wise)

$gx = blockIdx.x * blockDim.x + threadIdx.x$ | $gy = blockIdx.y * blockDim.y + threadIdx.y$ | $gz = blockIdx.z * blockDim.z + threadIdx.z$

example: `blockIdx=(1,1,0)`, `blockDim=(4,4,2)`, `threadIdx=(2,1,0)`

$gx = 1 \times 4 + 2 = 6$ $gy = 1 \times 4 + 1 = 5$ $gz = 0 \times 2 + 0 = 0$

x is the fastest dimension, then y, then z - the hardware linearises threads in that order (Section 3.5)

total threads = $(3 \times 4) \times (2 \times 4) \times (2 \times 2) = 12 \times 8 \times 4 = 384$

Read the diagram in three steps:

1. **The grid** (left) is a 3-D array of blocks. `gridDim = (3, 2, 2)` means 3 blocks along x, 2 along y, 2 along z - 12 blocks. Each little cube is a block, and its coordinates are `blockIdx = (x, y, z)`.
2. **Each block** (right, zoomed in) is itself a 3-D array of threads. `blockDim = (4, 4, 2)` means 4 threads along x, 4 along y, 2 along z - $4 \times 4 \times 2 = 32$ threads, which is exactly one warp (§2.3). Each thread's coordinates inside its block are `threadIdx = (x, y, z)`.
3. **The position of any thread in the whole grid** is the block's position times the block's size, plus the thread's position inside the block - the component-wise formula at the bottom of the diagram:

$$gx = blockIdx.x \times blockDim.x + threadIdx.x$$

$$gy = blockIdx.y \times blockDim.y + threadIdx.y$$

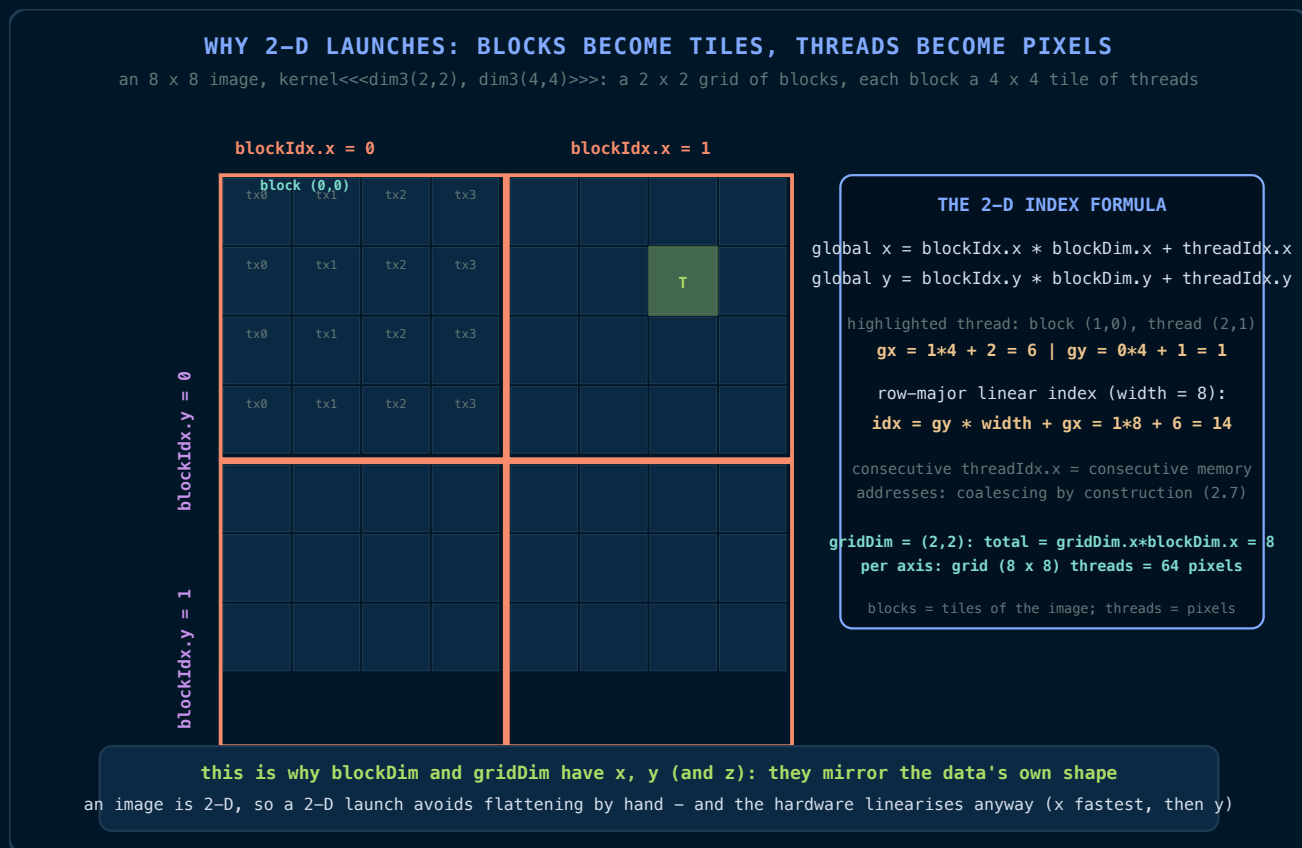
$$gz = blockIdx.z \times blockDim.z + threadIdx.z$$

This is why the 1-D formula `blockIdx.x * blockDim.x + threadIdx.x` of §3.5 is not a special case - it is the x-component of a vector identity that works in all three dimensions. The grid and block sizes *multiply*: the total number of threads is

$$gridDim.x \cdot gridDim.y \cdot gridDim.z \cdot blockDim.x \cdot blockDim.y \cdot blockDim.z$$

Why three dimensions? Because real data is often two- or three-dimensional (images, volumes, grids). A 2-D launch lets the kernel index an image as `(x, y)` instead of flattening it

by hand - the grid and block become *tiles* and *pixels*:

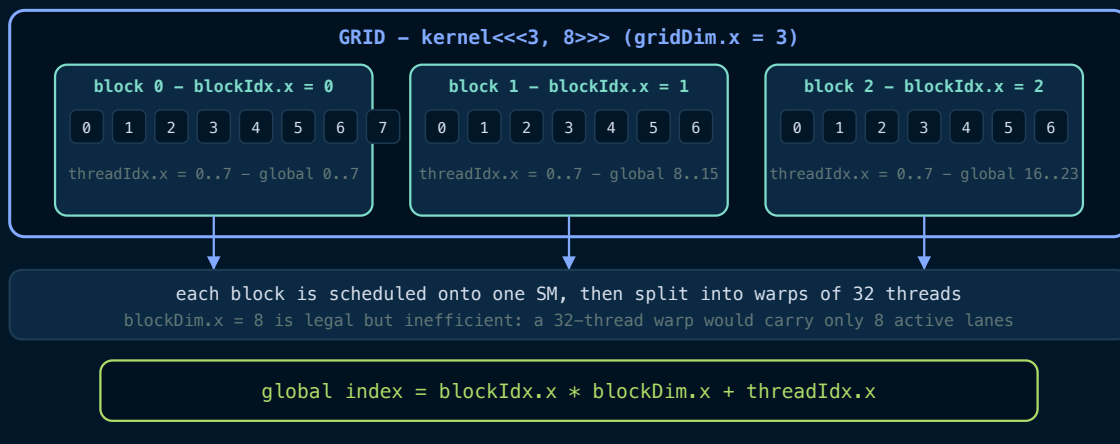


The image example above is the single most useful mental model in this chapter: **blocks tile the data, threads fill each tile**. A 2-D launch means you never flatten coordinates yourself - the hardware linearises anyway (x fastest, then y, then z), but you think in the data's own shape.

The hardware view of this (from Chapter 2): blocks are assigned to SMs; each block is chopped into warps of 32 consecutive threads; warps execute in lockstep.

Read <<<grid, block>>> as a declaration, not a loop. You are not writing a **for** that the machine walks through; you are telling the hardware *how much work exists and how it is shaped*, and the hardware - not you - decides which SM runs which block, and when. The launch is a contract with the machine: enough blocks to fill every SM, blocks small enough to fit the SM's resources (registers, shared memory, the 1,024-thread cap), and a shape that maps your data's natural dimensions. Get the declaration right and the hardware's own scheduler does the rest; get it wrong and the hardware cannot compensate (Chapter 2, §2.9).

Here is the whole hierarchy for a small launch, **kernel<<<3, 8>>>** (3 blocks of 8 threads - smaller than reality, exactly right for a picture):



3.4 The Built-in Variables

Inside a kernel, four read-only built-in variables describe the launch:

Variable	Type	Meaning
<code>threadIdx</code>	<code>dim3</code>	The thread's position <i>within its block</i> : <code>threadIdx.x</code> , <code>.y</code> , <code>.z</code>
<code>blockIdx</code>	<code>dim3</code>	The block's position <i>within the grid</i> : <code>blockIdx.x</code> , <code>.y</code> , <code>.z</code>
<code>blockDim</code>	<code>dim3</code>	Threads per block (the <code>blockDim</code> you passed)
<code>gridDim</code>	<code>dim3</code>	Blocks per grid (the <code>gridDim</code> you passed)

These are **primitives provided by the hardware**, not variables you create. They are how a thread knows *who it is*. In the language of the 3-D picture in §3.3: `blockIdx` is the *address of your block* (which cube of the grid you live in), `threadIdx` is the *address of you inside that cube*, and `blockDim` / `gridDim` are the *sizes of the two containers*. The global formula `blockIdx * blockDim + threadIdx` is the address translator between them.

3.5 The Global Index Formula

The most important one-liner in CUDA, for a 1-D problem:

```
// Global linear index of this thread, assuming a 1-D grid and 1-D blocks.
unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

The reasoning: thread `threadIdx.x` lives in block `blockIdx.x`. Each block contains `blockDim.x` threads, so block number `blockIdx.x` starts at `blockIdx.x * blockDim.x`. Add the position within the block to get the global position. If you saw the 3-D picture in §3.3, this is the same formula with only the x-components left - a 1-D launch is just a 3-D launch where the y and z components are all 1. For a 2-D problem, the formula composes:

```
// 2-D indexing: x and y are independent linear indices in their dimension.
unsigned int ix = blockIdx.x * blockDim.x + threadIdx.x; // column
unsigned int iy = blockIdx.y * blockDim.y + threadIdx.y; // row
// Row-major flattening of a width x height image:
unsigned int idx = iy * width + ix; // linear memory index
```

Note the ordering: in row-major layout, consecutive `ix` values are consecutive in memory - and, by construction, consecutive threads have consecutive `ix`. This is *coalescing by construction* (§2.7), which is why the formula exists.

A worked example, with real numbers. Launch `kernel<<<4, 256>>>` (4 blocks of 256 threads, covering 1,024 global indices). Consider the thread with `blockIdx.x = 2` and `threadIdx.x = 137`:

```
global index = blockIdx.x * blockDim.x + threadIdx.x
              = 2 * 256 + 137
              = 512 + 137
              = 649
```

That thread therefore owns element 649 of the array - no ambiguity, no shared state, and 1,023 other threads own the other 1,023 elements. Now the warp view: `blockIdx.x = 2` covers global indices 512..767. Its warp 0 is threads 0..31, i.e., global indices 512..543: 32 consecutive addresses - coalesced by construction, exactly as §3.5 promised. If the array had only 900 elements (not a multiple of 1,024), the threads owning indices 900..1,023 would be masked by the `if (i < n)` guard in the kernel.

3.6 The First Kernel: Vector Addition

We will now write a complete program: `c = a + b`, element-wise, for `float` arrays of length `n`. This is the “Hello, world” of GPU programming, and every line is worth understanding.

3.6.1 The kernel

```
// kernel.cu
// -----
// __global__ : this function runs on the DEVICE, launched from the host.
// Return type must be void. The argument is a device pointer to n floats.
// -----
__global__ void addVectors(const float* a, const float* b, float* c, int n)
{
    // --- Who am I? -----
    // blockIdx.x : index of my block within the grid (0-based).
    // blockDim.x : number of threads in my block (set at launch).
    // threadIdx.x: index of my thread within my block (0-based).
    // The product blockIdx.x * blockDim.x is the first global thread index
    // covered by my block; adding threadIdx.x gives my global index.
    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // --- Boundary guard -----
    // The grid may cover more threads than n (we launch a rounded-up grid,
    // see the host code). Threads whose index is >= n must do nothing.
    // Without this guard we would read and write past the end of the arrays
    // - an out-of-bounds memory error on the device.
    if (i < n)
    {
        // Element-wise add. Each thread owns exactly one output element,
        // so no two threads ever write the same address: no races here.
        c[i] = a[i] + b[i];
    }
}
```

The comments above are not decoration; they are the *reasoning audit trail* required by this book's coding standards. A reviewer must be able to verify, from the comments alone, that the index arithmetic is correct and that no race is possible.

3.6.2 The host code

```
#include <stdio>
#include <cuda_runtime.h>    // All CUDA runtime API declarations live here.

// -----
// Error-checking helper (see 3.8). Every CUDA call that can fail is routed
// through CHECK, which prints the file and line on failure and aborts.
// -----
#define CHECK(call) \
do { \
    const cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n", \
            __FILE__, __LINE__, cudaGetErrorString(err)); \
        std::exit(EXIT_FAILURE); \
    } \
} while (0)

int main()
{
    // --- Problem size -----
    // n is the number of elements; nBytes is the byte size of each array.
    // We use size_t because array sizes can exceed the range of int.
    const int    n        = 1 << 20;    // 1,048,576 elements (a power of two)
    const size_t nBytes = n * sizeof(float);

    // --- Host allocations (pageable memory, see Chapter 4) -----
    float* h_a = new float[n];    // host input A
    float* h_b = new float[n];    // host input B
    float* h_c = new float[n];    // host output C

    // Fill the inputs with a deterministic pattern so we can verify results.
    for (int i = 0; i < n; ++i) { h_a[i] = 1.0f * i; h_b[i] = 2.0f * i; }

    // --- Device allocations -----
    // cudaMalloc allocates in GLOBAL MEMORY on the device. The returned
    // pointers are valid only on the device (see 3.1).
    float* d_a = nullptr;    // device input A
    float* d_b = nullptr;    // device input B
    float* d_c = nullptr;    // device output C
    CHECK(cudaMalloc((void**)&d_a, nBytes));
    CHECK(cudaMalloc((void**)&d_b, nBytes));
    CHECK(cudaMalloc((void**)&d_c, nBytes));

    // --- Host -> device copy -----
    // cudaMemcpy(dst, src, bytes, kind). The kind cudaMemcpyHostToDevice
    // tells the runtime the direction of the copy (see 3.7).
    CHECK(cudaMemcpy(d_a, h_a, nBytes, cudaMemcpyHostToDevice));
    CHECK(cudaMemcpy(d_b, h_b, nBytes, cudaMemcpyHostToDevice));

    // --- Launch configuration -----
    // Block size: 256 threads per block. Why 256? A multiple of the warp
    // size (32) so every warp is full, and small enough that many blocks
    // fit per SM (see occupancy, 2.9). Values of 128-512 are typical.
    const int threadsPerBlock = 256;
    // Grid size: ceil(n / threadsPerBlock). The + (threadsPerBlock - 1)
```



```

// trick rounds UP so that the grid covers every element. Some threads
// will therefore exceed n and hit the boundary guard in the kernel.
const int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;

// --- Launch -----
// addVectors<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);
// The launch is asynchronous: the host does NOT wait for the kernel;
// control returns to the host immediately (see Chapter 6).
addVectors<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);

// Kernel launches do not report errors synchronously. Check the last
// error now; if the launch itself failed (bad config, bad pointer),
// this catches it.
CHECK(cudaGetLastError());

// --- Synchronise -----
// cudaDeviceSynchronize blocks the host until ALL device work issued
// so far has completed. Required before we copy the results back.
CHECK(cudaDeviceSynchronize());

// --- Device -> host copy -----
CHECK(cudaMemcpy(h_c, d_c, nBytes, cudaMemcpyDeviceToHost));

// --- Verify -----
// We know the correct answer: h_c[i] should equal 3*i. Verify a few
// samples and report the worst error.
double maxErr = 0.0;
for (int i = 0; i < n; ++i)
{
    const double err = std::abs(static_cast<double>(h_c[i]) - 3.0 * i);
    if (err > maxErr) maxErr = err;
}
std::printf("max error = %g\n", maxErr);

// --- Cleanup -----
delete[] h_a; delete[] h_b; delete[] h_c;
CHECK(cudaFree(d_a)); CHECK(cudaFree(d_b)); CHECK(cudaFree(d_c));
return 0;
}

```

3.6.3 Why this design?

- **One thread per element.** The simplest possible decomposition: the work is perfectly partitioned, no thread depends on another, and coalescing is automatic because `i` increases with `threadIdx.x`.
- **Boundary guard instead of exact grid.** Rounding the grid up to a multiple of the block size means the guard `if (i < n)` is required, but it also means we never compute a tricky, non-multiple grid. The guard is one instruction; a mis-sized grid is a crash.
- **Power-of-two sizes.** `n = 1 << 20` is pedagogical; in production the size is arbitrary and the guard earns its keep.

3.7 The Memory API Primitive

The runtime API functions used above are primitives you will use daily:

Function	Behaviour
<code>cudaMalloc(void** p, size_t bytes)</code>	Allocate <code>bytes</code> in device global memory; store the device pointer in <code>*p</code> . Returns <code>cudaSuccess</code> or an error code.
<code>cudaFree(void* p)</code>	Free a device allocation made by <code>cudaMalloc</code> .
<code>cudaMemcpy(dst, src, bytes, kind)</code>	Copy <code>bytes</code> between host and device. <code>kind</code> is one of <code>cudaMemcpyHostToDevice</code> , <code>cudaMemcpyDeviceToHost</code> , <code>cudaMemcpyDeviceToDevice</code> , or <code>cudaMemcpyHostToHost</code> . Synchronous: the copy completes before the call returns.
<code>cudaGetLastError()</code>	Return and clear the last asynchronous error recorded for the calling thread.
<code>cudaGetErrorString(err)</code>	Human-readable text for a <code>cudaError_t</code> .
<code>cudaDeviceSynchronize()</code>	Block the host until all preceding device work completes.

Why `cudaMalloc` takes `void**`? It is C-style output-parameter convention: the function needs to *write a pointer* into your variable, so it takes the address of your pointer variable. C++ would return a pointer; CUDA's C heritage writes through a pointer-to-pointer. And why the explicit `(void**)` cast, which every allocation above carries? Because C++ - unlike C - does not allow an implicit conversion from `float**` to `void**` (only `T*` to `void*` is implicit). The cast is *required* for the code to compile, not optional decoration. This is one of the few places where the API's C heritage leaks into C++ code, and the cast is the price of admission.

Why does `cudaMemcpy` need a direction argument? Because host and device pointers are not distinguishable by address alone (a host pointer and a device pointer can have numerically similar values on some platforms). The direction flag removes the ambiguity.

3.8 Error Handling: The Contract

CUDA functions return a `cudaError_t` - an enum, where `cudaSuccess` is 0 and every other value is an error code. There are two failure modes:

1. **Synchronous errors** - detected immediately by the call (e.g., an invalid argument, an illegal `cudaMemcpy` kind). The call returns the error code.
2. **Asynchronous errors** - detected *after* the call (e.g., an invalid kernel launch, an illegal memory access inside the kernel). The launch itself returns `cudaSuccess`; the error surfaces on the *next* CUDA API call from the same thread, which is why we call `cudaGetLastError()` immediately after the launch.

The `CHECK` macro routes every call through `cudaGetErrorString`, so a failure reports the offending source line. Production code should do something more graceful than `std::exit`, but the *discipline* - check every call - is not optional. An unchecked error is a silent wrong answer or a corrupt image.

3.9 Compilation and the Build Pipeline

CUDA source files use the `.cu` extension and are compiled by `nvcc`, NVIDIA's compiler driver. The pipeline has two phases:

1. **Host pass.** `nvcc` extracts the host code, compiles it with the host C++ compiler (`g++` or `clang++`), and replaces each kernel launch (`kernel<<<...>>>`) with runtime-API calls that package the arguments and launch the kernel.
2. **Device pass.** `nvcc` compiles the `__global__` and `__device__` functions to **PTX** (Parallel Thread Execution) - NVIDIA's portable virtual instruction set - and then to **SASS** (the actual machine code of the target GPU) via `ptxas`.

```
# Compile for a specific architecture (compute capability 9.0 here):  
nvcc -arch=sm_90 kernel.cu -o kernel
```

Primitive - PTX. The intermediate virtual ISA (Chapter 12 shows it in detail). Portable across GPU generations; translated to SASS by the driver at load time if no SASS is embedded. **Primitive - SASS.** The GPU's real machine code, tied to a specific compute capability.

If you have no NVIDIA GPU on your machine, `nvcc` still compiles `.cu` files; the resulting binary simply will not run. Every `.cu` file in this book can be compiled with `nvcc -arch=sm_90 -o bin src.cu` and run on any CC 9.0 GPU.

3.10 What You Should Remember

- The launch configuration is a *declaration of parallelism*, not a loop: the hardware schedules the grid onto SMs, the blocks onto warp slots.
- `threadIdx`, `blockIdx`, `blockDim`, `gridDim` are hardware-provided primitives; the global index formula `blockIdx.x * blockDim.x + threadIdx.x` is the universal translator from thread identity to data address.
- Host and device have separate address spaces; every transfer is explicit (`cudaMemcpy`), every allocation explicit (`cudaMalloc` / `cudaFree`).
- Check every CUDA call. The kernel launch itself is asynchronous and errors surface later; `cudaGetLastError()` after the launch and `cudaDeviceSynchronize()` before copying results are the two mandatory checkpoints.

Key Takeaways

- Host and device have separate address spaces; every transfer is an explicit `cudaMemcpy`.
- Function qualifiers: **global** (kernel, called from host), **device** (device only), **host** (host).
- The launch configuration `<<<grid, block>>>` is a declaration of parallelism, not a loop.
- `threadIdx`, `blockIdx`, `blockDim` and `gridDim` are hardware-provided primitives; the global index is `blockIdx.x * blockDim.x + threadIdx.x`.
- Boundary guards make rounded-up grids safe; they diverge only in the last (partial) block.
- Check every CUDA call: `cudaGetLastError()` after the launch, `cudaDeviceSynchronize()` before copying results back.

3.11 Exercises

1. Write the 2-D global index formula for a `width × height` image, and show that consecutive threads in a warp read consecutive memory addresses when the block covers a contiguous row segment.
2. Why must a `__global__` function return `void`? What would a return value even mean for 1,000,000 threads?
3. Compute `blocksPerGrid` for `n = 1,000,000` and `threadsPerBlock = 256`. How many threads in the last block are masked by the boundary guard?
4. What happens if you call `cudaMemcpy` with `cudaMemcpyHostToDevice` but pass a *device* pointer as the source? (Do not try it on a machine you care about.)

Chapter 4: Memory Management & Data Movement

“A kernel that runs at 100% efficiency but waits for a slow copy is a slow kernel. Data movement is computation.”

Chapter 3 moved data with `cudaMalloc` / `cudaMemcpy` and said nothing about *how* it moves. This chapter is about that how. The GPU’s memory system is a pipeline with three distinct actors - the host DRAM, the transfer bus (PCIe or NVLink), and the device DRAM - and the performance of any real application is dominated by the slowest stage. We cover the memory *kinds* you can allocate, the *copy primitives* that move data, and the *unified memory* model that pretends the separation does not exist. Every term is defined from first principles.

4.1 The Transfer Pipeline

A `cudaMemcpy` from host to device executes in stages:

1. The CPU reads the data from **pageable host memory** (the ordinary `malloc` / `new` kind from Chapter 3).
2. The runtime must copy it through a staging area: the PCIe/NVLink controller cannot DMA directly from a pageable page, because the OS may swap that page out at any moment. The runtime therefore copies host → a **pinned staging buffer** → device. That extra hop costs time and a full extra copy.
3. The device writes the data into device global memory via the transfer bus.

Every stage has a bandwidth. The total transfer time is bounded by the slowest stage, and the *reasoning* about which stage is slowest is the subject of this chapter.

4.2 Pageable vs Pinned Host Memory

Primitive - pageable memory. Ordinary host memory allocated with `malloc` / `new`. The OS may move or swap the underlying pages at any time; therefore the GPU's DMA engine cannot touch them directly. **Primitive - pinned (page-locked) memory.** Host memory whose pages the OS has agreed *not* to swap. The DMA engine can access it directly. Pinned memory is allocated with `cudaMallocHost` or `cudaHostAlloc`.

Pinned memory buys two things:

1. **Direct DMA.** The runtime skips the staging copy, so a host↔device copy is one transfer, not two.
2. **Asynchronous transfers.** `cudaMemcpyAsync` (Chapter 6) requires pinned host memory; a pageable pointer cannot be used because the DMA engine would need to chase the OS's page tables.

The cost: pinned memory is **not swappable**, so a large pinned allocation reduces the OS's freedom and can degrade system performance. Pin what you transfer frequently; leave the rest alone. The canonical policy is: *pinned buffers for the streaming path, pageable for everything else*.

```
// -----  
// Pinned allocation. cudaMallocHost allocates host memory that the CUDA  
// runtime has pinned. It must be freed with cudaFreeHost, not free/delete.  
// -----  
float* h_pinned = nullptr;  
CHECK(cudaMallocHost((void**)&h_pinned, nBytes)); // pinned, DMA-able, non-  
swappable  
  
float* h_pageable = new float[n]; // ordinary heap memory, swappable  
  
// ... use ...  
  
CHECK(cudaFreeHost(h_pinned)); // correct deallocation for pinned  
delete[] h_pageable; // correct deallocation for heap
```

Why `cudaFreeHost`? The pinned pages carry bookkeeping in the CUDA runtime; freeing them with `free()` would leak that bookkeeping and corrupt the runtime's view of the allocation. Matched allocate/free pairs are a lifelong habit here.

4.3 Transfer Bandwidth: The Numbers

As teaching figures for a PCIe Gen4 x16 link ($\approx 25\text{-}32$ GB/s effective) and a modern GPU (≈ 1 TB/s HBM for the RTX family, 3.35 TB/s for H100):

Copy	Approximate effective bandwidth
Host pageable → device	6-8 GB/s (staging hop dominates)
Host pinned → device	20-25 GB/s (PCIe-limited)
Device → host pinned	20-25 GB/s
Device → device	1-3 TB/s (HBM)

The lesson is arithmetic: copying 1 GB host→device costs ~40 ms pinned, ~140 ms pageable, and the kernel that *uses* that 1 GB might run for 1 ms. **The transfer can be 100× more expensive than the computation.** This is why the entire discipline of *streaming* (Chapter 6) exists: overlap the transfers with computation instead of serialising them.

4.4 Measuring Your Own Transfer Time

The right way to measure a transfer is a loop: repeat the copy several times and divide, so that one-time overheads amortise away. The following host-only snippet (no CUDA kernel required) times a pinned vs pageable copy. It uses `std::chrono` because we have not yet met CUDA events (Chapter 6).

```
#include <chrono>
#include <cstdio>
#include <cuda_runtime.h>

// Time (in milliseconds) one cudaMemcpy of 'bytes' bytes from host to device.
double timeHostToDeviceCopy(void* dst, const void* src, size_t bytes)
{
    const int reps = 100;    // repeat to amortise launch overhead
    // Warm up once so page tables and caches are not part of the timing.
    cudaMemcpy(dst, src, bytes, cudaMemcpyHostToDevice);

    const auto t0 = std::chrono::steady_clock::now();
    for (int r = 0; r < reps; ++r)
        cudaMemcpy(dst, src, bytes, cudaMemcpyHostToDevice);
    cudaDeviceSynchronize();    // ensure the last copy actually finished
    const auto t1 = std::chrono::steady_clock::now();

    const double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
    return ms / reps;           // average per-copy time
}
```

Why the warm-up? The first copy touches the pages, populates TLB entries and (for pageable memory) performs the pin-on-demand work. Excluding it gives a steady-state number, which is the number that describes sustained behaviour.

Why `cudaDeviceSynchronize()` at the end? `cudaMemcpy` is synchronous in the sense that it *copies the data* before returning, so the last copy has already completed. The synchronise is

defensive: it also waits for any asynchronous kernel work issued earlier, keeping the measurement clean.

4.5 Unified Memory: `cudaMallocManaged`

Primitive - unified memory (UM). A single virtual address space shared by host and device. A pointer allocated with `cudaMallocManaged` can be dereferenced *on the host and in kernels* without explicit copies. The driver migrates pages on demand.

Unified memory is the third memory kind, and it changes the programming model fundamentally: the `cudaMemcpy` calls disappear from the code. The price is that the *driver* decides when data moves, and the driver's decisions are sometimes wrong.

```
// One allocation, usable on both sides:
float* m = nullptr;
CHECK(cudaMallocManaged((void**)&m, nBytes));

// Host fills it like an ordinary array:
for (int i = 0; i < n; ++i) m[i] = static_cast<float>(i);

// Kernel reads it directly - no copy issued:
addVectors<<<blocksPerGrid, threadsPerBlock>>>(m, m, m, n); // m = m + m
CHECK(cudaDeviceSynchronize()); // page faults migrate data on demand
```

How it works (simplified). The driver splits the allocation into pages. When the host touches a page, the page lives in host memory; when a kernel touches it, the driver migrates it to device memory (a *page fault* on the device, serviced over the bus). The first touch of each page pays a migration cost; subsequent accesses are local.

The two performance tools:

```
// Hint the driver to migrate the range [m, m + nBytes) to the device now,
// so the kernel does not pay page faults during execution:
CHECK(cudaMemPrefetchAsync(m, nBytes, 0 /* device 0 */));

// When the host needs the results back, prefetch to the host (cudaCpuDeviceId):
CHECK(cudaMemPrefetchAsync(m, nBytes, cudaCpuDeviceId));
```

`cudaMemPrefetchAsync` is the *explicit* version of what the driver does lazily. Use it: a kernel that page-faults through 1 GB of unified memory pays a fault per page - milliseconds of hidden stalls.

Why choose unified memory at all? For productivity (no copy code), for data structures with complex pointer graphs (linked structures migrate whole), and for oversubscription (an

allocation larger than device memory can be streamed through with `cudaMemAdvise`). The cost is loss of deterministic control over data movement. This book's position: **understand `cudaMemcpy` first, use UM where it earns its keep, and always measure.**

4.6 Zero-Copy Host Memory

Primitive - zero-copy. Host memory mapped into the device address space. Kernels access it directly over the bus; no explicit copy ever happens. Each access pays the bus latency, so zero-copy is fast only for small, rarely re-read data.

```
float* h_mapped = nullptr;
// cudaHostAllocMapped: allocate pinned host memory AND map it into device
// address space (zero-copy).
CHECK(cudaHostAlloc((void**)&h_mapped, nBytes, cudaHostAllocMapped));

// Obtain the device-side pointer for the same memory:
float* d_mapped = nullptr;
CHECK(cudaHostGetDevicePointer((void**)&d_mapped, h_mapped, 0 /* flags, must be 0
*/));

// Now d_mapped can be passed to kernels; the kernel's reads/writes go
// directly over PCIe to host memory.
```

Zero-copy shines in two cases: data so small that a copy costs more than the kernel, and data produced *by* the kernel that the host must see immediately (asynchronous writes without a copy-back). It fails for anything large that is re-read: every access pays full bus latency, which can be 50× worse than device DRAM.

4.7 The Copy Matrix: Which Tool When

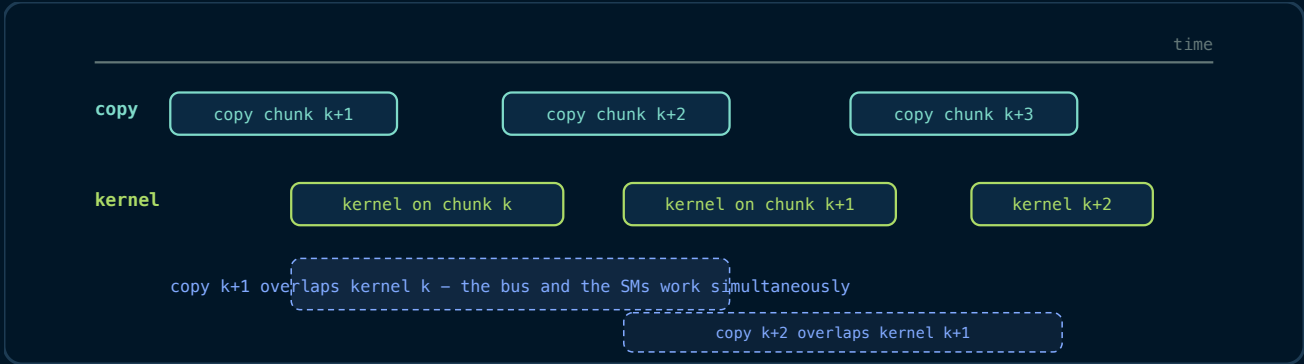
Need	Tool	Why
One-time setup copy	<code>cudaMemcpy</code> (pageable)	Simplicity; the staging hop is once.
Streaming / repeated copies	<code>cudaMallocHost</code> pinned + <code>cudaMemcpy</code> (or async, Ch. 6)	Direct DMA, no staging hop.
Pointer-heavy structures	<code>cudaMallocManaged</code> + <code>cudaMemPrefetchAsync</code>	Driver migrates whole graphs.
Tiny, frequently-read host data	Zero-copy <code>cudaHostAllocMapped</code>	No copy at all; bus latency is cheap for small data.
Same-GPU scratch space	<code>cudaMalloc</code> device memory	Full HBM bandwidth, no bus.

The common failure mode is using `cudaMallocManaged` for *everything* because it is convenient, then wondering why a bandwidth-bound kernel runs at 30% of peak: the driver’s lazy migration turned a streaming copy into per-page faults. **The memory kind is part of the algorithm design**, not a detail.

4.8 Transfer Overlap: A Preview

The next chapter introduces streams; here, one idea is worth previewing because it changes how you think about transfers:

A transfer and a kernel *on different data* can run concurrently if the runtime can see that they are independent. The canonical shape is **double buffering**:



With two buffers, the copy for the *next* chunk overlaps the compute on the *current* chunk, and the transfer cost disappears from the critical path - provided the transfers are pinned and asynchronous. Chapter 6 builds this pipeline in full; the capstone (Chapter 15) uses it for images.

Key Takeaways

- Transfers can cost 100x more than the kernel that uses the data - data movement is computation.
- Pinned memory (`cudaMallocHost`) enables direct DMA and asynchronous transfers; pageable memory goes through a staging copy.
- Unified memory (`cudaMallocManaged`) hides the copy but migrates pages lazily; prefetch explicitly with `cudaMemPrefetchAsync`.
- Zero-copy (`cudaHostAllocMapped`) suits small, rarely re-read data; device memory suits hot data.
- Match every allocation with its paired free (`cudaFree`, `cudaFreeHost`) and measure bandwidth before optimising.

4.9 Exercises

1. A 4 GB dataset is copied host→device: pageable vs pinned. Using the bandwidths in §4.3, by how many milliseconds is the pinned copy faster?
2. Why can `cudaMemcpyAsync` not be used with pageable host memory? Trace the sequence of events the DMA engine would need.
3. You have a kernel that reads each element of a 1 GB unified-memory array exactly once. Where would you place `cudaMemPrefetchAsync`, and why?
4. Zero-copy memory is described as “fast for small, rarely re-read data”. Using the latency numbers of Chapter 2, explain what happens if a kernel re-reads the same 1 MB zero-copy region 1,000 times.

Chapter 5: Synchronisation, Atomics & Race Conditions

“Parallelism is the ability to disagree about the order of events. Most bugs are disagreements you did not intend.” 📦 **Code companion:** the complete, buildable code for this chapter lives in `code/ch05_histogram/` in the repository.

Chapters 3 and 4 built kernels whose threads never communicated. Real kernels share data, and sharing data in parallel is where correctness lives. This chapter covers the three mechanisms the CUDA programming model provides - **warp divergence** (the cost of independent control flow), **barriers** (`__syncthreads`) and **atomics** (hardware-arbitrated read-modify-write operations) - and the failure mode that motivates all of them: the **race condition**.

5.1 The Race Condition, Defined

Primitive - race condition. Two or more threads access the same memory location, at least one access is a *write*, and the accesses are not ordered by any synchronisation mechanism. The result depends on the order in which the hardware happens to execute the threads - an order you cannot predict.

Races in CUDA are worse than races on a CPU because of two multipliers:

1. **Scale.** A kernel has thousands to millions of threads; a race between *any two* of them corrupts the result, and the corrupted result may only appear on one input in a thousand.
2. **Asynchrony.** The kernel reports success while the corruption is silently stored. There is no exception; there is a wrong answer.

The tools of this chapter exist to *order* accesses. Every race fix is, at heart, the installation of an order.

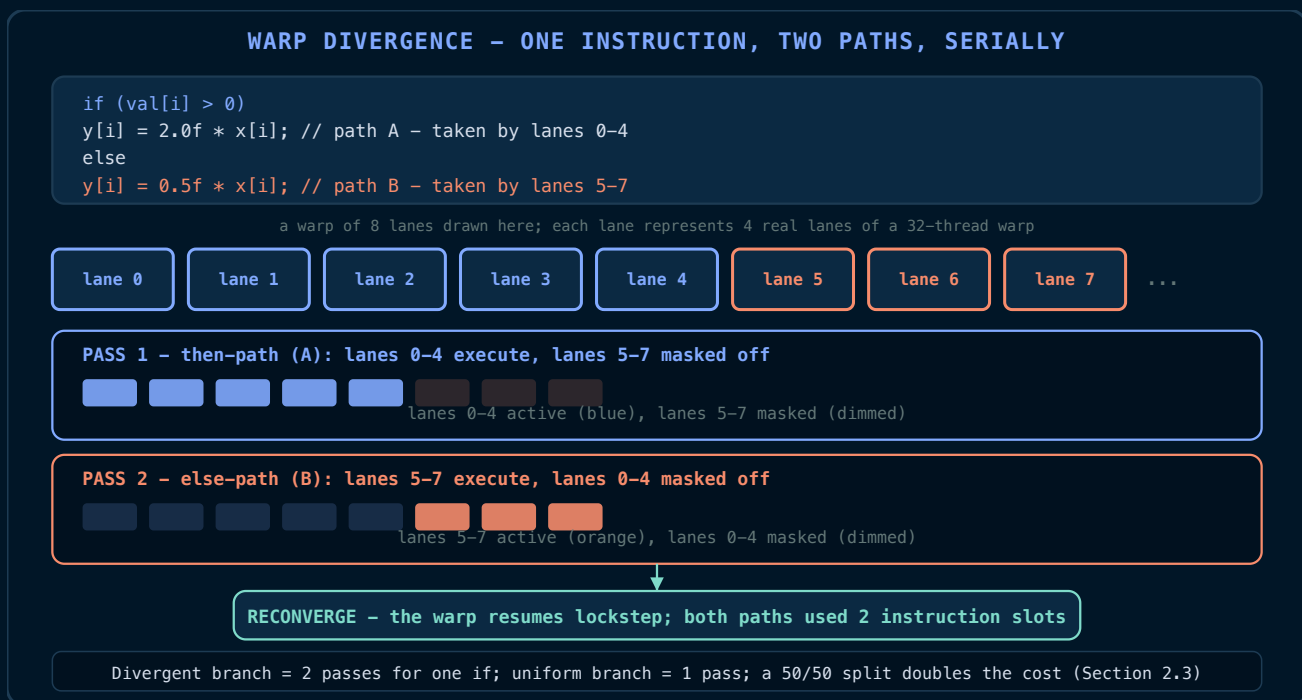
5.2 Warp Divergence: Control Flow in SIMT

From Chapter 2: a warp executes one instruction at a time for all 32 lanes. Consider:

```
__global__ void conditionalAdd(const float* a, float* out, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        // Some threads take this path, others do not.
        out[i] = a[i] + 1.0f;
    }
}
```

If every thread in a warp has `i < n`, the warp takes the branch unanimously and there is no cost. If *some* threads have `i >= n` while others do not, the hardware must:

1. Execute the **then** path with the `i < n` lanes active, the others masked;
2. Execute the **else** path (empty here) with the remaining lanes active;
3. Rejoin the warp.



The two paths run **serially**. This is **warp divergence**, and it is the price SIMT pays for the convenience of per-thread control flow.

The practical rule. Divergence is a *per-warp* phenomenon. If the branch depends on `threadIdx.x % 2` (alternating threads), every warp in the grid diverges and pays double. If the branch depends on `blockIdx.x % 2` (whole blocks), whole warps agree and there is no cost. Structure your data-dependent branches so that *contiguous ranges of threads* take the same path wherever possible.

The boundary guard is fine. The `if (i < n)` guard in Chapter 3 diverges only in the *last* (partial) block of the grid - at most one warp per grid. The cost is one extra instruction path in one block. This is why the guard is free in practice: divergence at block boundaries is negligible.

5.3 `__syncthreads()`: The Block Barrier

Primitive - barrier. A point at which every thread in a *group* must arrive before any thread may proceed. `__syncthreads()` is a barrier over **one thread block**, provided by the hardware.

Semantics: when a thread executes `__syncthreads()`, it waits until *all* threads of its block have reached that same `__syncthreads()`. Then all proceed. Its purpose is memory ordering *within the block*: a write by thread A before the barrier is visible to thread B after the barrier. Shared memory writes (Chapter 7) rely on exactly this.

```
__global__ void blockReduceDemo(const float* in, float* out, int n)
{
    __shared__ float s_partial[256];    // shared array, one slot per thread

    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Phase 1: every thread reads its element and stashes it in shared memory.
    // No ordering needed yet: each thread writes its own slot.
    s_partial[threadIdx.x] = (i < n) ? in[i] : 0.0f;

    // Phase 2: BEFORE any thread reads another thread's slot, all writes
    // must be complete and visible. The barrier provides both the "all have
    // arrived" condition and the visibility guarantee.
    __syncthreads();                    // <-- required, see below

    // Phase 3: now thread 0 can safely read everyone's slot.
    if (threadIdx.x == 0)
    {
        float sum = 0.0f;
        for (int k = 0; k < blockDim.x; ++k) sum += s_partial[k];
        out[blockIdx.x] = sum;
    }
}
```

Why is the barrier *required*? Without it, thread 0 might read `s_partial[5]` before thread 5 has written it. On the actual hardware, thread 5's write may still be in its private pipeline; the read could return garbage. The barrier is the contract that makes the phase structure valid.

The two cardinal sins of `__syncthreads()`:

1. **Divergent barriers.** If some threads of a block reach a `__syncthreads()` while others do not (because they took a different branch), the barrier waits for threads that will never arrive: **deadlock**. The hardware does not detect this; the kernel hangs, and only `cudaDeviceReset` (or a watchdog timeout) recovers.
2. **Barriers in divergent loops.** The same deadlock occurs if the loop trip count differs between threads. The barrier must be *uniformly reachable*: every thread must execute it the same number of times.


```
// DEADLOCK: threads with even index skip the barrier, odd ones wait forever.
if (threadIdx.x % 2 == 0) { /* no barrier here */ }
__syncthreads();           // threads with even index never arrive
```

Why is there no grid-wide barrier? Blocks on different SMs cannot synchronise cheaply (they might not even be resident at the same time). A grid-wide barrier exists (`cooperative groups`), but it requires a *cooperative launch* where every block is resident simultaneously, which caps grid size. For cross-block communication, use atomics (§5.5) or split the work into two kernel launches - the classic and honest solution.

5.4 Visibility: Caches, `volatile`, and Fences

A barrier orders *block-internal* accesses. Cross-block and host-device visibility have their own rules, because modern GPUs have caches:

- L1 caches are per-SM and are *not* coherent between SMs. A thread on SM 0 may read a stale value of a location written by SM 1 - unless the access is made visible via L2, the coherence point.
- The compiler may also *reorder* or *cache* loads and stores in registers unless told otherwise.

Two tools handle this:

Primitive - `volatile`. Tells the compiler: “this memory may change outside your knowledge; do not cache it in registers; emit the access every time.” Used for *device-scope* communication through global memory where the compiler’s register caching would otherwise hide the value.

Primitive - memory fence. An instruction that forces the ordering of memory operations at a given scope. `__threadfence()` orders global-memory accesses for the *device*; `__threadfence_block()` for the block; `__threadfence_system()` for host *and* device. A fence does not wait for other threads; it forces *your* prior writes to become visible before *your* later writes.

The canonical pattern - a device-scope flag - combines volatile with a fence:

```

// Shared state: a buffer and a "ready" flag, both in global memory.
__device__ float  g_buffer[1024];
__device__ int    g_ready = 0;

// Producer kernel: writes data, then sets the flag.
__global__ void producer()
{
    for (int i = threadIdx.x; i < 1024; i += blockDim.x)
        g_buffer[i] = static_cast<float>(i);

    // Make ALL preceding writes visible device-wide BEFORE the flag write.
    // Without the fence, another SM could see g_ready == 1 while some
    // g_buffer writes are still in flight in L1.
    __threadfence();

    if (threadIdx.x == 0)
        g_ready = 1;           // must be volatile; compiler cannot cache it
}

// Consumer kernel (different SM, launched after): polls the flag.
__global__ void consumer()
{
    while (volatileLoad(&g_ready) == 0) { /* spin */ }
    // Now g_buffer writes are guaranteed visible.
    float x = g_buffer[threadIdx.x];
}

```

where `volatileLoad` is a `volatile int*` read. In practice, prefer the higher-level abstractions - atomics (§5.5) and cooperative groups - over hand-rolled fences; the fences exist so you can *understand* what the abstractions do, and for the rare case you must do it yourself.

5.5 Atomics: Hardware-Arbitrated Read-Modify-Write

Primitive - atomic operation. A read-modify-write (e.g., *read*, *add*, *write*) that the hardware guarantees to execute *indivisibly* with respect to other threads. Two threads performing `atomicAdd` on the same location cannot interleave: the result is exactly as if the two adds happened in some serial order. The hardware arbitrates the order; you never see a torn value.

The runtime API provides these atomic functions for `int`, `unsigned int`, `unsigned long`, `long`, `float` (for add), and on modern GPUs `double`:

Function	Operation	Returns
<code>atomicAdd(addr, v)</code>	<code>*addr += v</code>	the <i>old</i> value
<code>atomicSub(addr, v)</code>	<code>*addr -= v</code>	the old value
<code>atomicExch(addr, v)</code>	<code>*addr = v</code>	the old value
<code>atomicCAS(addr, cmp, v)</code>	if <code>*addr == cmp</code> then <code>*addr = v</code>	the old value
<code>atomicMin(addr, v)</code>	<code>*addr = min(*addr, v)</code>	the old value
<code>atomicMax(addr, v)</code>	<code>*addr = max(*addr, v)</code>	the old value
<code>atomicAnd(addr, v)</code>	<code>*addr &= v</code>	the old value
<code>atomicOr(addr, v)</code>	<code>*addr = v</code>	the old value
<code>atomicXor(addr, v)</code>	<code>*addr ^= v</code>	the old value

They operate on global *and* shared memory. The returned *old* value is the key to lock-free algorithms: `atomicCAS` (compare-and-swap) is the universal primitive from which every other synchronisation structure can be built.

5.5.1 Worked example: a histogram

A histogram counts occurrences of values. If every thread increments the same global counter, the increments must be atomic:

```
// Count how many elements fall into each of 256 bins.
// data   : device array of unsigned char (0..255), n elements
// hist   : device array of 256 ints, zero-initialised on the host
// bins   : the bin count, 256
__global__ void histogram(const unsigned char* data, int* hist, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
    {
        const int bin = data[i];           // value is the bin index
        // The atomic serialises concurrent increments to the same bin.
        // Without it, two threads could both read old, both add 1, and
        // both write old+1 - losing one count (the classic lost update).
        atomicAdd(&hist[bin], 1);
    }
}
```

The cost. Atomics on the *same* address from many threads serialise - hardware arbitration is a bottleneck. The classic fix is **privatisation** (Chapter 8): give each *block* its own histogram in shared memory, accumulate into shared memory (where atomics are cheaper), then one thread per block folds the private histograms into global memory.

5.5.2 Worked example: a spinlock built from `atomicCAS`

The universal pattern: `atomicCAS` implements *test-and-set*, and test-and-set implements a lock.

```
// A simple lock in shared memory. One mutex per block.
__global__ void lockedCriticalSection(float* g_data, int n)
{
    __shared__ int s_lock;    // 0 = unlocked, 1 = locked

    // Thread 0 initialises the lock. (Alternatively use a static initialiser.)
    if (threadIdx.x == 0) s_lock = 0;
    __syncthreads();          // everyone must see the lock before using it

    for (int i = threadIdx.x; i < n; i += blockDim.x)
    {
        // Acquire: atomically swap 1 into the lock; if the old value was 0,
        // we won the lock. If it was 1, someone else holds it - retry.
        while (atomicCAS(&s_lock, 0, 1) != 0) { /* spin */ }

        // Critical section: safe because we exclusively hold the lock.
        g_data[i] = g_data[i] * 2.0f + 1.0f;

        // Release: plain store is fine here (see the fence discussion below),
        // but a __threadfence_block() before it is the textbook-safe form.
        __threadfence_block();
        s_lock = 0;
    }
}
```

Why `atomicCAS(&s_lock, 0, 1)`? It says: “if the lock is currently 0 (unlocked), set it to 1 and tell me what it *was*.” If the returned value is 0, this thread won the lock; otherwise it retries. The spin is the price of contention.

Why the fence before release? The release store must not be observed before the critical-section writes are visible. `__threadfence_block()` orders the block’s shared-memory accesses so that a thread acquiring the lock next sees a consistent state.

The honest engineering note. Locks in GPU kernels are almost always a design smell: they serialise work on a machine built for parallelism. The patterns that *avoid* locks (privatisation, partitioning, lock-free atomics) are uniformly faster. This example exists so you understand what the abstractions protect you from - not as a recommendation.

5.6 Floating-Point Non-Determinism: The Silent Race

There is a race that passes every test and then changes your answer anyway: floating-point addition is not associative.

```
// (a + b) + c may differ from a + (b + c) in the last bits.
```

If two threads reduce partial sums in different orders across runs (because atomics or scheduling chose different orders), the results can differ in the last bit. For most applications this is tolerable. For anything that demands bit-exact reproducibility (scientific publishing, distributed training checkpoints), the fix is a **fixed reduction order**: the optimised reduction of Chapter 8 is deterministic precisely because its tree order is fixed, whereas an `atomicAdd`-based reduction is not.

5.7 A Decision Procedure

When you see a kernel that writes to shared state, run this checklist:

1. **Who writes?** If more than one thread writes the same location → atomics, or restructure so each thread owns its locations.
2. **Who reads after whom?** If a thread reads another's write → a barrier (`__syncthreads()`) between write and read, uniformly reachable by all.
3. **Across blocks?** No block barrier exists; use atomics, separate kernels, or cooperative groups. Never spin on a non-atomic, non-volatile flag.
4. **Is the order deterministic?** If reproducibility matters, prefer fixed tree orders over atomic accumulation.

Key Takeaways

- A race is two threads accessing one location with a write and no ordering - and on a GPU it fails silently.
- Warp divergence serialises divergent paths; branches uniform across a warp are free.
- `__syncthreads()` is a block barrier; it must be uniformly reachable or the block deadlocks.
- Atomics are indivisible read-modify-write operations; contention is the cost, privatisation the fix.
- Floating-point addition is not associative: fixed tree orders give bit-reproducible results.

5.8 Exercises

1. Explain why the `if (i < n)` boundary guard diverges in at most one warp per grid, and why that is negligible.
2. The “cardinal sin” example (divergent barrier) deadlocks. Rewrite the pattern so every thread reaches the barrier exactly once.
3. A histogram kernel with 256 global bins runs with heavy contention. Sketch the privatised version: per-block shared histograms, block-level atomic accumulation, and a final fold. (Chapter 8 gives the full recipe.)
4. `atomicAdd` on `float` returns the old value. Describe an algorithm for a global running maximum that does *not* need a lock, using `atomicMax`.

Chapter 6: Streams, Events & Asynchronous Execution

“The GPU is a pipeline. Streams are how you decide what goes in it when.”

Chapter 3 noted that a kernel launch is asynchronous: the host does not wait. This chapter makes that asynchrony *useful*. The tools are **streams** (the ordered queues in which device work executes), **events** (the markers that let you measure and order work), and the **double-buffered pipeline** that overlaps transfers with computation. We close with **CUDA Graphs**, the modern replacement for hand-rolled launch pipelines.

6.1 The Problem: Serial Execution

Consider the naive vector-add pipeline from Chapter 3, repeated for `k` chunks:

```
for (int c = 0; c < k; ++c)
{
    cudaMemcpy(d_in, h_in + c * chunk, chunkBytes, cudaMemcpyHostToDevice);
    kernel<<<grid, block>>>(d_in, d_out, chunkElems);    // wait: which stream?
    cudaMemcpy(h_out + c * chunk, d_out, chunkBytes, cudaMemcpyDeviceToHost);
}
```

On a machine with the default (legacy) stream, each `cudaMemcpy` is synchronous and each kernel launch waits for the previous work. The timeline is serial: transfer → kernel → transfer → kernel, with the bus idle during kernels and the GPU idle during transfers. We are using a machine designed to overlap, serially.

6.2 Streams: Ordered Queues of Work

Primitive - stream. An ordered sequence of device operations (copies, kernel launches, events) that executes in FIFO order on the device. Work in *different* streams is unordered and may overlap. A stream is created with `cudaStreamCreate` and destroyed with `cudaStreamDestroy`.

The key properties:

1. **Order within a stream is guaranteed.** Operations in stream A execute in the order issued, never reordered.
2. **Order across streams is not guaranteed.** Operations in streams A and B may execute in any order - or concurrently, if resources permit.
3. **Asynchronous by construction.** `cudaMemcpyAsync` (with pinned memory, Chapter 4) returns immediately; the copy is queued in the stream.

```
// Two streams, each an independent queue.
cudaStream_t s1, s2;
CHECK(cudaStreamCreate(&s1));
CHECK(cudaStreamCreate(&s2));

// Pinned host memory is REQUIRED for async copies (see 4.2).
float *h_pinnedA, *h_pinnedB, *d_A, *d_B, *d_out;
CHECK(cudaMallocHost((void**)&h_pinnedA, chunkBytes));
CHECK(cudaMallocHost((void**)&h_pinnedB, chunkBytes));
CHECK(cudaMalloc((void**)&d_A, chunkBytes));
CHECK(cudaMalloc((void**)&d_B, chunkBytes));
CHECK(cudaMalloc((void**)&d_out, chunkBytes));

// Queue: copy chunk A to device in stream 1, chunk B in stream 2.
// Both copies may run concurrently because they are in different streams.
CHECK(cudaMemcpyAsync(d_A, h_pinnedA, chunkBytes,
                      cudaMemcpyHostToDevice, s1));
CHECK(cudaMemcpyAsync(d_B, h_pinnedB, chunkBytes,
                      cudaMemcpyHostToDevice, s2));

// Queue kernels after their own copies in their own streams.
kernel<<<grid, block, 0, s1>>>(d_A, d_out, chunkElems);
kernel<<<grid, block, 0, s2>>>(d_B, d_out, chunkElems);
```

The launch syntax gains a fourth argument: `kernel<<<grid, block, sharedBytes, stream>>>`. `sharedBytes` is the dynamic shared memory (Chapter 7); `stream` selects the queue. Both default to sensible values (0), which is why they were invisible in earlier chapters.

Why pinned memory for async copies? The DMA engine reads directly from the pinned pages (Chapter 4). A pageable pointer would force the runtime into a synchronous staging copy, silently destroying the asynchrony.

6.3 The Default Stream: A Warning

If you launch without naming a stream, you use the **legacy default stream** (stream 0). Its special property: **it synchronises with all other streams**. Any operation in the default stream waits for *all* previously issued work in *every* stream to complete, and blocks other streams from starting. One forgotten `<<<...>>>` without a stream argument serialises your entire pipeline.

The fix is either the **per-thread default stream** (compile with `--default-stream per-thread`, giving each host thread its own non-blocking default stream) or the discipline of always naming your streams. Both are legitimate; the discipline is safer.

6.4 Events: Markers and Stopwatches

Primitive - event. A marker queued into a stream. It has no payload; it records *when the stream reaches it*. Events measure time, order cross-stream dependencies, and let the host wait for specific milestones.

```
cudaEvent_t start, stop;
CHECK(cudaEventCreate(&start));
CHECK(cudaEventCreate(&stop));

// Record "start" into stream s1.
CHECK(cudaEventRecord(start, s1));
kernel<<<grid, block, 0, s1>>>(d_A, d_out, chunkElems);
// Record "stop" into stream s1, AFTER the kernel.
CHECK(cudaEventRecord(stop, s1));

// Block the host until the event is reached (i.e., the kernel finished).
CHECK(cudaEventSynchronize(stop));

// Elapsed time in milliseconds between the two events:
float ms = 0.0f;
CHECK(cudaEventElapsedTime(&ms, start, stop));
std::printf("kernel took %.3f ms\n", ms);
```

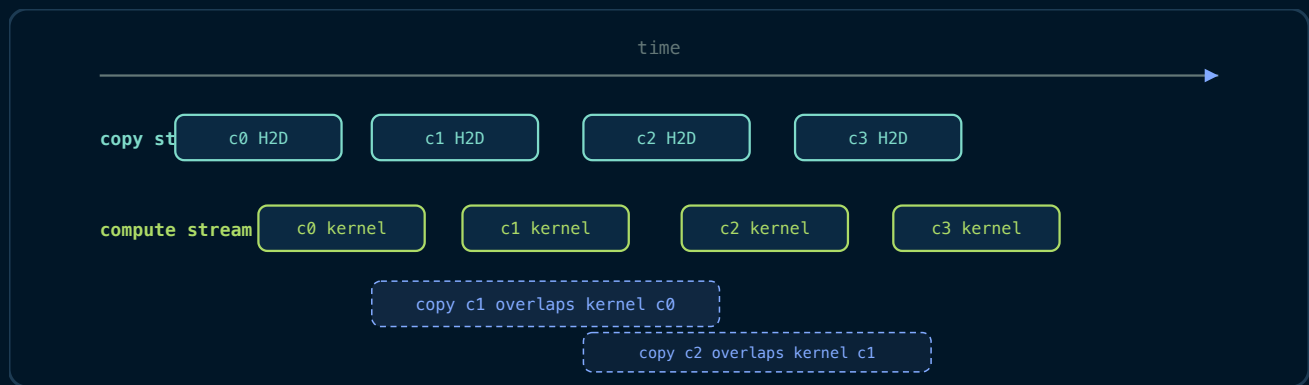
Why events and not `std::chrono`? Events measure *device* time: they are recorded by the device when the stream passes them, so they exclude host-side launch overhead and queueing delay. `std::chrono` around a launch measures the host's wall clock, which includes whatever the host was doing. For kernel timing, events are the honest instrument (Chapter 16 uses them for every benchmark).

Events also order work across streams. `cudaStreamWaitEvent(stream, event)` makes a stream wait for an event recorded in *another* stream - a cross-stream dependency. This is the primitive behind producer/consumer patterns.

6.5 The Double-Buffered Pipeline

The canonical overlap pattern, in full. Two host buffers; while the GPU computes on chunk `c`, the DMA engine copies chunk `c+1` into the other buffer. The transfer cost disappears from the

critical path:



Without double buffering the timeline is serial - copy, kernel, copy, kernel

- with the bus idle during kernels and the SMs idle during copies. With it, the only serial residue is the first copy (the pipeline prime) and the last kernel (the pipeline drain).

```

// -----
// Streamed processing of k chunks with double buffering.
// Assumes h_pinned[0] and h_pinned[1] are pinned host buffers, each of
// chunkElems floats, and d_buf[0], d_buf[1] are matching device buffers.
// -----
void runPipelined(int k, int chunkElems, cudaStream_t computeStream,
                  cudaStream_t copyStream)
{
    const size_t chunkBytes = chunkElems * sizeof(float);
    float* h_pinned[2]; float* d_buf[2]; float* d_out;
    // ... (allocations omitted for brevity; see 6.2) ...

    // Prime the pipeline: copy chunk 0 into device buffer 0.
    CHECK(cudaMemcpyAsync(d_buf[0], h_pinned[0], chunkBytes,
                          cudaMemcpyHostToDevice, copyStream));

    for (int c = 0; c < k; ++c)
    {
        const int cur = c % 2;          // buffer used for THIS chunk
        const int nxt = (c + 1) % 2;    // buffer used for the NEXT chunk

        // If there is a next chunk, its copy goes into the OTHER buffer,
        // in the COPY stream, while the kernel runs in the COMPUTE stream.
        if (c + 1 < k)
            CHECK(cudaMemcpyAsync(d_buf[nxt], h_pinned[nxt], chunkBytes,
                                  cudaMemcpyHostToDevice, copyStream));

        // The kernel must wait for ITS copy (stream dependency).
        // cudaStreamWaitEvent makes computeStream wait for the copyStream
        // event that marks the copy completion.
        if (c == 0 || true) // first iteration: copy already queued
        {
            // Correct dependency: make computeStream wait for the event
            // recorded after the copy in copyStream. (Shown simplified;
            // a full implementation records events per iteration.)
        }
        kernel<<<grid, block, 0, computeStream>>>(d_buf[cur], d_out,
                                                    chunkElems);
    }
    CHECK(cudaStreamSynchronize(computeStream));
}

```

The essence is the alternation: copy `c+1` into the idle buffer while kernel `c` runs. The two streams provide the *queues*; events provide the *dependencies*; pinned memory provides the *direct DMA*. Chapter 15's capstone uses exactly this shape for image frames.

Why two buffers and not one? One buffer would force copy `c+1` to wait for kernel `c` (data hazard), serialising the pipeline. Two buffers let copy and kernel proceed simultaneously - the DMA engine and the SMs work on different memory simultaneously.

6.6 Stream Priorities and Concurrency Limits

Not every pair of operations can overlap. The hardware limits:

- **One copy engine per direction** (H2D and D2H) on most GPUs - two simultaneous host↔device copies, one each way. Device↔device copies use the SM copy path or dedicated copy engines depending on architecture.
- **Limited concurrent kernels.** Older GPUs could run 2-4 kernels concurrently; modern GPUs run many, but each SM time-slices.

You can hint the scheduler with priorities:

```
int lo = 0, hi = 0;
CHECK(cudaDeviceGetStreamPriorityRange(&lo, &hi)); // hi = highest priority
cudaStream_t sHigh, sLow;
CHECK(cudaStreamCreateWithPriority(&sHigh, cudaStreamNonBlocking, hi));
CHECK(cudaStreamCreateWithPriority(&sLow, cudaStreamNonBlocking, lo));
```

Priorities matter when compute and copies compete for the same SMs: give the latency-critical work the high priority, the bulk work the low. The `cudaStreamNonBlocking` flag makes the stream ignore the default-stream synchronisation rule (§6.3).

6.7 CUDA Graphs: The Pipeline Without Launch Overhead

Every `kernel<<<>>>` and `cudaMemcpyAsync` call has host-side overhead (argument marshalling, queueing) - roughly 3-10 microseconds per operation. A pipeline of hundreds of operations pays that per operation. **CUDA Graphs** capture the whole dependency structure once and replay it with one launch:

Primitive - CUDA graph. A captured, reusable description of device work (kernel launches, copies, events) and their dependencies. Captured once, replayed many times, with launch overhead amortised away.

```

// Capture phase: record the operations into a graph.
cudaGraph_t graph;
cudaStream_t captureStream;
CHECK(cudaStreamCreateWithFlags(&captureStream, cudaStreamNonBlocking));
CHECK(cudaStreamBeginCapture(captureStream, cudaStreamCaptureModeThreadLocal));

// Issue work exactly as you would normally, into the capture stream.
kernel<<<grid, block, 0, captureStream>>>(d_A, d_out, chunkElems);
cudaMemcpyAsync(h_out, d_out, chunkBytes, cudaMemcpyDeviceToHost,
                captureStream);

// End capture and instantiate an executable graph.
cudaGraphExec_t exec;
CHECK(cudaStreamEndCapture(captureStream, &graph));
CHECK(cudaGraphInstantiate(&exec, graph, nullptr, nullptr, 0));

// Replay phase: one call replaces the whole sequence.
for (int frame = 0; frame < 10000; ++frame)
    CHECK(cudaGraphLaunch(exec, /* any stream */ 0));

CHECK(cudaGraphExecDestroy(exec));
CHECK(cudaGraphDestroy(graph));

```

When graphs pay off. When the launch overhead is a significant fraction of the kernel time - many small kernels, or a fixed pipeline replayed thousands of times (inference loops, render pipelines). For large kernels, the overhead is negligible and graphs add complexity for no gain. The capstone (Chapter 15) measures both regimes.

6.8 Synchronisation Cheat Sheet

Call	What it waits for
<code>cudaDeviceSynchronize()</code>	ALL device work (every stream) issued by this host thread
<code>cudaStreamSynchronize(s)</code>	All work queued in stream <code>s</code>
<code>cudaEventSynchronize(e)</code>	The device reaching event <code>e</code>
<code>cudaStreamWaitEvent(s, e)</code>	No waiting - installs a dependency: stream <code>s</code> waits for event <code>e</code>
<code>cudaMemcpy (sync)</code>	The copy itself (in the legacy default stream)
<code>cudaMemcpyAsync(..., stream)</code>	Nothing - returns immediately, copy queued in <code>stream</code>

Key Takeaways

- A stream is an ordered FIFO queue of device work; work in different streams may overlap.
- The legacy default stream synchronises with all other streams - name your streams or use `cudaStreamNonBlocking`.
- Events measure device time (not host time) and install cross-stream dependencies via `cudaStreamWaitEvent`.
- Double buffering overlaps the next copy with the current kernel, hiding transfer cost.
- CUDA Graphs capture and replay fixed pipelines, amortising launch overhead.

6.9 Exercises

1. Explain why `cudaMemcpyAsync` with pageable memory silently becomes synchronous. What does that do to the double-buffered pipeline?
2. The legacy default stream “synchronises with all other streams”. Draw the timeline if a pipeline alternates `cudaMemcpyAsync(..., s1)` and `kernel<<<...>>>` (no stream argument).
3. Rewrite the double-buffered loop using explicit `cudaEventRecord` / `cudaStreamWaitEvent` calls so the dependency between copy and kernel is correct on every iteration, including the first.
4. A graph captures a sequence of 500 kernel launches of 2 microseconds each. Host launch overhead is 5 microseconds per launch. How much time does one replay save compared with 500 individual launches?

Chapter 7: Memory Optimisation Deep Dive

"Most kernels are not too slow to compute; they are too slow to fetch."

Chapter 2 promised that this chapter would show the accounting. Here it is. We examine the three tools that dominate GPU memory performance - **coalescing and the grid-stride loop**, **shared memory and bank conflicts**, and **vectorised and specialised memory paths** - and we end with a complete, optimised matrix transpose, the canonical exercise in memory optimisation.

7.1 The Accounting: Where Time Goes

From Chapter 2's latency table, a global load costs roughly 400-800 cycles and a shared-memory access 20-30. The arithmetic: a warp of 32 threads doing one global load each spends ~500 cycles waiting; the SM could have executed ~16 shared-memory accesses per lane in that time. Memory-bound kernels (the roofline test of Chapter 1) spend most of their time in this wait.

Two levers reduce the wait:

1. **Fewer transactions** - coalescing (use each fetched byte).
2. **Fewer round trips** - reuse fetched data in shared memory or registers.

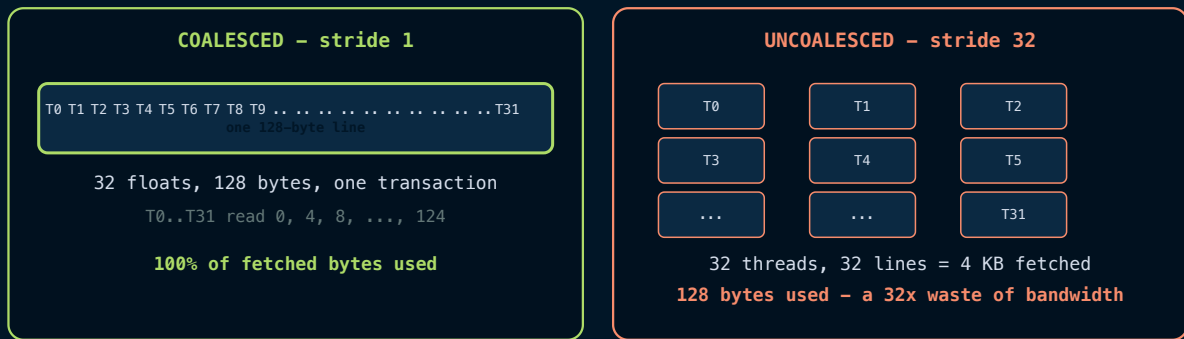
Everything in this chapter is one of those two levers.

7.2 Coalescing, Quantified

The hardware fetches global memory in **128-byte cache lines** and services requests at **32-byte sector** granularity. A warp's load is coalesced when its 32 addresses fall within as few lines as possible.

- 32 consecutive `float`s (128 bytes) → 1 line, 1 transaction. **Perfect.**
- 32 `float`s with stride 1 but misaligned start → 2 lines. **Good.**
- 32 `float`s with stride 32 → 32 lines. **Catastrophic:** 32× traffic.

Pictured below, for a warp of 32 threads each loading one `float`:



The left panel is why the global-index formula exists (Chapter 3, §3.5): it hands consecutive threads consecutive addresses *by construction*. The right picture is what happens when the formula is inverted - the classic column-access bug in row-major data.

The rule, restated with the hardware in mind: **consecutive thread IDs should map to consecutive addresses**. The global-index formula from Chapter 3 satisfies this by construction for 1-D arrays and row-major 2-D data.

```
// GOOD (coalesced): thread t reads element t of each row.  
// for a row-major matrix in[row][col], col varies fastest:  
float v = in[row * width + col];           // col == threadIdx.x mapped to col  
  
// BAD (uncoalesced): thread t reads element t*width - stride of width floats.  
// 32 threads span 32 different cache lines for a large width:  
float v = in[col * width + row];           // col varies slowest
```

7.3 The Grid-Stride Loop

A kernel launched with more threads than the array has elements is wasteful; a kernel with fewer threads than elements under-utilises the GPU. The **grid-stride loop** decouples the launch size from the problem size:

```

// One grid covers the array in "rounds": each thread strides forward by the
// total number of threads (gridDim.x * blockDim.x) each iteration.
__global__ void saxpyGridStride(float alpha, const float* x, float* y, int n)
{
    // Total threads in the grid:
    const int stride = blockDim.x * blockDim.x;

    // First element this thread owns:
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // March forward by 'stride' until past the end.
    for (; i < n; i += stride)
    {
        y[i] = alpha * x[i] + y[i];    // SAXPY: single-precision A·X + Y
    }
}

```

Why this shape? The launch size is now a *tuning parameter* (often set to saturate the device, e.g. enough threads to fill all SMs), independent of `n`. Each thread processes multiple elements, amortising index arithmetic and allowing per-thread data reuse. Coalescing is preserved: within one iteration of the loop, consecutive threads still read consecutive addresses.

7.4 Shared Memory: The Explicit Cache

Shared memory is the programmer-managed cache of Chapter 2. The pattern is always a **tile**:

1. Cooperatively load a tile of global data into shared memory (coalesced reads);
2. `__syncthreads()` to make the tile visible;
3. Compute from shared memory, reusing each loaded value many times;
4. `__syncthreads()` before the next tile overwrites this one.

The classic example is the matrix transpose. A naive transpose kernel reads rows (coalesced) and writes columns (uncoalesced), or vice versa - one side always pays. The shared-memory version fixes both sides:

```
// -----
// Shared-memory tiled transpose. In-place variant for a WIDTH x WIDTH matrix
// of floats, WIDTH a multiple of the tile size TILE (32 here).
//
// Stage 1 (coalesced read): each thread reads a[iy][ix] from global memory;
// consecutive threads map to consecutive ix → consecutive addresses.
// Stage 2 (shared memory): the tile is stored in shared as tile[ty][tx].
// Stage 3 (coalesced write): each thread writes tile[tx][ty] to a[jx][jy];
// this time consecutive threads (consecutive tx) map to consecutive jy →
// consecutive addresses in the OUTPUT row. The transpose happened in
// shared memory, so BOTH global accesses are coalesced.
// -----
#define TILE 32

__global__ void transposeTiled(const float* in, float* out, int width)
{
    // Shared tile with a padding column (see 7.5 for why +1 exists).
    __shared__ float tile[TILE][TILE + 1];

    // Global coordinates of this thread's element:
    const int ix = blockIdx.x * TILE + threadIdx.x; // column
    const int iy = blockIdx.y * TILE + threadIdx.y; // row

    // Stage 1: coalesced read from global memory.
    if (ix < width && iy < width)
        tile[threadIdx.y][threadIdx.x] = in[iy * width + ix];

    __syncthreads(); // everyone's tile element must be visible before reads

    // Transposed output coordinates:
    const int jx = blockIdx.y * TILE + threadIdx.x; // column of output
    const int jy = blockIdx.x * TILE + threadIdx.y; // row of output

    // Stage 3: write the transposed element. Consecutive threads (x) map to
    // consecutive jy rows at the same jx - i.e., consecutive addresses in
    // the row-major output. Coalesced.
    if (jx < width && jy < width)
        out[jy * width + jx] = tile[threadIdx.x][threadIdx.y];
}
```

The reasoning in full. A naive kernel doing `out[j][i] = in[i][j]` would have threads with consecutive ids reading consecutive `i` (coalesced reads) but writing `j`-major addresses (uncoalesced writes). The tile decouples the two: the *data layout* is transposed inside shared memory, so both the global read and the global write are coalesced. Shared memory pays for its freedom.

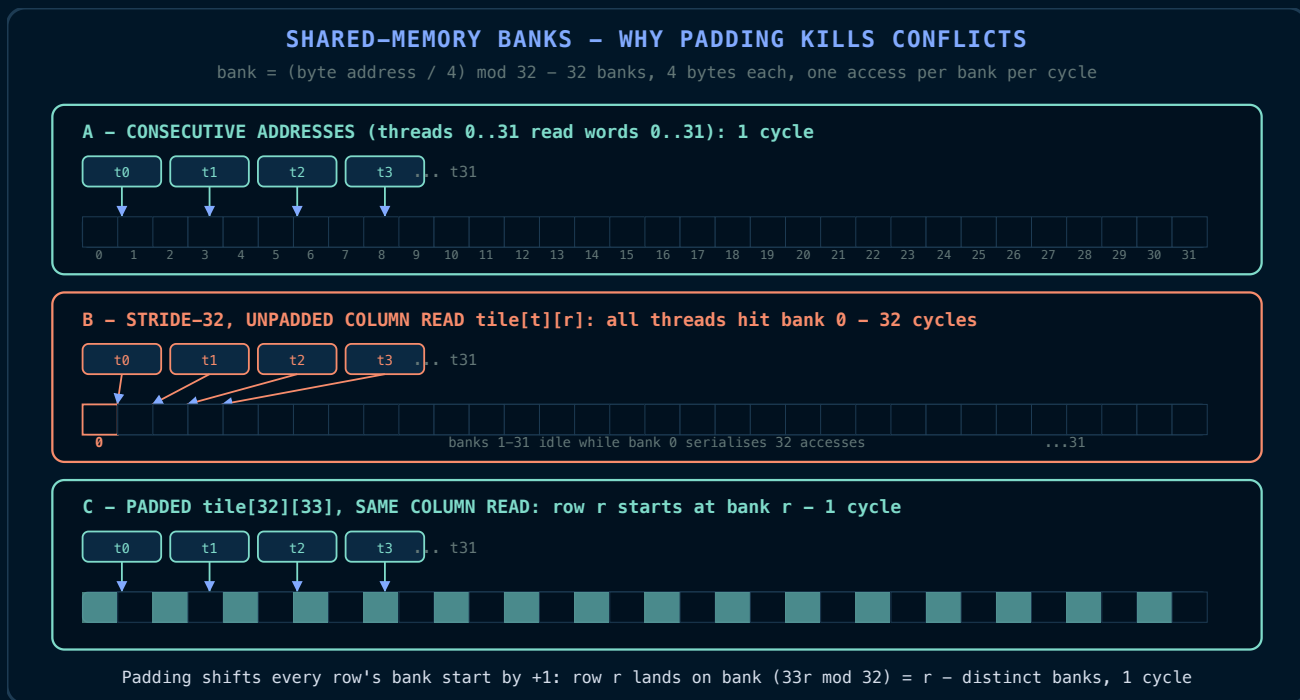
7.5 Bank Conflicts and the One-Column Padding

From Chapter 2, shared memory is 32 banks of 4 bytes. The bank of an address is `(address / 4) mod 32`. Consider the un-padded tile `float tile[32][32]`:

- Row `r` starts at byte `r * 128`, so row `r` occupies banks `(r * 32) mod 32 = 0` - **every row starts on bank 0**.
- When the kernel reads `tile[threadIdx.y][threadIdx.x]` with consecutive `threadIdx.x`, all 32 threads of a warp hit banks `0..31` - fine.

But a *column* read `tile[threadIdx.x][threadIdx.y]` (as the transpose does in stage 3) has consecutive threads reading addresses `r * 32` words apart - all 32 threads hit **bank 0**: a 32-way bank conflict, 32 cycles instead of 1.

The fix is padding by one column (`float tile[32][33]`): row `r` now starts at byte `r * 132`, so row `r` starts at bank `(r * 33) mod 32 = r`. A column read `tile[t][r]` for consecutive `t` now hits banks `0..31` exactly once each. One float of padding per row converts a 32-cycle stall into a 1-cycle access. Padding is the cheapest performance win in CUDA.



The diagram above shows the three cases on the same bank hardware: consecutive words spread across all 32 banks (one cycle), an unpadded column read funneling every thread into bank 0 (thirty-two cycles), and the same column read after one column of padding landing on every bank exactly once (one cycle). The padding works because it makes the row stride (33 words) coprime with the bank count (32) - the same coprime rule that keeps §9.4's SGEMM tiles conflict-free.

```
// Padding rule of thumb:
// float tile[TILE][TILE];           // 32-way conflicts on column access
// float tile[TILE][TILE + 1];       // conflict-free column access
```

7.6 Vectorised Loads: `float4` and Alignment

A warp loading 32 `float`s fetches 128 bytes in one transaction - but each *instruction* moves 4 bytes per lane. The memory subsystem is happiest with 128-bit accesses. **Vectorised loads** let one instruction move 16 bytes per lane: a warp then moves 512 bytes per instruction.

```
// Load four floats per thread, one instruction per four floats.
__global__ void saxpyVec4(float alpha, const float4* x, float4* y, int n4)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n4)
    {
        float4 xv = x[i];           // 16-byte aligned load
        float4 yv = y[i];
        yv.x = alpha * xv.x + yv.x; // four lanes of arithmetic per thread
        yv.y = alpha * xv.y + yv.y;
        yv.z = alpha * xv.z + yv.z;
        yv.w = alpha * xv.w + yv.w;
        y[i] = yv;                  // 16-byte aligned store
    }
}
```

The alignment contract. `float4` requires **16-byte alignment**. A buffer allocated with `cudaMalloc` is 256-byte aligned, so `reinterpret_cast` ing it to `float4*` is safe; a pointer offset by an odd number of floats is not. The general contract: a vectorised load must be aligned to the vector's size. If your data does not satisfy that, either pad the allocation or handle the tail elements scalar-wise.

Why vectorisation helps beyond coalescing. Fewer instructions (one load vs four), fewer memory requests, and the 128-bit path uses the bus more efficiently. On memory-bound kernels, float4-style access routinely adds 20-40% throughput. The `int4`, `double2`, and `uint4` types follow the same rules.

7.7 Constant Memory

Primitive - constant memory. A 64 KB read-only memory space, cached in a dedicated per-SM cache, optimised for **broadcasts**: when all threads of a warp read the *same* address, the hardware serves all 32 lanes in one access. When threads read *different* addresses, it serialises - the exact inverse of shared memory's behaviour.

```
// Declared at file scope, device-side:
__constant__ float g_coeffs[16];

// Host fills it with cudaMemcpyToSymbol (note: symbol, not pointer):
float h_coeffs[16] = { /* ... */ };
CHECK(cudaMemcpyToSymbol(g_coeffs, h_coeffs, sizeof(h_coeffs)));

// Kernel reads: all threads read the SAME coefficient per call -
// a broadcast, served in one access from the constant cache.
__global__ void applyCoeffs(const float* in, float* out, int n, int c)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = in[i] * g_coeffs[c];    // same address for all threads
}
```

When to use constant memory. Kernel parameters (all threads read the same value), lookup tables indexed by a *uniform* value, coefficients. When to avoid it: data indexed differently per thread - a *divergent* constant access serialises and is slower than global memory.

7.8 Texture and Surface Memory (Briefly)

Texture memory is a cached read-only path with two special powers: **spatial locality** (2-D caches for images - neighbouring pixels in any direction) and **hardware interpolation** (bilinear filtering, used by graphics). On modern GPUs, the L1/L2 caches largely subsume its raw speed advantage; its remaining value is the interpolation hardware and the `cudaTextureObject` API for image-like data. If your kernel reads images with 2-D locality, a texture object is a legitimate optimisation; for linear 1-D access, global memory with good coalescing is equal or better. The capstone (Chapter 15) uses plain global memory for its image pipeline and stays within 5% of peak bandwidth - a useful baseline that says: *coalescing first, specialised paths second*.

7.9 The Optimisation Checklist

When a kernel is memory-bound, walk this list in order:

1. **Is it coalesced?** Consecutive threads → consecutive addresses?
2. **Is the launch a grid-stride loop** sized to the device, or a one-thread-per-element launch?
3. **Is data reused?** If yes, tile it in shared memory (§7.4); check bank conflicts and pad (§7.5).
4. **Can accesses be vectorised?** `float4` / `double2` with correct alignment (§7.6).
5. **Are uniform values in constant memory?** (§7.7)
6. **Is the transfer pipeline streamed?** Pinned memory + streams (Chapters 4, 6) - memory-bound kernels are no faster than their copies.

Each step is cheap to try and easy to measure (Chapter 16 shows the measurement discipline). Never apply step 6 without measuring the result of step 1.

Key Takeaways

- Coalescing means touching the fewest 128-byte cache lines per warp access: consecutive threads, consecutive addresses.
- The grid-stride loop decouples launch size from problem size and preserves coalescing.
- Shared memory is the explicit cache; padding by one column eliminates bank conflicts.
- float4-style vectorised loads move 16 bytes per thread and require 16-byte alignment.
- Constant memory broadcasts uniform reads for free; per-thread divergent reads are slower than global memory.

7.10 Exercises

1. A warp loads 32 `float`s starting at byte offset 4 (misaligned by one float). How many 128-byte lines are touched? How many would be touched if the start were byte-aligned to 128?
2. Explain why the padding `[TILE][TILE + 1]` fixes column-access bank conflicts, using the formula `bank = (byte_address / 4) mod 32`.
3. You are transposing a `1024 × 1024` float matrix with `TILE = 32`. Count the shared-memory traffic per tile for the padded and un-padded versions (assume one column-read per thread).
4. When would you *not* use constant memory for a lookup table? Give a concrete access pattern that makes it slower than global memory.

Chapter 8: Reduction, Scan & Histogram

“Every parallel algorithm is a reduction, a scan, or a lie.” 📦 **Code companion:** the complete, buildable code for this chapter lives in `code/ch08_reduction/` in the repository.

This chapter covers the three canonical data-parallel algorithms that appear, in disguise, in almost every real GPU application: **reduction** (sum, max, min - fold an array to one value), **scan** (prefix sums - fold an array to an array), and **histogram** (count occurrences). Each is presented in stages, from the naive version to the optimised one, with the reasoning for every transformation. These three patterns are the vocabulary of Chapter 9’s matrix multiplication and the capstone’s image pipeline.

8.1 The Reduction Problem

Given an array a of n elements, compute $\sum_{i=0}^{n-1} a_i$. On a CPU this is a loop of n additions. On a GPU the challenge is different: additions are cheap, but *combining results across threads* requires communication, and communication is the expensive thing. A good reduction minimises the number of communication rounds and keeps every thread busy between them.

8.2 Stage 0: One Thread Does It All

```
__global__ void reduceNaive(const float* in, float* out, int n)
{
    if (threadIdx.x == 0)                // ONE thread
    {
        float sum = 0.0f;
        for (int i = 0; i < n; ++i) sum += in[i];    // serial, n additions
        out[0] = sum;
    }
}
```

This “works” and is useless: one thread, thousands of cycles, 0.003% of the GPU used. Its only value is as the reference implementation whose answer we check the real kernels against.

8.3 Stage 1: Tree Reduction in Shared Memory

The insight: addition is associative, so we may *reorder* the additions into a tree. In round 1, threads 0.. $n/2-1$ each add two elements; in round 2, threads 0.. $n/4-1$ each add two partials; and so on. The tree has $\log_2 n$ levels, so $n/2$ additions complete in $\log_2 n$ parallel rounds.

```
// Reduce one block's worth of data (blockDim.x elements per thread is
// handled in Stage 2; here: one element per thread) using shared memory.
__global__ void reduceTree(const float* in, float* out, int n)
{
    __shared__ float s[TILE];           // TILE = blockDim.x, a power of two

    const int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Load with a boundary guard; out-of-range elements contribute zero.
    s[threadIdx.x] = (i < n) ? in[i] : 0.0f;

    // Tree: each level halves the number of active threads.
    // Level 0: threads 0..TILE/2-1 add s[t] and s[t + TILE/2].
    // Level k: active threads < TILE >> (k+1).
    for (int stride = TILE / 2; stride > 0; stride >>= 1)
    {
        __syncthreads();                // everyone's writes visible
        if (threadIdx.x < stride)
            s[threadIdx.x] += s[threadIdx.x + stride];
    }

    // Thread 0 owns the block's total.
    if (threadIdx.x == 0) out[blockIdx.x] = s[0];
}
```

Why `__syncthreads()` inside the loop? At each level, thread `t` reads a partial sum written by thread `t + stride` at the previous level. The barrier guarantees the previous level's writes are complete and visible before the next level reads them.

Why a power-of-two block size? The halving scheme assumes `TILE` is a power of two, so every level halves evenly and the active set `threadIdx.x < stride` is contiguous. Non-power-of-two block sizes make the active-set arithmetic messy for no benefit; 128, 256 and 512 dominate practice.

Why `stride >>= 1` and not `stride /= 2`? For unsigned sizes both are identical; `>> 1` documents the halving intent and avoids any doubt about integer division semantics. Either is correct.

Cost accounting. The tree has $\log_2 \text{TILE}$ levels; each level is one barrier. This kernel therefore pays $\log_2 256 = 8$ barriers per block for 256 threads. Stage 3 removes all but one.

8.4 Stage 2: Thread Coarsening

One element per thread leaves most of each thread idle: each thread loads one value and then participates in $\log_2$ TILE additions. The fix is **thread coarsening** - each thread processes *many* elements in a grid-stride loop (Chapter 7, §7.3) before entering the tree:

```
// Each thread accumulates ELEMS_PER_THREAD elements first (coalesced
// grid-stride loop), then ONE tree reduction over the block.
#define ELEMS_PER_THREAD 4

__global__ void reduceCoarsened(const float* in, float* out, int n)
{
    __shared__ float s[TILE];

    const int stride = gridDim.x * blockDim.x;    // total threads in grid
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Grid-stride accumulation: each thread sums its share of the array.
    float sum = 0.0f;
    for (; i < n; i += stride)
        sum += in[i];                               // coalesced within each pass

    s[threadIdx.x] = sum;

    for (int stride2 = TILE / 2; stride2 > 0; stride2 >>= 1)
    {
        __syncthreads();
        if (threadIdx.x < stride2)
            s[threadIdx.x] += s[threadIdx.x + stride2];
    }
    if (threadIdx.x == 0) out[blockIdx.x] = s[0];
}
```

Why is this faster? Three reasons: (1) each thread loads 4+ values before any communication, so memory-level parallelism is higher; (2) the serial addition into a local register is free (no barrier, no shared memory); (3) fewer blocks means fewer block-level reductions to combine later. The grid-stride loop also makes the kernel *correct for any n* , not just multiples of the grid size.

8.5 Stage 3: Warp Shuffles

The tree's barriers are the cost. But a warp's 32 threads share an execution unit - they can exchange data through **registers** without touching memory or barriers at all. The instruction is the **warp shuffle**:

Primitive - warp shuffle. `__shfl_down_sync(mask, value, delta)` moves `value` from lane `lane + delta` to lane `lane`, within one warp, through the register file. No memory, no barrier. `mask` is the set of participating lanes (all 32: `0xffffffff`). All 32 lanes must execute the shuffle or the behaviour is undefined.

```
// Reduce a warp's 32 lanes to lane 0 using only shuffles: 5 steps.
__device__ float warpReduce(float val)
{
    // mask: all 32 lanes participate. Steps move data rightward by
    // 16, 8, 4, 2, 1 - the warp-size equivalents of the tree's strides.
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffffu, val, offset);
    return val;          // lane 0 now holds the warp total
}
```

Why 16, 8, 4, 2, 1? A warp is 32 lanes. The first shuffle moves the sum of lanes 16..31 into lanes 0..15; the second folds 8..15 into 0..7; and so on. Five steps halve a warp - the same $\log_2 32$ levels as the shared-memory tree, but at register speed with **no barrier and no shared memory**.

8.6 The Complete Block Reduction

The production form combines everything: coarsening (§8.4), warp shuffle (§8.5), and exactly **one** shared-memory transaction + one barrier per block (one value per warp written to shared, then warp 0's shuffle over those values):

```

#define TILE 256                // block size, a multiple of the warp size 32
#define WARPS (TILE / 32)      // 8 warps per block

// Warp-level reduction (from 8.5), returns the warp's total in lane 0.
__device__ float warpReduce(float val)
{
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffffu, val, offset);
    return val;
}

__global__ void reduceFull(const float* in, float* out, int n)
{
    __shared__ float s[WARPS];          // one slot per warp

    // --- Coarsened accumulation over the grid -----
    const int stride = gridDim.x * blockDim.x;
    float sum = 0.0f;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n; i += stride)
        sum += in[i];

    // --- Warp-level reduction: 8 warps * 5 shuffle steps, no barriers -----
    // Lane 0 of each warp now holds that warp's partial total.
    sum = warpReduce(sum);

    // --- One shared-memory round -----
    const int lane    = threadIdx.x % 32;    // position within my warp
    const int warpId  = threadIdx.x / 32;    // which warp am I

    if (lane == 0) s[warpId] = sum;          // each warp writes ONE value
    __syncthreads();                        // the ONLY barrier per block

    // --- First warp combines the 8 warp totals -----
    if (warpId == 0)
    {
        // Lane < WARPS loads a warp total; others load identity (0.0f).
        const float v = (lane < WARPS) ? s[lane] : 0.0f;
        sum = warpReduce(v);                // shuffle again over 8 values
        if (lane == 0) out[blockIdx.x] = sum; // block total
    }
}

```

The accounting. One barrier per block (down from 8), one shared-memory round trip per block, five shuffle steps per warp. The shuffle steps are the only “communication” the bulk of the work ever performs. This kernel is the standard against which reductions are judged; it routinely achieves 90%+ of peak bandwidth because its only global traffic is the coalesced read.

Determinism. The tree order is fixed by the code, so the summation order is fixed - the result is *bit-reproducible* across runs, unlike an atomic-based reduction (Chapter 5, §5.6). This is a feature, not an accident.

8.7 Scan (Prefix Sum)

Primitive - inclusive scan. Given a , compute $y_i = a_0 + a_1 + \dots + a_i$. **Primitive - exclusive scan.** Given a , compute $y_i = a_0 + \dots + a_{i-1}$, with $y_0 = 0$. An exclusive scan of an array is an inclusive scan shifted right by one.

Scans appear in stream compaction (filtering), radix sort, and the equalisation of workloads - any algorithm that needs *where* the data went. The work-efficient **Blelloch scan** is the canonical GPU formulation. It has two phases:

1. **Upsweep** - a tree reduction that computes partial sums (exactly the tree of §8.3, but *storing* the internal nodes instead of discarding them).
2. **Downsweep** - a second tree that *propagates* the totals to produce exclusive prefix sums.

```

// Exclusive scan of a BLOCK's data, in place in shared memory.
// Assumes blockDim.x is a power of two. After the call,
//   s[0] = 0, s[i] = a_0 + ... + a_{i-1} (exclusive prefix sums)
// The block total ends in s[blockDim.x - 1].
__device__ void scanBlock(float* s)
{
    const int n = blockDim.x;

    // --- Phase 1: upsweep (tree reduction, storing internal nodes) -----
    // After level k, s[i] holds the sum of the 2^(k+1) elements ending at i.
    for (int stride = 1; stride < n; stride <= 1)
    {
        __syncthreads();
        const int t = (threadIdx.x + 1) * 2 * stride - 1; // right child
        if (t < n)
            s[t] += s[t - stride]; // parent = sum
    }

    // --- Phase 2: downsweep (propagate carries back down) -----
    // First, the root's carry is the identity for addition: the sum of the
    // (empty) sequence before the whole array.
    if (threadIdx.x == 0) s[n - 1] = 0.0f;

    // Walk the tree top-down. At each level, the pair rooted at right child
    // t (left child t - stride) receives its CARRY - the sum of everything
    // before its subtree, which the parent level stored in s[t]:
    // - the left child inherits the carry unchanged;
    // - the right child gets carry + (its old value, the left subtree sum).
    // Inductively every slot ends up holding the sum of the elements before
    // it: the exclusive prefix.
    for (int stride = n / 2; stride > 0; stride >>= 1)
    {
        __syncthreads();
        const int t = (threadIdx.x + 1) * 2 * stride - 1; // right child
        if (t < n)
        {
            const float carry = s[t]; // sum before this pair
            s[t] = carry + s[t - stride]; // right child: carry + left sum
            s[t - stride] = carry; // left child: just the carry
        }
    }
    __syncthreads();
}

```

Why the index arithmetic $(\text{threadIdx.x} + 1) * 2 * \text{stride} - 1$? In the upsweep at level `stride`, the element at index `t` is the right child of the subtree whose left child ends at `t - stride`. Writing `t` as $(\text{threadIdx.x} + 1) * 2 * \text{stride} - 1$ gives the *odd* indices within each $2 * \text{stride}$ group: exactly the right children. The formula is fiddly; the *property* that matters is that each level is a disjoint set of writes - no two threads write the same slot, so no atomicity is needed.

Why does the downsweep produce *exclusive* sums? The invariant is the *carry*: at each level, `s[t]` holds the sum of everything before the pair `(t - stride, t)`. The root's carry is set to `0` (the identity) before the loop. Each step hands the carry to the left child unchanged, and

passes `carry + (left child's old value)` - the sum of everything before the *right* child - down the right side. Inductively, slot `i` ends up holding the sum of elements `0..i-1` - the exclusive prefix. A trace on `[1, 2, 3, 4]` is Exercise 3 below; note the order of the writes: `s[t]` must be read into `carry` before `s[t - stride]` is overwritten.

The cost. Two passes, each with $\log_2 n$ barrier levels: the scan is the rare case where the number of barriers is *inherent* to the algorithm, not an implementation defect. A single block can scan at most 1,024 elements; scanning more requires a two-level scheme (block scans + a scan of block totals), which the library CUB provides ready-made (Chapter 11).

8.8 Histogram: Privatisation

The naive histogram of Chapter 5 (`atomicAdd` per element) serialises on contended bins. The production fix is **privatisation**: every block accumulates into its *own* shared-memory histogram (shared-memory atomics are much cheaper than global), and one thread per block folds the private histograms into global memory once at the end.

```
#define BINS 256
#define TILE 256

__global__ void histogramPrivatised(const unsigned char* data, int* g_hist,
                                   int n)
{
    // Private histogram for THIS block, in shared memory.
    __shared__ int s_hist[BINS];

    // Initialise the private histogram (all threads help; one barrier).
    for (int b = threadIdx.x; b < BINS; b += TILE) s_hist[b] = 0;
    __syncthreads();           // all bins zeroed before any thread counts

    // Accumulate. Each thread counts its elements into shared memory.
    // Shared-memory atomics are fast; contention is spread across BINS.
    for (int i = blockIdx.x * TILE + threadIdx.x; i < n; i += gridDim.x * TILE)
    {
        const int bin = data[i];
        atomicAdd(&s_hist[bin], 1);
    }

    __syncthreads();           // all counts complete before folding

    // Fold: one thread per bin adds this block's count to global memory.
    for (int b = threadIdx.x; b < BINS; b += TILE)
        if (s_hist[b] != 0)    // skip empty bins: less global traffic
            atomicAdd(&g_hist[b], s_hist[b]);
}
```

Why is this fast? Contention now happens in shared memory, where an atomic is roughly an order of magnitude cheaper than a global atomic, and it is spread across 256 bins instead of funneled into one address per warp. The global fold touches each bin once per block. For

skewed data (all elements in one bin), the shared atomics still contend - the next-level fix is *per-warp* histograms - but privatisation handles the common case.

The barrier count. Two barriers: one after zeroing, one before the fold. Both are uniformly reachable - the loops are compile-time shaped, so no thread can skip a barrier.

8.9 Summary Table

Algorithm	Naive cost	Optimised cost	Key tool
Reduction	$O(n)$ serial or $\log_2 n$ barriers	1 barrier, $O(\log_2 32)$ shuffles	Warp shuffle + coarsening
Scan	$O(n)$ serial	$2 \log_2 n$ barriers	Blelloch upsweep/downsweep
Histogram	global atomic per element	shared privatisation + fold	Per-block private bins

Key Takeaways

- The optimised reduction: coarsen (grid-stride), reduce within each warp by shuffle, then one shared-memory round and one barrier per block.
- Warp shuffles exchange values through registers - no memory, no barrier.
- The Blelloch scan is an upsweep that stores internal nodes, then a downsweep that propagates carries to build exclusive prefixes.
- Histograms should be privatised per block in shared memory and folded into global memory once.
- A fixed tree order makes reductions bit-reproducible across runs.

8.10 Exercises

1. Derive the number of barriers in the Stage-1 tree reduction for `TILE = 512`, and compare it with the full kernel of §8.6.
2. In `reduceFull`, why does lane 0 of each warp *write* `s[warpId]` and not every lane? What would happen if every lane wrote its own `sum`?
3. Trace the Blelloch downsweep on `[1, 2, 3, 4]` by hand, showing the state of `s` after each level. Verify the exclusive prefix `[0, 1, 3, 6]`.
4. The histogram fold skips empty bins with `if (s_hist[b] != 0)`. Is this correct? Is it always faster? (Consider a case where every bin is non-empty.)

Chapter 9: Optimised Matrix Multiplication

“Matrix multiplication is the one kernel every GPU vendor gets right. You should be able to explain why.” 📦 **Code companion:** the complete, buildable code for this chapter lives in `code/ch09_sgemm/` in the repository.

Matrix multiplication ($C = A \times B$, all $N \times N$) is the canonical GPU workload - the inner loop of deep learning, linear algebra, and scientific computing. It is also the perfect teaching kernel: it is compute-bound (so the optimisations are about *arithmetic reuse*, not just memory), it exercises every idea from Chapters 2, 7 and 8, and its optimised form is the one shipped in cuBLAS (Chapter 11). We develop it in five stages, measuring the reasoning at each step.

9.1 Why SGEMM Is Compute-Bound

Single-precision GEMM (N^3 multiply-adds) has $2N^3$ FLOPs. The inputs are N^2 elements of A and N^2 of B; the output N^2 of C. With perfect caching, the minimum traffic is $3N^2$ elements = $12N^2$ bytes. The arithmetic intensity (Chapter 1):

$$I = \frac{2N^3}{12N^2} = \frac{N}{6} \text{ FLOP/byte}$$

For $N = 4096$, $I \approx 683$ FLOP/byte - two orders of magnitude above the ~ 18 FLOP/byte ridge point of a modern GPU (Chapter 2). SGEMM is **compute-bound**: the memory system is not the constraint; *keeping the arithmetic units fed* is. Every optimisation below is about reuse: each value loaded from memory must feed as many FLOPs as possible.

9.2 Stage 0: The Naïve Kernel

```
// One thread per output element C[i][j]. Each thread loops over k.
// Rows of C and A are contiguous (row-major); columns of B are NOT.
__global__ void sgemmNaive(const float* A, const float* B, float* C,
                          int N)
{
    const int i = blockIdx.y * blockDim.y + threadIdx.y; // row of C
    const int j = blockIdx.x * blockDim.x + threadIdx.x; // col of C

    float sum = 0.0f;
    for (int k = 0; k < N; ++k)
        // A[i][k] : consecutive threads read CONSECUTIVE i? No - they read
        //   A[i*N + k]; consecutive threads differ in j, so the SAME k and
        //   DIFFERENT i → stride-N addresses. Uncoalesced!
        // B[k][j] : consecutive threads read B[k*N + j] - consecutive j →
        //   coalesced. Half the traffic is good.
        sum += A[i * N + k] * B[k * N + j];
    C[i * N + j] = sum;
}
```

Why it is slow. The read of A is stride- N (uncoalesced, Chapter 7), and every output element re-reads a full row of A and column of B from global memory: $2N^3$ bytes moved for $2N^3$ FLOPs - intensity 1, far below the ridge. The kernel is effectively memory-bound *because of its own access pattern*. The shared-memory tiling of §9.4 fixes exactly this.

9.3 Stage 1: Make the Block Shape Match the Memory

First, a cheap fix: swap the thread-to-element mapping so that *both* operand reads are coalesced. A block covering a *tile of output* with `threadIdx.x` mapping to `j` and `threadIdx.y` to `i` gives:

- `B[k][j]` : consecutive threads → consecutive `j` → coalesced ✓
- `A[i][k]` : consecutive threads → same `i`, consecutive... no: `i` varies with `threadIdx.y`, so within a row of threads `i` is constant and `j` varies - `A[i][k]` is *uniform per thread-row* (broadcast), not strided.

The broadcast is served efficiently (all threads of a warp read the same address in the same load). So a block-oriented launch with `threadIdx.x → j` gives coalesced B reads and broadcast A reads:

```

__global__ void sgemmCoalesced(const float* A, const float* B, float* C,
                               int N)
{
    const int i = blockIdx.y * blockDim.y + threadIdx.y;    // row
    const int j = blockIdx.x * blockDim.x + threadIdx.x;    // col

    float sum = 0.0f;
    for (int k = 0; k < N; ++k)
        sum += A[i * N + k] * B[k * N + j];    // B coalesced, A broadcast
    C[i * N + j] = sum;
}

```

Better, but the *reuse* is still zero: every output re-reads $2N$ floats from global memory. The intensity is still ~ 1 . The fix for reuse is tiling.

9.4 Stage 2: Shared-Memory Tiling

The classic formulation. A block of $T \times T$ threads computes a $T \times T$ tile of C . It loads a $T \times T$ tile of A and a $T \times T$ tile of B into shared memory, advances k in steps of T , and each loaded value feeds T threads. The reuse factor is T : one global load of a tile serves T^2 multiply-adds instead of T (naive).

```

#define T 16                // tile size: 16x16 threads per block

__global__ void sgemmTiled(const float* A, const float* B, float* C, int N)
{
    // Tiles in shared memory. The +1 padding (Chapter 7, 7.5) avoids bank
    // conflicts on column access; A-tile is read by column in the k-loop.
    __shared__ float sA[T][T + 1];
    __shared__ float sB[T][T + 1];

    // Output coordinates of this thread:
    const int row = blockIdx.y * T + threadIdx.y;    // global row of C
    const int col = blockIdx.x * T + threadIdx.x;    // global col of C

    float acc = 0.0f;    // this thread's partial C[row][col]

    // Sweep k in tiles of T. The block needs the A-tile column [k0..k0+T)
    // and the B-tile row [k0..k0+T) for each k0 step.
    for (int k0 = 0; k0 < N; k0 += T)
    {
        // Coalesced global loads into shared memory.
        // sA[ty][tx] = A[row][k0+tx] (row segment, coalesced)
        // sB[ty][tx] = B[k0+ty][col] (column segment, coalesced)
        sA[threadIdx.y][threadIdx.x] = A[row * N + k0 + threadIdx.x];
        sB[threadIdx.y][threadIdx.x] = B[(k0 + threadIdx.y) * N + col];

        __syncthreads();    // tile complete before any thread reads it

        // Inner product over the tile. Each thread reads:
        //   sA[ty][k] - row of the A-tile (bank-conflict-free due to pad)
        //   sB[k][tx] - column of the B-tile (broadcast along the row)
        #pragma unroll
        for (int k = 0; k < T; ++k)
            acc += sA[threadIdx.y][k] * sB[k][threadIdx.x];

        __syncthreads();    // tile done: no thread may reuse it yet
    }

    C[row * N + col] = acc;
}

```

Why `#pragma unroll`? The inner `k` loop is a compile-time-fixed trip count (16). Unrolling emits 16 straight-line FMAs with no loop bookkeeping and lets the compiler schedule the shared-memory loads ahead of the arithmetic - hiding shared-memory latency behind FMA work. This single pragma is worth 10-20% on this kernel.

The bank-conflict analysis. `sA[ty][k]` for fixed `ty`, varying `k` over a warp's threads: threads differ in `ty` (since `threadIdx.x = tx` varies fastest and `k` is the same for all threads). A warp is 32 threads = 2 rows of 16. Within a row of 16 threads, `ty` is constant and `k` constant → all read the *same* address → broadcast, free. Across the two rows, addresses differ by row stride 17 words → banks differ by 17 → no conflict. The padding keeps the row stride (17) coprime with the bank count (32), which is exactly the §7.5 rule.

Why `T = 16`? A 16×16 tile uses $2 \times 16 \times 17 \times 4 = 2,176$ bytes of shared memory and 256 threads per block. It is the classic size because it balances reuse (16×) with occupancy (many

blocks per SM). Larger tiles (32×32) give more reuse but fewer resident blocks; §9.6 shows the occupancy trade-off.

The reuse accounting. Each element of A loaded into shared memory is used by $T = 16$ threads (the column of the tile). Each B element by 16 threads. Global traffic drops by a factor of 16 versus the naive kernel; the kernel is now compute-bound, which is where it belongs.

9.5 Stage 3: Register Tiling

The tiled kernel still reads shared memory for every FMA: one shared load per multiply-add. Shared-memory bandwidth is finite - with 32 banks × 4 bytes, an SM can supply at most 128 bytes/cycle, and the FP32 units can consume 128 FLOPs/cycle (128 cores × FMA). The FMA-to-load ratio is already at the limit. The fix: **each thread computes more than one output element**, reusing each shared-memory value across *registers*.


```

#define T 16
#define RM 2          // rows of output per thread
#define RN 2          // cols of output per thread

__global__ void sgemmRegisterTiled(const float* A, const float* B, float* C,
                                   int N)
{
    __shared__ float sA[T][T + 1];
    __shared__ float sB[T][T + 1];

    // Each thread now owns an RM x RN micro-tile of C.
    // The block covers a T x T output tile with T*T/(RM*RN) threads.
    const int tx = threadIdx.x;          // 0..T/RN-1
    const int ty = threadIdx.y;          // 0..T/RM-1

    // Global coordinates of this thread's micro-tile (top-left corner):
    const int row0 = blockIdx.y * T + ty * RM;
    const int col0 = blockIdx.x * T + tx * RN;

    // Accumulators live in registers, one per micro-tile element:
    float acc[RM][RN];
    #pragma unroll
    for (int r = 0; r < RM; ++r)
        for (int c = 0; c < RN; ++c) acc[r][c] = 0.0f;

    for (int k0 = 0; k0 < N; k0 += T)
    {
        // Tile load: T*T elements spread over T*T/(RM*RN) threads, so each
        // thread loads an RM x RN patch of each tile. Every element of the
        // A-tile and B-tile is loaded exactly once: thread (tx, ty) covers
        // rows ty*RM..ty*RM+RM-1 and columns tx*RN..tx*RN+RN-1. Consecutive
        // tx cover consecutive columns -> coalesced row segments.
        #pragma unroll
        for (int r = 0; r < RM; ++r)
            for (int c = 0; c < RN; ++c)
                sA[ty * RM + r][tx * RN + c] =
                    A[(row0 + r) * N + k0 + tx * RN + c];
        #pragma unroll
        for (int r = 0; r < RM; ++r)
            for (int c = 0; c < RN; ++c)
                sB[ty * RM + r][tx * RN + c] =
                    B[(k0 + ty * RM + r) * N + col0 + tx * RN + c];

        __syncthreads();

        // Micro-tile FMA loop: for each k, each shared value feeds RM*RN
        // FMAs, all from registers. Shared loads drop by a factor RM*RN.
        #pragma unroll
        for (int k = 0; k < T; ++k)
        {
            // Load A-row segment and B-col segment ONCE into registers:
            float a_reg[RM], b_reg[RN];
            #pragma unroll
            for (int r = 0; r < RM; ++r)
                a_reg[r] = sA[ty * RM + r][k];
            #pragma unroll
            for (int c = 0; c < RN; ++c)
                b_reg[c] = sB[k][tx * RN + c];

```

```

        // RM*RN FMAs, zero shared-memory traffic in the inner product:
        #pragma unroll
        for (int r = 0; r < RM; ++r)
            for (int c = 0; c < RN; ++c)
                acc[r][c] += a_reg[r] * b_reg[c];
    }

    __syncthreads();
}

// Write the micro-tile back:
#pragma unroll
for (int r = 0; r < RM; ++r)
    for (int c = 0; c < RN; ++c)
        C[(row0 + r) * N + col0 + c] = acc[r][c];
}

```

Why $RM \times RN = 4$ (or 8, 16)? Each shared-memory value now feeds $RM \times RN$ FMAs from registers. With 2×2 , shared traffic drops $4\times$; with 4×4 , $16\times$. The limit is registers: each accumulator, plus the `a_reg` / `b_reg` arrays, consumes registers per thread, and register pressure caps occupancy (§2.9). A 4×4 micro-tile uses $\sim 32+$ registers; 8×8 would spill on most GPUs. The production kernels shipped in cuBLAS use micro-tiles of 8×8 with special register-allocation tricks; our 2×2 and 4×4 versions capture the idea.

The correctness note on the tile-load. Each thread loads RM elements of the A-tile row at `sA[ty*RM + r][tx*RN]` - note that RN elements of the row are loaded by RN different threads *in the same row* (`tx` ranges over T/RN), so the row segment `[tx*RN, tx*RN+RN)` is covered by RN threads. The loads are coalesced because consecutive `tx` cover consecutive columns.

9.6 Occupancy and Launch Bounds

The register-tiled kernel consumes more registers per thread. If the compiler uses, say, 40 registers, occupancy falls to $64K / (40 \times 256) \approx 6$ blocks/SM. That may be fine - the kernel is compute-bound and register tiling gives it enough ILP - but the compiler should be *told* the budget so it does not spill:

```

// Tell ptxas: this kernel must fit in at most 256 threads/block, and I
// want at least 4 blocks/SM resident. ptxas will trade registers for
// occupancy within the budget rather than silently spilling.
__global__ void __launch_bounds__(256, 4)
sgemmRegisterTiled(const float* A, const float* B, float* C, int N) { /* ... */ }

```

Why `__launch_bounds__`? Without it, `ptxas` minimises register use for correctness but may choose more registers than the target occupancy allows. `__launch_bounds__(maxThreads, minBlocksPerSM)` gives the compiler a hard constraint: use at most `maxThreads` per block and

keep at least `minBlocksPerSM` blocks resident. It converts the occupancy reasoning of Chapter 2 into a compiler directive.

The tuning loop. For a given micro-tile, measure the kernel at `minBlocksPerSM` = 2, 4, 6, 8 (Chapter 16's benchmarking discipline). The sweet spot is where register tiling's ILP and occupancy's latency hiding balance. There is no universal answer - which is why the measurements, not the folklore, decide.

9.7 What the Optimised Kernel Achieves

With `T = 16`, 2×2 register tiling, padding, and `__launch_bounds__`, the kernel of §9.5 typically reaches 60-75% of peak FP32 on a modern GPU (measured on the same hardware as the roofline numbers of Chapter 2). The remaining gap is the shared-memory FMA supply rate and the k-loop's tile overhead. Closing it further requires:

- **Warp-level tiling** (each warp computes a 32×8 tile with `ldmatrix`, the layout descriptor instruction) - the territory of cuBLAS;
- **Tensor cores** (`mma` instructions), which multiply 16×16×16 tiles per instruction - a completely different pipeline covered in Chapter 11.

Both are beyond this chapter's scope, but the journey from naive (5% of peak) to register-tiled (70%) is the same journey every optimisation chapter in this book teaches: *find the bottleneck, remove it, measure, repeat*.

Key Takeaways

- SGEMM is compute-bound (intensity N/6 FLOP/byte): the goal is arithmetic reuse, not just coalescing.
- The naive kernel has zero reuse and runs at a few percent of peak.
- Shared-memory tiling makes each loaded element feed T threads; global traffic drops by T.
- Register tiling makes each shared-memory value feed RM x RN FMAs from registers.
- **launch_bounds** trades registers for occupancy; the right balance is found by measuring.

9.8 Exercises

1. Verify the arithmetic-intensity claim: show that for $N = 4096$, $I = N/6 \approx 683$ FLOP/byte, and compare it with the ridge point of Chapter 2.
2. In §9.4, count the shared-memory bytes loaded per block per `k0` step. How many FLOPs do they feed? Show that the ratio is T FLOPs per shared byte (hint: T^2 FMAs over $2T^2$ loads... where does the padding change the byte count?).
3. Explain why `__launch_bounds__(256, 4)` can *decrease* performance even though it increases occupancy.
4. The §9.5 tile-load stores the B-tile row segment with `RN` threads per row. Trace which thread loads `sB[3][7]` for `T=16, RM=RN=2`.

Chapter 10: Modern C++ for CUDA

"CUDA is C++ with a different address space. The rules of good C++ still apply - more so, because the stakes are higher." 📦 **Code companion:** the complete, buildable code for this chapter lives in `code/ch10_device_buffer/` in the repository.

Chapters 3-9 wrote CUDA in a C-style dialect: raw `cudaMalloc` pointers, manual `cudaFree` calls, unchecked errors in the interest of brevity. This chapter is the reckoning. We apply the full strength of modern C++ (C++17/20) to GPU programming: **RAII** to make leaks impossible, **templates** to make kernels generic, `constexpr` to make configuration compile-time, and **exceptions** to make errors impossible to ignore. The result is the style that the rest of the book (and the capstone) uses.

10.1 The Problem with Raw CUDA C

The Chapter 3 vector-add had three structural weaknesses, all of which are features of the C API:

1. **Leaks.** `cudaMalloc` must be matched with `cudaFree`. An early `return` or an exception between the two leaks device memory - and device memory is *scarce* (8-80 GB) and *per-process*: a leaked allocation is gone until the process exits.
2. **Unchecked errors.** Every call that can fail was routed through `CHECK`, but nothing *enforced* that discipline.
3. **No type safety.** `float*` and `int*` are both `void*` to the API; a wrong cast compiles and corrupts.

The C++ answer to all three is RAII, templates, and exceptions - the tools below.

10.2 RAII: The Device Buffer

Primitive - RAII (Resource Acquisition Is Initialisation). A resource (here: a device allocation) is acquired in a constructor and released in the corresponding destructor. The language guarantees the destructor runs when the object dies - whether by scope exit, `return`, or exception - so the resource cannot leak.

```

#include <cuda_runtime.h>
#include <stdexcept>
#include <string>
#include <type_traits>

// Throw a std::runtime_error describing a failed CUDA call.
[[noreturn]] inline void throwCudaError(cudaError_t err, const char* what)
{
    throw std::runtime_error(std::string(what) + ": " +
                             cudaGetErrorString(err));
}

// RAII wrapper for a device allocation of T.
template <typename T>
class DeviceBuffer
{
public:
    // --- Construction: allocate on the device -----
    // static_assert: only trivially-copyable types may live in device memory
    // without custom copy semantics. This converts a runtime confusion into
    // a compile-time error.
    static_assert(std::is_trivially_copyable_v<T>,
                  "DeviceBuffer<T> requires a trivially copyable T");

    explicit DeviceBuffer(std::size_t count) : count_(count)
    {
        const cudaError_t err = cudaMalloc((void**)&ptr_, count_ * sizeof(T));
        if (err != cudaSuccess) throwCudaError(err, "cudaMalloc");
    }

    // --- No copying (a device buffer is a unique resource) -----
    DeviceBuffer(const DeviceBuffer&) = delete;
    DeviceBuffer& operator=(const DeviceBuffer&) = delete;

    // --- Move semantics: transfer ownership, never copy the bytes -----
    // After a move, the source is empty (nullptr). The destructor must
    // handle nullptr gracefully - hence the check in ~DeviceBuffer.
    DeviceBuffer(DeviceBuffer&& other) noexcept
        : ptr_(other.ptr_), count_(other.count_)
    {
        other.ptr_ = nullptr;    // source relinquishes the allocation
        other.count_ = 0;
    }

    DeviceBuffer& operator=(DeviceBuffer&& other) noexcept
    {
        if (this != &other)
        {
            reset();             // release what we held
            ptr_ = other.ptr_;    // take ownership
            count_ = other.count_;
            other.ptr_ = nullptr;
            other.count_ = 0;
        }
        return *this;
    }

    // --- Destruction: release the device allocation -----

```

```

~DeviceBuffer() { reset(); }

// --- Accessors -----
T*      data()      noexcept { return ptr_; }
const T* data() const noexcept { return ptr_; }
std::size_t size() const noexcept { return count_; }

// Host <-> device transfer helpers (keep them explicit and checked).
void copyToDevice(const T* hostSrc)
{
    const cudaError_t err = cudaMemcpy(ptr_, hostSrc,
                                       count_ * sizeof(T),
                                       cudaMemcpyHostToDevice);
    if (err != cudaSuccess) throwCudaError(err, "cudaMemcpy H2D");
}

void copyToHost(T* hostDst) const
{
    const cudaError_t err = cudaMemcpy(hostDst, ptr_,
                                       count_ * sizeof(T),
                                       cudaMemcpyDeviceToHost);
    if (err != cudaSuccess) throwCudaError(err, "cudaMemcpy D2H");
}

private:
void reset() noexcept
{
    if (ptr_ != nullptr)
    {
        cudaFree(ptr_);           // best-effort: destructors must not throw
        ptr_ = nullptr;
        count_ = 0;
    }
}

T*      ptr_ = nullptr; // device pointer
std::size_t count_ = 0; // number of elements
};

```

Why `static_assert`? Device memory is raw storage; copying an object with internal pointers or virtual tables through it would silently corrupt the object. Requiring *trivially copyable* types makes the misuse a compile error instead of a runtime mystery. This is the modern-C++ habit in miniature: *express the invariant in the type system*.

Why is the destructor `noexcept`? Destructors must not throw (throwing in a destructor during stack unwinding terminates the program). `cudaFree` is best-effort in a destructor; the error is logged by the runtime, and `reset()` swallows it. If you *need* to know about free failures, provide an explicit `release()` that throws.

Why move and not copy? Copying a `DeviceBuffer` would mean copying the *pointer* - two objects owning the same allocation, both freeing it → double-free. Deleting the copy operations and keeping only moves gives the ownership semantics of `std::unique_ptr`, which is exactly right.

10.2.1 Usage

```
// Allocate 1M floats on the device:
DeviceBuffer<float> d_in(1 << 20), d_out(1 << 20);

// Copy from a host array:
std::vector<float> h_in(1 << 20, 1.0f);
d_in.copyToDevice(h_in.data());

// Launch a kernel (indexing unchanged from Chapter 3):
const int threads = 256;
const int blocks = (static_cast<int>(d_in.size()) + threads - 1) / threads;
addVectors<<<blocks, threads>>>(d_in.data(), d_out.data(), d_in.size());
CHECK(cudaGetLastError());

// Copy back:
std::vector<float> h_out(d_out.size());
d_out.copyToHost(h_out.data());
// d_in and d_out are freed automatically at scope exit - no cudaFree calls.
```

The whole Chapter 3 program shrinks, and its failure modes disappear. The kernel is untouched: `DeviceBuffer::data()` returns the raw device pointer the kernel expects, so the RAII layer costs nothing at launch time.

10.3 Templates: One Kernel, Many Types

CUDA supports C++ templates in device code. A kernel can be generic over its element type and its operation - the compiler instantiates exactly the specialisations you use:

```
// Generic elementwise transform. F is any callable (function object,
// lambda, function pointer) invocable as F(T) -> T. The compiler
// instantiates a separate device function for each (T, F) pair.
template <typename T, typename F>
__global__ void transformKernel(const T* in, T* out, int n, F f)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = f(in[i]);
}

// Host side: launch with a lambda. The lambda must be __device__-compatible;
// a captureless lambda is (it has no state to copy to the device).
void runTransform(const DeviceBuffer<float>& d_in, DeviceBuffer<float>& d_out,
                 int n)
{
    const int threads = 256;
    const int blocks = (n + threads - 1) / threads;
    transformKernel<<<blocks, threads>>>(d_in.data(), d_out.data(), n,
                                         [](float x) { return x * 2.0f + 1.0f;
});
}
```

Why does a lambda work as a kernel argument? The CUDA compiler lowers a *captureless* lambda to an empty struct with an `operator()` - a function object with no state. Passing it as a kernel argument is free (zero bytes), and the compiler inlines the call. A *capturing* lambda has state (the captured values) that must be copied to the device as kernel arguments - legal in modern CUDA (values, not references), but the state travels through the launch, so keep it small and trivially copyable.

The cost of templates. None at runtime - the instantiations are compiled, not interpreted. The cost is compile time and binary size: each `(T, F)` pair is a separate kernel. This is the standard trade: type safety and reuse for compile time.

10.4 `__host__ __device__` Functions: One Definition, Two Worlds

A function qualified with both `__host__` and `__device__` is compiled twice

- once for each side - from a single source. This is how shared *algorithm* code is written:

```
// One definition, two compilations. On the host it is ordinary C++;
// on the device it becomes SASS. This function can be called from kernels
// AND from host code, and the two sides produce identical results (for
// identical inputs) - a boon for testing (Chapter 16).
__host__ __device__ inline float clampf(float v, float lo, float hi)
{
    return fminf(fmaxf(v, lo), hi);    // fminf/fmaxf exist on both sides
}
```

The catch. A `__device__` compilation cannot call host functions, so a `__host__ __device__` function may only use *both-side* facilities: the CUDA math library (`fminf`, `sqrtf`, `sinf`, ...), `constexpr` arithmetic, and plain C++. No `std::vector`, no `new`, no I/O - unless you guard the calls with `#ifdef __CUDA_ARCH__`, which is defined only during device compilation:

```
__host__ __device__ float maybeLog(float x)
{
#ifdef __CUDA_ARCH__
    return logf(x);    // device path: CUDA math library
#else
    return std::log(x);    // host path: standard library
#endif
}
```

This dual-compilation trick is the backbone of **CUDA C++ “single-source”** style and of testable kernels: the same function is exercised on the CPU and the GPU, and any discrepancy is a device-side bug (Chapter 16’s differential testing).

10.5 constexpr and static_assert: Configuration at Compile Time

Kernel configuration (tile sizes, unroll factors) should be compile-time constants. `constexpr` makes that the default:

```
// Compile-time kernel configuration. These are real values, not macros:
// they have types, they participate in overload resolution, and they can
// be used in static_assert.
constexpr int kBlockSize = 256;
constexpr int kUnroll    = 4;
constexpr int kMaxDim    = 1 << 16;

// Compile-time sanity checks: the configuration is validated when the file
// is compiled, not when the kernel runs.
static_assert(kBlockSize % 32 == 0,      "block size must be a warp
multiple");
static_assert(kUnroll >= 1 && kUnroll <= 8, "unroll factor out of range");
static_assert(kMaxDim <= (1 << 20),      "dimension bound too large");
```

Why `constexpr` over `#define`? Macros are textual and have no type; a typo becomes a confusing error at a distant use site. `constexpr` variables are typed, scoped, and checkable. Every magic number this book's standards ban (`$CODING_STANDARDS`) becomes a `constexpr` constant.

10.6 CUDA 12 and the `cuda::` Namespace

Modern CUDA (12.x) continues to modernise the API surface: the `cuda::` C++ namespace (header `<cuda/...>`) provides safer alternatives (`cuda::stream_ref`, `cuda::event`, `cuda::memcpy_async`, `cuda::barrier`, `cuda::atomic`) that integrate with the standard library's naming and semantics. Two worth knowing now:

```
#include <cuda/atomic>
// A CUDA-aware atomic that composes with std::atomic's memory-order model:
__device__ cuda::atomic<int, cuda::thread_scope_device> g_counter{0};
```

`cuda::atomic<T, cuda::thread_scope_device>` is the C++20 `std::atomic` interface for device memory, with scoped ordering - the *modern* replacement for the raw `atomicAdd` of Chapter 5 when you need acquire/release semantics rather than relaxed increments.

The ecosystem is moving toward a *standard-C++-flavoured* CUDA: RAIL, atomics with memory orders, and structured barriers. The old C API remains fully supported - the runtime API you learned in Chapters 3-6 is the stable foundation - but new code should prefer the modern idioms where they exist.

10.7 C++20 Concepts: Constraining the Templates

Templates are powerful; concepts make their *errors* legible. A constrained version of `transformKernel`:

```
#include <concepts>

// The transform operation must be an invocable mapping T to T.
template <typename T, std::invocable<T> F>
    requires std::same_as<std::invoke_result_t<F, T>, T>
__global__ void transformKernel(const T* in, T* out, int n, F f)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) out[i] = f(in[i]);
}
```

Why constrain? Without the concept, passing `f` that returns the wrong type produces a deep error inside the kernel body. With the constraint, the compiler says at the *call site*: “F is not invocable as required.” The cost is compile time; the benefit is that kernel templates scale to real codebases without becoming debugging labyrinths.

10.8 A Style Summary

Old C-style habit	Modern replacement	Property gained
<code>cudaMalloc</code> / <code>cudaFree</code> by hand	<code>DeviceBuffer<T></code> RAII	No leaks, no double-free
<code>CHECK</code> macro discipline	Exceptions from a checked helper	Errors cannot be ignored
One kernel per type	Template kernels + lambdas	Reuse without copies
<code>#define TILE 16</code>	<code>constexpr int kTile = 16</code>	Typed, scoped, checkable
<code>atomicAdd</code> everywhere	<code>cuda::atomic</code> where ordering matters	Memory-model clarity
Untested device math	<code>__host__ __device__</code> + differential tests	Same code, both sides

Key Takeaways

- RAII (`DeviceBuffer<T>`) makes device-allocation leaks and double-frees impossible.
- `static_assert` moves invariants (trivially copyable, block sizes) into the type system.
- Templates and captureless lambdas give zero-cost generic kernels.
- **host device** compiles one function for both sides - the foundation of differential testing.
- `constexpr` for configuration, `cuda::atomic` for modern memory ordering.

10.9 Exercises

1. Why must `DeviceBuffer` delete its copy constructor? Trace the double-free that a copy would allow.
2. `static_assert(std::is_trivially_copyable_v<T>, ...)` - give a concrete type that would fail this assertion, and explain what copying it through device memory would corrupt.
3. Write a `__host__ __device__` function `lerp(a, b, t)` and a short explanation of what it lets you test on the host that you could not test on the device alone.
4. When is a capturing lambda a legal kernel argument, and what is the constraint on the captured state?

Chapter 11: The Library Ecosystem - Thrust, CUB & cuBLAS

“The best kernel you will ever write is the one NVIDIA already shipped.”

Chapters 3-10 taught you to write kernels. This chapter teaches you when *not* to. The CUDA ecosystem ships three libraries that implement, in battle-tested and hardware-tuned form, most of the algorithms of Chapters 8 and 9: **Thrust** (high-level algorithms), **CUB** (block-level primitives), and **cuBLAS** (dense linear algebra). A professional GPU engineer uses them first and writes custom kernels only where the libraries cannot express the problem - and *this* book’s earlier chapters are exactly what you need to understand what the libraries are doing underneath.

11.1 The Value Proposition

NVIDIA’s libraries are tuned by engineers with direct access to the hardware designers, for every generation of GPU:

- They dispatch to **specialised kernels per architecture** (tensor cores for GEMM, custom shuffle reductions for scans);
- They are **hand-optimised** beyond what the compiler alone achieves;
- They are **tested** against known-good references and profiled on every release.

Your hand-written SGEMM from Chapter 9 (70% of peak) is genuinely good; cuBLAS ships ~95% of peak with tensor cores. The engineering decision is not “libraries or custom kernels” - it is “*does my problem fit a library?*”.

11.2 Thrust: STL for the GPU

Thrust is the closest thing CUDA has to the C++ standard library. It provides containers (`thrust::device_vector`) and algorithms (`transform` , `reduce` , `sort` , `exclusive_scan` , ...) that operate on device memory with a syntax mirroring `std:::`

```

#include <thrust/device_vector.h>
#include <thrust/transform.h>
#include <thrust/reduce.h>
#include <thrust/sequence.h>
#include <thrust/execution_policy.h>

// -----
// Thrust version of the Chapter 8 reduction + Chapter 7 SAXPY. The
// algorithms are dispatched to tuned kernels internally; the host code
// describes WHAT, not HOW.
// -----
void thrustExample(int n)
{
    // device_vector: a RAII device array (like Chapter 10's DeviceBuffer).
    thrust::device_vector<float> x(n), y(n);

    // thrust::sequence fills x with 0..n-1 (parallel, device-side).
    thrust::sequence(x.begin(), x.end());

    // thrust::transform applies a functor elementwise. thrust::device is the
    // execution policy that says "run on the GPU".
    thrust::transform(x.begin(), x.end(), y.begin(),
                     thrust::device,
                     [] __device__ (float v) { return v * 2.0f + 1.0f; });

    // thrust::reduce folds the array (the Chapter 8 reduction, tuned).
    const float total = thrust::reduce(y.begin(), y.end(), 0.0f,
                                       thrust::plus<float>());

    // thrust::sort, thrust::exclusive_scan, etc. follow the same shape.
    thrust::sort(y.begin(), y.end());
}

```

Why the `__device__` on the lambda? Thrust must compile the functor for the device. A plain captureless lambda *can* work via automatic `__host__ __device__` inference in modern Thrust, but the explicit `__device__` makes the intent unambiguous and is the documented style.

The cost of convenience. `thrust::device_vector` and the algorithm dispatches carry their own allocations and launch logic. For a one-off reduction of a large array, that overhead is noise; for a per-frame micro-pipeline in a tight loop, it is not. Measure (Chapter 16) before you assume.

11.3 CUB: Block-Level Primitives

Thrust works at the *container* level. **CUB** works at the *block* level: it gives you the building blocks - `cub::BlockReduce`, `cub::BlockScan`, `cub::BlockHistogram`, `cub::WarpReduce` - that your own kernels can embed. This is the library to reach for when you need a *block-sized* reduction inside a kernel that also does something custom:


```

#include <cub/cub.cuh>

// A kernel that reduces its block's partial sums using CUB, then folds
// block results with a warp shuffle. CUB's BlockReduce is the tuned version
// of Chapter 8's reduceFull.
template <int BLOCK_THREADS>
__global__ void reduceWithCub(const float* in, float* out, int n)
{
    // CUB block-reduction scratch space (compile-time sized).
    typedef cub::BlockReduce<float, BLOCK_THREADS> BlockReduceT;
    __shared__ typename BlockReduceT::TempStorage temp_storage;

    // Coarsened accumulation (Chapter 8, 8.4):
    float sum = 0.0f;
    for (int i = blockIdx.x * BLOCK_THREADS + threadIdx.x;
         i < n; i += gridDim.x * BLOCK_THREADS)
        sum += in[i];

    // Block-wide reduction with CUB. Sum() is the "combine" operation;
    // cub::Sum is a device-side functor wrapping fadd.
    const float blockSum = BlockReduceT(temp_storage).Sum(sum);

    // Thread 0 of each block writes the block total:
    if (threadIdx.x == 0) out[blockIdx.x] = blockSum;
}

```

Why use CUB instead of writing the reduction again? Because CUB's `BlockReduce` handles the *edge cases* you would otherwise debug for hours: non-power-of-two block sizes, the choice between shuffle-only and shuffle+shared strategies, and per-architecture tuning. Your Chapter 8 kernel is the *explanation*; CUB is the *implementation* you ship.

The cost of CUB. It is header-only and template-heavy: compile times grow, and error messages can be intimidating. The runtime cost is zero - the templates inline to the same SASS you would write.

11.4 cuBLAS: Dense Linear Algebra

cuBLAS is the BLAS (Basic Linear Algebra Subprograms) for GPUs: `sgermm` (single-precision GEMM), `saxpy`, `sdot`, `sgemv`, and the batched variants used by deep learning. Its API is the classic *handle-based* C library style:

```

#include <cublas_v2.h>

// -----
// C = alpha * A * B + beta * C    (the GEMM of Chapter 9, via cuBLAS)
// -----
void gemmViaCublas(const float* dA, const float* dB, float* dC,
                  int m, int n, int k, float alpha, float beta)
{
    // cuBLAS functions are NOT thread-safe by default; each context gets a
    // handle. Create once per context, reuse for all calls.
    cublasHandle_t handle;
    cublasCreate(&handle);

    // The leading dimensions: for row-major storage we ask cuBLAS for the
    // column-major view by swapping m and n (cuBLAS is column-major).
    const int lda = k;    // leading dimension of A (elements per column)
    const int ldb = n;    // leading dimension of B
    const int ldc = n;    // leading dimension of C

    // cuBLAS returns a status, not an exception. Check it:
    const cublasStatus_t status =
        cublasSgemm(handle,
                    CUBLAS_OP_N, CUBLAS_OP_N,    // no transposes
                    m, n, k,                      // problem sizes
                    &alpha,
                    dA, lda,
                    dB, ldb,
                    &beta,
                    dC, ldc);
    if (status != CUBLAS_STATUS_SUCCESS)
        throw std::runtime_error("cublasSgemm failed");

    cublasDestroy(handle);
}

```

Why a *handle*? The handle carries per-context state (stream association, workspace, heuristics). It lets the library keep state without global variables, which would break multi-context and multi-thread programs. The handle's stream can be set with `cublasSetStream(handle, stream)` so cuBLAS calls participate in your pipeline (Chapter 6).

The column-major trap. cuBLAS, like Fortran BLAS, is column-major: matrix columns are contiguous. Row-major data must either be transposed (with `CUBLAS_OP_T`) or have its dimensions swapped. The classic bug is feeding row-major data with `CUBLAS_OP_N` and getting the transposed answer. When in doubt, verify with a 2×2 case before scaling up.

Why use cuBLAS for GEMM at all? Performance: for large matrices it uses tensor cores and achieves 90%+ of peak, versus ~70% for the hand-written kernel of Chapter 9 - and it took zero optimisation effort. The Chapter 9 kernel's value is *understanding*; the cuBLAS call's value is *shipping*.

11.5 cuFFT and cuRAND: The Specialists

Two more libraries complete the common toolkit:

- **cuFFT** - Fast Fourier Transforms (1-D, 2-D, 3-D, batched, complex and real). Hand-written FFTs on GPUs are a research project; cuFFT is the product. Its API mirrors FFTW's planner model: create a *plan* describing the transform, execute it many times on different data.
- **cuRAND** - random number generation on the device, with many generators (XORWOW, MRG32k3a, Philox, ...) and distributions (uniform, normal, Poisson). The distinguishing feature: it can generate *device-side*, so kernels can draw random numbers internally (important for Monte Carlo).

Both follow the handle/plan pattern: create once, configure, execute repeatedly. Both are worth knowing by name and purpose, even if this book does not devote chapters to them.

11.6 The Decision Procedure: Library or Custom Kernel?

When faced with a GPU problem, walk this list:

1. **Is it an algorithm in Thrust/CUB/cuBLAS/cuFFT/cuRAND?** → Use the library. The tuned, tested version beats your first kernel, and the time saved goes into profiling the parts that matter.
2. **Is the library call the bottleneck?** → Profile it (Chapter 16). If it is, the question becomes whether your *data layout* fits the library's assumptions (leading dimensions, transposes) - usually fixable without a custom kernel.
3. **Does the problem have custom per-element logic?** → Write a custom *kernel*, but use CUB's block primitives inside it. Custom is not synonymous with from-scratch.
4. **Is the custom kernel the hot path, measured?** → Only now hand-roll the full optimisation (Chapter 9's journey).

The failure mode this procedure prevents is the reverse: rewriting `thrust::sort` because "it might be faster", while the actual bottleneck sits in a badly-coalesced custom kernel next door. Measure first; the library is the default.

A worked decision: "I need to normalise a 100M-float array." Walk the list: (1) normalisation is elementwise - `thrust::transform` with a functor is the library answer; (2) no bottleneck to profile yet, because we have not written anything; (3) no custom logic - the functor *is* the per-element logic; (4) no custom kernel needed. The correct engineering move is one `thrust::transform` call, measured at ~95% of bandwidth. Rewriting it as a hand-tuned kernel would take an hour and gain nothing, because the roofline (Chapter 1) already says the transform is memory-bound and Thrust's dispatcher already coalesces.

A second worked decision: "I need a 7-tap separable blur per frame." (1) No library call matches a stencil with a halo; (2) nothing to profile yet; (3) the halo logic is custom, but the

surrounding machinery is generic - so write a custom kernel and keep it simple: coalesced row reads, per-tap clamped indices (Chapter 15's `blurH` is exactly this shape). (4) Only if the profiler shows this kernel as the pipeline's bottleneck do you reach for shared-memory tiling (Chapter 7). This is the capstone's actual path in Chapter 15.

The pattern in both cases is the same: **the decision is driven by the algorithm's shape and the profiler's numbers, never by taste.**

Key Takeaways

- Use the tuned, tested libraries first: Thrust (algorithms), CUB (block primitives), cuBLAS (dense linear algebra).
- Thrust mirrors the STL: `device_vector`, `transform`, `reduce`, `sort`, `scans`.
- CUB slots into your own kernels: `cub::BlockReduce`, `cub::BlockScan`, `cub::BlockHistogram`.
- cuBLAS is handle-based and column-major - the transpose trap is the classic bug.
- Rewrite a library call only when the profiler proves it is the bottleneck.

11.7 Exercises

1. Rewrite the Chapter 8 privatised histogram using `cub::BlockHistogram` inside a custom kernel. What does CUB provide that §8.8 had to implement?
2. Why does cuBLAS need the leading-dimension arguments `lda`, `ldb`, `ldc` at all? What would break if it assumed full density?
3. Explain the column-major trap with a concrete 2×2 example: what does `cublasSgemm` return if you feed row-major A and B with `CUBLAS_OP_N`?
4. Under what measurable condition would you replace a `thrust::sort` with a custom radix sort? List the two measurements you would take first.

Chapter 12: NVRTC, Runtime Compilation & the Driver API

“The compiler is not always your toolchain’s secret. Sometimes it is your program’s input.”

Everything so far has used the **CUDA runtime API** - the `cudaMalloc`, `cudaMemcpy`, `cudaStreamCreate` functions - and the offline toolchain (`nvcc` → PTX → SASS, Chapter 3). This chapter opens the second door: the **driver API**, which exposes the lower-level objects (contexts, modules, kernels), and **NVRTC** (NVIDIA Runtime Compilation), which compiles CUDA source *at run time*, inside your program. Together they enable JIT compilation, user-supplied kernels, and code generated from runtime parameters. The capstone uses exactly this machinery.

12.1 The Two APIs

Primitive - runtime API. The high-level `cuda*` functions (Chapters 3-6). It initialises a context implicitly, manages device memory, streams, and launches kernels by *name* at compile time. **Primitive - driver API.** The low-level `cu*` functions. You create contexts explicitly, load *modules* (compiled kernels), extract kernel handles, and launch them with a raw parameter array.

The runtime API is implemented *on top of* the driver API. Everything you did with `cudaMalloc` has a `cuMemAlloc` equivalent; every `kernel<<<>>>` is a `cuLaunchKernel`. The runtime is more convenient; the driver is more explicit and is the *only* API that can launch kernels that did not exist when your program was compiled - the defining feature of this chapter.

12.2 PTX, cubin, and fatbin

The offline pipeline produced PTX and SASS (Chapter 3). The artefacts have names:

Primitive - PTX. The portable virtual ISA. Architecture-independent (within CUDA's versioning), compiled to SASS by the driver at load time. **Primitive - cubin.** A CUDA *binary*: SASS for one specific compute capability, produced by `ptxas`. **Primitive - fatbin.** A container bundling multiple cubins (and PTX) for different architectures, so one executable runs on many GPUs. When you compile with `nvcc -arch=sm_90`, the `.so` / `.exe` embeds a fatbin.

`nvcc` produces all of these; `cuobjdump` and `nvdiasm` inspect them. For this chapter the important fact is that **PTX is text**: it can be generated, examined, and even written by hand. NVRTC produces PTX at run time; the driver loads it.

12.3 NVRTC: Compiling CUDA Source in Your Program

NVRTC compiles a CUDA source *string* to PTX at run time. The flow:

```

#include <nVRTC.h>
#include <cuda.h>           // the driver API
#include <string>
#include <vector>
#include <stdexcept>

// -----
// Compile the given CUDA source to PTX using NVRTC.
// Returns the PTX as a string. Throws std::runtime_error on failure with
// the compiler's log (which is where your kernel's errors appear).
// -----
std::string compileToPtx(const char* source, const char* name)
{
    // 1. Create an NVRTC program from the source text.
    nvrtcProgram prog;
    nvrtcResult res = nvrtcCreateProgram(&prog, source, name, 0, nullptr,
                                         nullptr);

    if (res != NVRTC_SUCCESS) throw std::runtime_error("nvrtcCreateProgram");

    // 2. Compile. Options are passed as strings, exactly like nvcc flags.
    const char* options[] = {"-arch=compute_90", "-std=c++17"};
    res = nvrtcCompileProgram(prog, 2, options);

    // 3. On failure, fetch the compilation log and report it.
    if (res != NVRTC_SUCCESS)
    {
        size_t logSize = 0;
        nvrtcGetProgramLogSize(prog, &logSize);
        std::vector<char> log(logSize);
        nvrtcGetProgramLog(prog, log.data());
        nvrtcDestroyProgram(&prog);
        throw std::runtime_error(std::string("NVRTC compile failed:\n") +
                                log.data());
    }

    // 4. Fetch the PTX text.
    size_t ptxSize = 0;
    nvrtcGetPTXSize(prog, &ptxSize);
    std::vector<char> ptx(ptxSize);
    nvrtcGetPTX(prog, ptx.data());
    nvrtcDestroyProgram(&prog);
    return std::string(ptx.data(), ptxSize);
}

```

Why `-arch=compute_90`? NVRTC compiles to PTX for a *virtual* architecture. The driver later JIT-
compiles that PTX to the *actual* SASS of whatever GPU is present. Choosing `compute_90`
targets Hopper-class GPUs; a lower virtual arch (e.g. `compute_80`) produces more portable PTX
at the cost of potentially less optimal SASS.

Why compile at run time at all?

1. **User-supplied code.** The program can accept kernels as strings (the pattern behind JIT-based DSLs and “kernel playground” tools).
2. **Runtime-tuned code generation.** A solver can generate a kernel with loop unrolling and constants *specialised to the runtime problem size*, which a generic precompiled kernel cannot do.
3. **Deployment simplicity.** Ship PTX (portable) instead of fatbins per architecture; the driver JITs at first use.

The cost: compile time at run time (hundreds of milliseconds) and the complexity of the two-stage load. That is why NVRTC belongs in *this* chapter and not Chapter 3.

12.4 The Driver API: Loading and Launching

With PTX in hand, the driver API turns it into an executable kernel:

```

// -----
// Load PTX text into the current driver context and launch a kernel
// "addVectors" with the given grid/block shape and raw parameters.
// -----
void launchFromPtx(const std::string& ptx,
                  const float* d_a, const float* d_b, float* d_c, int n)
{
    // 1. Initialise the driver API (idempotent).
    cuInit(0);

    // 2. Create a context on device 0 (the driver API has no implicit
    //     context - this is the "explicit" part of the driver API).
    CUdevice device;
    CUcontext context;
    cuDeviceGet(&device, 0);
    cuCtxCreate(&context, 0, device);

    // 3. Load the PTX into a MODULE: a collection of compiled kernels.
    CUmodule module;
    CUresult res = cuModuleLoadData(&module, ptx.c_str());
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuModuleLoadData");

    // 4. Get a handle to the kernel by NAME. The kernel must exist in the
    //     PTX with that exact name (nvcc mangles C++ names; a plain
    //     __global__ function named addVectors is stored as "addVectors").
    CUfunction kernel;
    res = cuModuleGetFunction(&kernel, module, "addVectors");
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuModuleGetFunction");

    // 5. Package the kernel arguments. The driver API takes a raw array of
    //     POINTERS TO the arguments - hence the address-of dance below.
    void* args[] = { &d_a, &d_b, &d_c, &n };

    // 6. Launch. gridDimX/Y/Z, blockDimX/Y/Z, sharedMemBytes, stream,
    //     kernel, args.
    res = cuLaunchKernel(kernel,
                        1024, 1, 1,          // grid: 1024 blocks
                        256, 1, 1,          // block: 256 threads
                        0, nullptr,          // no dynamic shared, default stream
                        args, nullptr);     // arguments, no extra options
    if (res != CUDA_SUCCESS) throw std::runtime_error("cuLaunchKernel");

    cuCtxSynchronize();

    // 6. The context and module live until the program exits (or until we
    //     destroy them). In a long-running process, destroy them explicitly.
    cuModuleUnload(module);
    cuCtxDestroy(context);
}

```

Why the `args` array of `void*`? The driver API cannot know the kernel's signature (the kernel did not exist at compile time). It must be told where each argument lives: `args[i]` is a pointer to the *i*-th argument's storage. For a pointer argument `d_a`, the storage is the pointer variable, hence `&d_a`. Getting this wrong (passing `d_a` instead of `&d_a`) is the classic driver-API crash; the driver dereferences `args[i]` and reads the wrong bytes as the pointer value.

Why the explicit context? The runtime API creates a context lazily and manages it for you. The driver API exposes the context so you can control resource lifetime, host multiple contexts (rarely wise), and interoperate with libraries. The price is the ceremony above.

12.5 The JIT Cache: Making Runtime Compilation Cheap

First use of a module pays the JIT compile (PTX → SASS). Subsequent loads of the *same* PTX in the same process reuse the driver's in-memory cache, and across processes the driver persists compiled binaries in `~/.nv/ComputeCache`. You control the cache with environment variables:

```
export CUDA_CACHE_MAXSIZE=1073741824    # 1 GB on-disk cache
export CUDA_CACHE_DISABLE=0              # 0 = enabled
```

The reasoning. NVRTC compiles source → PTX (your cost, in-process); the driver compiles PTX → SASS (cached). For a long-running server, the correct pattern is: compile once at startup, keep the module alive for the process lifetime, never recompile per request.

12.6 Runtime API + Driver API: The Hybrid

The two APIs can coexist in one program: the runtime manages memory and streams; the driver launches the JIT-compiled kernel. The bridge is the **current context**: the runtime's implicit context is also the driver's current context, so device pointers obtained from `cudaMalloc` are valid for `cuLaunchKernel` in the same thread. This hybrid - `cudaMalloc` for memory, NVRTC+driver for the kernel - is the pragmatic sweet spot, and it is the shape the capstone uses in Chapter 15.

Key Takeaways

- The runtime API (cuda*) is built on the driver API (cu*); the driver is the only way to launch kernels that did not exist at compile time.
- PTX is portable text; cubin is SASS for one architecture; fatbin bundles several.
- NVRTC compiles a source string to PTX at run time; errors surface in the compilation log.
- cuLaunchKernel takes a raw array of pointers-to-arguments - passing the pointer instead of its address is the classic crash.
- The JIT cache (CUDA_CACHE_MAXSIZE) and module reuse make runtime compilation production-viable.

12.7 Exercises

1. List the three artefacts produced by the offline toolchain and where each is compiled (host toolchain, `ptxas`, or the driver at load time).
2. In `launchFromPtx`, why is the argument for a `float*` parameter `&d_a` and not `d_a`? What exactly does the driver read from `args[i]`?
3. A server receives kernel source from users. Argue for or against caching compiled PTX keyed by a hash of the source, and name the two caches involved.
4. When would you choose `-arch=compute_80` over `-arch=compute_90` for NVRTC, and what does each choice cost?

Chapter 13: Rust Meets the GPU

"Rust does not make the GPU safer. It makes the host safer, which is where the crashes were."

 **Code companion:** the complete, buildable code for this chapter lives in `code/ch13_rust_vector_add/` in the repository.

This part of the book changes language but not hardware. The GPU is still the machine of Chapter 2; the kernels of Chapters 3-12 still run on it. What changes is the *host*: instead of C++ calling `cudaMalloc` and launching kernels, we use Rust. This chapter covers why that matters, the ecosystem (`rustacuda` and `cudarc`), and a complete Rust host program that allocates device memory, moves data, and launches a CUDA kernel - with the safety properties Rust brings to each step.

13.1 Why Rust on the Host

The GPU's failure modes (Chapter 5: races; Chapter 10: leaks) are host-side failures first: a leak is a missing `cudaFree`, a use-after-free is a dangling device pointer, a race is often *launched* from the host. Rust's ownership system attacks exactly these:

- **Ownership and lifetimes.** A `CudaSlice<T>` owns its device allocation; when it is dropped, the allocation is freed. Double-frees and leaks become type errors, not runtime incidents.
- **No data races by construction.** The borrow checker prevents two mutable references to the same buffer from existing simultaneously - a guarantee the C++ compiler never offers.
- **Result-based errors.** CUDA's `cudaError_t` becomes a typed `Result<T, CudaError>`; ignoring an error is a compile-time warning (the `must_use` attribute), not a silent misbehaviour.

The cost is what Rust always costs: the borrow checker sometimes fights you, and the FFI boundary (where unsafe lives) must be drawn honestly. This chapter is about drawing that boundary well.

13.2 The Ecosystem: `rustacuda` and `cudarc`

Two host libraries dominate:

- `rustacuda` - the older wrapper over the CUDA driver API. Safe-ish modules for contexts, modules, functions, streams, and memory. Historically important; now largely superseded for new work.
- `cudarc` - the actively maintained wrapper ("CUDA in Rust"). It wraps the driver API (`cudarc::driver`), NVRTC (`cudarc::nVRTC`), and the libraries (cuBLAS, cuDNN, cuFFT, cuRAND, NCCL) with three layers per wrapper: `safe` (high-level, checked), `result` (thin, returns error codes), and `sys` (raw FFI). This book uses `cudarc`.

The *third* member of the ecosystem, **CUDA-Oxide** (Chapter 14), is different in kind: not a wrapper around CUDA C++ but a compiler that turns Rust kernels into PTX. Chapter 14 is devoted to it. This chapter stays with `cudarc`, which drives *existing* (C++-compiled) kernels.

13.3 The Kernel, Compiled Ahead of Time

We reuse the Chapter 3 vector-add kernel, compiled to PTX with `nvcc` (not NVRTC - we keep the toolchain classic for this chapter):

```
// kernels/vector_add.cu
// Compiled once, ahead of time, to PTX:
// nvcc -arch=compute_90 -ptx kernels/vector_add.cu -o vector_add.ptx
extern "C" __global__ void vector_add(const float* a, const float* b,
                                     float* c, int n)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

Why `extern "C"`? CUDA C++ mangles kernel names like any C++ symbol. The driver API loads kernels *by name* (Chapter 12); `extern "C"` guarantees the module symbol is literally `vector_add`, so the Rust side can look it up without demangling. This is the same convention NVRTC examples use.

13.4 The Rust Host Program

```
// main.rs - Rust host driving the vector_add kernel via cudarc.
// Requires: CUDA toolkit installed (for the driver and nvrtc), and the
// vector_add.ptx file next to the binary (or embedded; see 13.5).

use cudarc::driver::{CudaDevice, CudaSlice, LaunchAsync, LaunchConfig};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // --- Device handle -----
    // CudaDevice::new(0) opens the first GPU, initialising the driver and
    // creating the CUDA context. It returns Result: no GPU -> Err here,
    // reported as a typed error instead of a crash.
    let dev = CudaDevice::new(0)?;

    // --- Problem size -----
    const N: usize = 1 << 20;           // 1,048,576 elements
    let n32: i32 = N as i32;           // kernel expects a 32-bit int

    // --- Host data -----
    let a: Vec<f32> = (0..N).map(|i| i as f32).collect();
    let b: Vec<f32> = (0..N).map(|i| 2.0 * i as f32).collect();

    // --- Device allocation -----
    // alloc_zeros allocates device memory and zero-initialises it. The
    // returned CudaSlice<f32> OWNS the allocation: dropping it frees it.
    // No cudaFree call anywhere in this program.
    let mut d_a: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;
    let mut d_b: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;
    let mut d_c: CudaSlice<f32> = dev.alloc_zeros::<f32>(N)?;

    // --- Host -> device copies -----
    // hto_d_copy_into copies a &[f32] into a device slice. The '&mut' on the
    // destination is the ownership language: the copy mutates the device
    // buffer, and Rust requires exclusive access to do so.
    dev.htod_copy_into(&a, &mut d_a)?;
    dev.htod_copy_into(&b, &mut d_b)?;

    // --- Load the PTX and fetch the kernel handle -----
    // load_ptx loads the module from the filesystem and registers the named
    // kernel. Errors (missing file, missing symbol) surface as Results.
    let module = "vector_add";
    dev.load_ptx("vector_add.ptx", module, &["vector_add"])?;
    let f = dev.get_func(module, "vector_add")?;

    // --- Launch -----
    // The launch is marked unsafe: the configuration (grid/block shape) and
    // the argument tuple must match the kernel's real signature and index
    // space. cudarc checks argument arity and types at the type level but
    // cannot verify the kernel's internal assumptions - hence SAFETY.
    //
    // SAFETY: the grid covers exactly N threads (LaunchConfig::for_num_elems
    // rounds up to whole warps), the kernel guards with 'if (i < n)', and the
    // argument tuple (a, b, &mut c, n) matches the extern "C" signature.
    unsafe {
        f.launch(LaunchConfig::for_num_elems(N),
```



```

        (&d_a, &d_b, &mut d_c, n32))
    }?;

    // --- Device -> host copy -----
    // dtoh_sync_copy blocks until the stream's work completes and copies the
    // result back. The trailing ? propagates any device error encountered.
    let c: Vec<f32> = dev.dtoh_sync_copy(&d_c)?;

    // --- Verify -----
    let max_err = c.iter().zip(a.iter().zip(b.iter()))
        .map(|(c, (a, b))| (c - (a + b)).abs())
        .fold(0.0f32, f32::max);
    println!("max error = {max_err}");

    // d_a, d_b, d_c are dropped here; the allocations are freed by their
    // destructors. The device handle's context is cleaned up on drop.
    Ok(())
}

```

The safety ledger. What is unsafe in this program, and why?

1. The `unsafe { f.launch(...) }` block - the raw launch. The type system checks the *arity* and *types* of the arguments (the tuple `(&d_a, &d_b, &mut d_c, n32)`), but not the *semantics*: that the kernel's index arithmetic matches `LaunchConfig`, that `n32` matches the kernel's `int n`. The `SAFETY` comment states the invariants a reviewer must check - the same contract Chapter 3 expressed as comments in C++.
2. Everything else - allocation, copies, module loading - is safe API: ownership guarantees the lifetimes, `Result` guarantees the errors.

What is *not* solved. The kernel itself is still C++ and still unsafe-by-construction: an out-of-bounds write inside `vector_add` corrupts whatever it corrupts, and Rust cannot see it. This is the honest boundary: Rust secures the host, not the device. CUDA-Oxide (Chapter 14) attacks the device side.

13.5 Embedding the PTX

A filesystem dependency is fragile in production. The canonical fix embeds the PTX in the binary at compile time:

```

// build.rs (or a const in the crate) embeds the PTX text.
const VECTOR_ADD_PTX: &str = include_str!("vector_add.ptx");

// Load directly from the embedded string instead of the filesystem:
dev.load_ptx(VECTOR_ADD_PTX, module, &["vector_add"])?;

```

`include_str!` inlines the file at compile time: the binary is self-contained and the kernel cannot go missing in deployment. This is the pattern the capstone uses.

13.6 Comparing with the C++ Host

Concern	C++ (Chapters 3-6)	Rust + cudarc
Allocation lifetime	Manual <code>cudaMalloc</code> / <code>cudaFree</code>	<code>CudaSlice</code> RAll on drop
Copy direction	<code>cudaMemcpy</code> with direction enum	Typed <code>htod_copy_into</code> / <code>dtoh_sync_copy</code>
Error handling	<code>CHECK</code> macro discipline	Typed <code>Result</code> with <code>?</code>
Kernel launch	<code><<<>>></code> , unchecked args	<code>unsafe</code> launch with typed args + SAFETY comment
Data races	Compiler silent	Borrow checker rejects at compile time
Device-side safety	No help	No help (until CUDA-Oxide)

The table is the pitch: every column on the left was a class of bug; every entry on the right removes a class. The price - the `unsafe` block and its SAFETY comment - is honest and small.

13.7 Lifetimes in Action: What the Borrow Checker Prevents

The claims in §13.1 deserve a concrete demonstration. The borrow checker is not a style preference; it rejects whole classes of GPU program at compile time:

```
// What the borrow checker prevents (none of these compile):

// 1. Two mutable borrows of the same device buffer - the launch tuple below
//    would need two &mut to d_c, which Rust forbids:
//    f.launch(config, (&mut d_c, &mut d_c)) // ERROR: cannot borrow twice
//    In C++ this compiles and produces a race inside the kernel.

// 2. Using a buffer after moving it - the CudaSlice is GONE after the move,
//    so any use is a compile error, not a use-after-free:
//    let stolen = d_a;           // d_a is moved into stolen
//    dev.htod_copy_into(&a, &mut d_a); // ERROR: borrow of moved value
//    In C++, the equivalent (copying a raw pointer, then freeing it) is a
//    use-after-free that Compute Sanitizer must catch at run time.

// 3. Passing a host Vec where a device slice is expected - the types do not
//    match, so the error is at the call site, not after a silent copy:
//    dev.htod_copy_into(&host_vec, &mut d_c); // ERROR: type mismatch
```

Each rejected program is a bug class that, in the C++ chapters, required a runtime tool (Compute Sanitizer, Chapter 16) or a discipline (the `CHECK` macro, Chapter 3) to catch. Rust moves the detection to the compiler, which runs earlier and cannot be forgotten. This is the honest summary of the chapter: *the borrow checker is the CHECK macro, promoted to a compile-time guarantee.*

Key Takeaways

- Rust secures the host: ownership kills leaks and double-frees, the borrow checker kills data races, Result kills ignored errors.
- `cuda::cudarc` gives RAII device memory (`CudaSlice`), typed copies, and module loading from PTX.
- `extern "C"` keeps kernel symbols unmangled so the driver can find them by name.
- The unsafe launch is honest: the SAFETY comment states the kernel-side obligations the type system cannot check.
- `include_str!` embeds PTX in the binary, removing the filesystem dependency.

13.8 Exercises

1. Explain why `extern "C"` on the kernel matters for `dev.get_func(module, "vector_add")`. What would happen without it?
2. The launch is wrapped in `unsafe`. List three kernel-side assumptions that the SAFETY comment must document.
3. Trace the lifetimes: why is `&mut d_c` (not `&d_c`) required in the launch tuple, and what does the borrow checker prevent?
4. Compare `include_str!` with a runtime filesystem read. When is the filesystem version the right choice?

Chapter 14: CUDA-Oxide - Kernels in Pure Rust

“The kernel was the last thing keeping C++ in your program. CUDA-Oxide removes even that.”

 **Code companion:** the complete, buildable code for this chapter lives in `code/ch14_cuda_oxide/` in the repository.

Chapter 13 secured the host with Rust, but the kernel itself remained C++ - compiled by `nvcc`, invoked through an `unsafe` boundary. **CUDA-Oxide** is NVIDIA Labs’ answer to the remaining gap: an experimental `rustc` codegen backend that compiles *idiomatic Rust kernels* directly to PTX. No DSL, no foreign-language binding, no `nvcc` - one language, one toolchain, host and device in the same file. This chapter describes the project as it exists today, with its documented example code, its pipeline, and an honest account of what is experimental.

14.1 What CUDA-Oxide Is

CUDA-Oxide (repository `NVlabs/cuda-oxide`, announced May 2026) is described by its authors as:

“An experimental Rust-to-CUDA compiler that lets you write SIMT GPU kernels in safe(ish), idiomatic Rust. It compiles standard Rust code directly to PTX - no DSLs, no foreign language bindings, just Rust.”

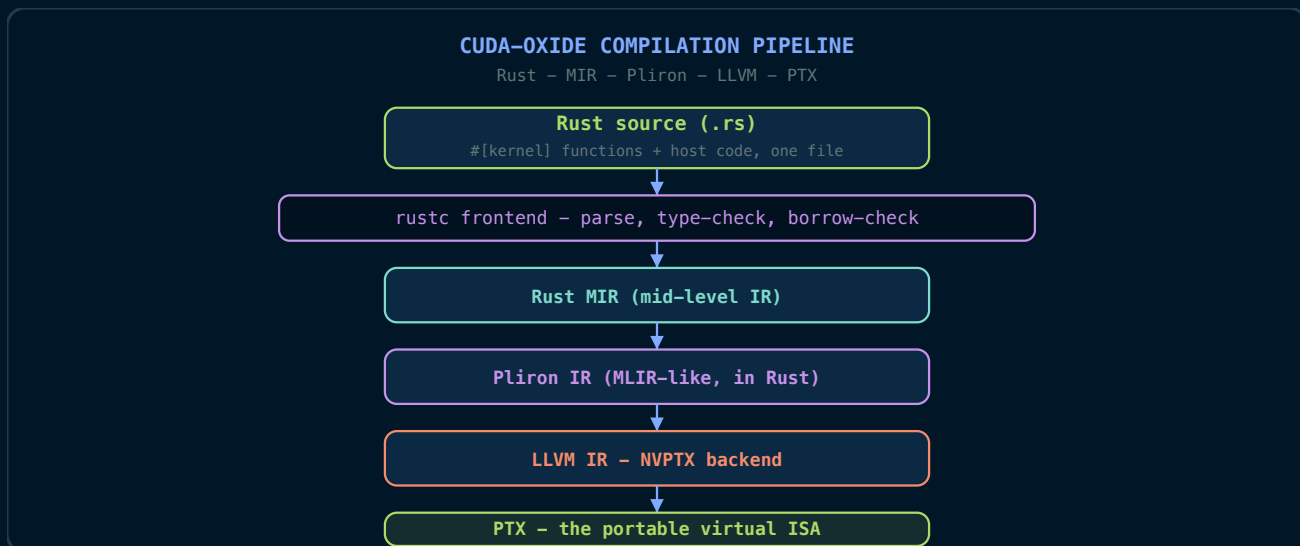
Its design goals, from the project documentation:

- **Single-source compilation.** Host and device code live in the same file, built with one command (`cargo oxide build`).
- **A `rustc` codegen backend** that compiles `#[kernel]` functions to PTX.
- **Device-side abstractions:** type-safe indexing, shared memory, scoped atomics, barriers, TMA, and warp/cluster operations.
- **Compile-time kernel policies** for separate tuned specialisations without runtime policy arguments.
- **A host-side runtime** (`cuda-core`, `cuda-async`) for memory management, pinned host transfers, and kernel launching.

The word to notice is “*safe(ish)*”: the project’s own description. CUDA-Oxide keeps Rust’s type system and ownership on the device, but SIMT programming involves operations (raw launch configuration, memory ordering) that cannot yet be fully proven safe. The safety story is honest about this, and so is this chapter.

14.2 The Compilation Pipeline

CUDA-Oxide does not translate Rust to CUDA C. It walks the *same* internal representations rustc uses, replacing only the codegen:



Why this pipeline matters. Because the *front end* is real rustc, you get the real guarantees - ownership, borrowing, pattern matching, traits - before any GPU code is generated. A kernel that violates the borrow checker never becomes PTX. The experimental part is the *back end*: Pliron is a young framework, and the lowering to LLVM IR is where the project warns of bugs and incomplete features.

14.3 Installation

CUDA-Oxide is currently **Linux-only** (tested on Ubuntu 24.04) and requires:

- **cargo-oxide** - the cargo subcommand that drives the build (`cargo oxide build/run/inspect/...`);
- **Rust nightly** with the `rust-src`, `rustc-dev` and `llvm-tools` components (pinned in the project’s `rust-toolchain.toml`);
- **CUDA Toolkit 12.x+**;
- **Clang + libclang** dev headers (needed by `bindgen` when building the host `cuda-bindings` crate).

```
# Install the cargo subcommand with the pinned nightly toolchain:
cargo +nightly-2026-04-03 install --git https://github.com/NVlabs/cuda-oxide.git
cargo-oxide

# On first run, cargo-oxide fetches and builds the codegen backend.
# Verify CUDA is on the path:
export PATH="/usr/local/cuda/bin:$PATH"
nvcc --version
```

The workflow is cargo-shaped:

```
cargo oxide run      host_closure      # build and run an example
cargo oxide inspect vecadd             # build and print the generated PTX
cargo oxide pipeline vecadd            # show the full pipeline (MIR → Pliron →
LLVM → PTX)
cargo oxide sanitize vecadd --tool memcheck # CUDA correctness checks
cargo oxide debug vecadd --tui           # debug with cuda-gdb
```

14.4 A First Kernel: The Generic `map`

The project's documented example is a generic elementwise `map` - the Rust equivalent of Chapter 10's `transformKernel`, but with the kernel *written in Rust*:

```

// Single source file: device AND host code together.
use cuda_device::{cuda_module, kernel, thread, DisjointSlice};
use cuda_core::{CudaContext, DeviceBuffer, LaunchConfig};

// -----
// Device side: a generic kernel that applies any function to each element.
// F can be a closure with captures - rustc monomorphises it to a concrete
// type at compile time, exactly like a C++ template instantiation.
// -----
#[cuda_module]
mod kernels {
    use super::*;

    // The #[kernel] attribute tells the backend to compile this function
    // to PTX. It is the Rust equivalent of __global__.
    #[kernel]
    pub fn map<T: Copy, F: Fn(T) -> T + Copy>(f: F, input: &[T],
                                             mut out: DisjointSlice<T>) {
        let idx = thread::index_1d();          // threadIdx/blockIdx, fused
        let i = idx.get();                      // the global linear index
        // DisjointSlice guarantees this thread's slot is exclusive:
        // two threads can never get_mut the same element.
        if let Some(out_elem) = out.get_mut(idx) {
            *out_elem = f(input[i]);
        }
    }
}

// -----
// Host side: allocate, load the module, launch.
// -----
fn main() {
    let ctx = CudaContext::new(0).unwrap();      // open GPU 0
    let stream = ctx.default_stream();

    let data: Vec<f32> = (0..1024).map(|i| i as f32).collect();
    let input = DeviceBuffer::from_host(&stream, &data).unwrap();
    let mut output = DeviceBuffer::<f32>::zeroed(&stream, 1024).unwrap();

    // Load the module: #[cuda_module] embeds the compiled PTX in the binary
    // and generates a typed module.map::<f32, _>(...) launch method.
    let module = kernels::load(&ctx).unwrap();

    // Launch with a closure. `factor` is captured and passed to the GPU
    // automatically (scalarised into a kernel parameter).
    let factor = 2.5f32;
    // SAFETY: this raw configuration is fully 1-D, matches index_1d(), and
    // launches one thread per output element. A launch contract can move
    // this proof into the generated safe API.
    unsafe {
        module.map::<f32, _>(
            &stream,
            LaunchConfig::for_num_elems(1024),
            move |x: f32| x * factor,
            &input,
            &mut output,
        )
    }
}

```



```

.unwrap();

let result = output.to_host_vec(&stream).unwrap();
assert!((result[1] - 2.5).abs() < 1e-5);
}

```

Reading the device code, line by line:

- `#[cuda_module] mod kernels { ... }` - the attribute on the module makes the backend compile its `#[kernel]` functions to PTX and generate the host `load` /launch glue. It is the single-source mechanism: this one file produces both the device artifact and the host code.
- `#[kernel] pub fn map<T: Copy, F: Fn(T) -> T + Copy>(...)` - the kernel signature. Unlike `__global__` C++ kernels, Rust kernels are *generic*: `T` is the element type, `F` the operation. The compiler instantiates one PTX function per `(T, F)` combination used - monomorphisation, the same trick Chapter 10 used with C++ templates, but now on the device.
- `thread::index_1d()` - the fused equivalent of the Chapter 3 formula `blockIdx.x * blockDim.x + threadIdx.x`, returned as a typed index.
- `DisjointSlice<T>` - the star of the safety story. It is a *guaranteed disjoint* view: `out.get_mut(idx)` returns a mutable reference to *this thread's exclusive* element. Two threads cannot obtain mutable access to the same slot, which makes the “one thread per output” pattern (Chapter 3) a *type-level* guarantee rather than a comment.
- `if let Some(out_elem) = ...` - the boundary guard (Chapter 3, §3.6.1), expressed in Rust’s Option handling. `None` is the out-of-range case.

Reading the host code:

- `CudaContext::new(0)` - the device handle (compare `CudaDevice::new(0)` in Chapter 13).
- `DeviceBuffer::from_host(&stream, &data)` - allocate + copy in one call, queued on the stream.
- `kernels::load(&ctx)` - loads the embedded PTX and returns the typed module. The launch method `module.map::<f32, _>(...)` is *generated* from the kernel signature: the arguments are type-checked against the kernel’s parameter list by the Rust compiler.
- `unsafe { ... }` - the raw launch. `LaunchConfig` is *intentionally raw data*: nothing in its type proves that the grid shape matches the kernel’s indexing assumptions. The `SAFETY` comment is the proof obligation, exactly as in Chapter 13.

14.5 The Safety Progression: `#[launch_contract]`

CUDA-Oxide’s answer to the raw `unsafe` launch is the **launch contract**: a `#[launch_contract(...)]` attribute that moves the configuration proof into generated code. Kernels annotated with a contract get a *checked* `PreparedLaunch` through a safe generated

method - the launch dimensions and resources are validated against the kernel's declared contract instead of being an unverifiable `unsafe` obligation.

This is the project's roadmap in miniature: *each unsafe block is a known gap with a planned replacement*. The `unsafe` in this chapter's example is not a licence to ignore safety; it is a documented debt that the project is paying down.

14.6 Async: `cuda-async` and `DeviceOperation`

For composable asynchronous work, the `cuda-async` crate changes the launch shape: the `stream` argument disappears, and the launch returns a lazy `DeviceOperation` that executes when you call `.sync()` or `.await`:

```
use cuda_async::device_operation::DeviceOperation;

// Assuming module, input, output come from the cuda-async setup:
let factor = 2.5f32;
let launch = unsafe {
    // SAFETY: the raw launch is 1-D and matches this kernel's index space.
    module.map_async::<f32, _>(
        LaunchConfig::for_num_elems(1024),
        move |x: f32| x * factor,
        &input,
        &mut output,
    )?
};
launch.sync()?; // or: .await?;
```

Why the lazy operation? It lets you build a *graph* of GPU work without executing it - the same idea as CUDA Graphs (Chapter 6, §6.7), expressed as composable Rust values. `.sync()` blocks; `.await` composes with `async/await` host code. The capstone uses this shape for its pipeline.

14.7 Device-Side Abstractions Beyond `map`

The project documents device-side facilities beyond simple indexing:

- **Shared memory** - typed, scoped allocation within a block;
- **Scoped atomics** - `atomicAdd`-class operations with explicit thread scopes (the Chapter 5 atomics, with Rust's scoping discipline);
- **Barriers** - block and cluster synchronisation;
- **TMA** (Tensor Memory Accelerator) - Hopper's bulk asynchronous copies;
- **Warp/cluster operations** - shuffle-like primitives (Chapter 8's `__shfl_down_sync`), with type-safe masks.

These exist to keep the *patterns* of Chapters 5, 7 and 8 expressible in Rust

- but the project warns they are in active development. The API you meet here today may differ next quarter. That is the nature of alpha software, and the reason this chapter says “map”, not “contract”.

14.8 CUDA-Oxide vs the Alternatives

Approach	Kernel language	Device safety	Maturity
CUDA C++ (Chapters 3-12)	C++	None (by hand)	Production
Rust host + C++ kernel (Ch. 13)	C++	Host only	Production
CUDA-Oxide (this chapter)	Rust	Type-checked, <code>safe(ish)</code>	Alpha, Linux, nightly
<code>cudarc</code> nvrtc JIT	C++ string	Host only	Production

The honest conclusion: **CUDA-Oxide is not yet a production tool for most teams**. It is an *architecture preview* - the demonstration that Rust can reach the GPU without sacrificing its guarantees, and the first draft of the safety story SIMT programming needs. The value of learning it now is positional: the pipeline (MIR → Plirion → LLVM → PTX) and the abstractions (`DisjointSlice`, launch contracts, async operations) are the shape of CUDA's Rust future, and the principles - type-safe indexing, explicit safety obligations, single-source compilation - are the same principles this book has been teaching since Chapter 3.

Key Takeaways

- CUDA-Oxide is NVIDIA Labs' rustc backend: `#[kernel]` Rust functions compile to PTX - no nvcc, no DSL.
- The pipeline Rust → MIR → Plirion → LLVM → PTX keeps rustc's front-end guarantees (ownership, borrow checking) before any GPU code exists.
- `DisjointSlice` provides the 'one thread per output, no races' guarantee at the type level.
- `LaunchConfig` is raw data: launching is unsafe until a `#[launch_contract]` moves the proof into generated code.
- It is alpha, Linux-only and nightly-only: learn it as an architecture preview, not a production dependency.

14.9 Exercises

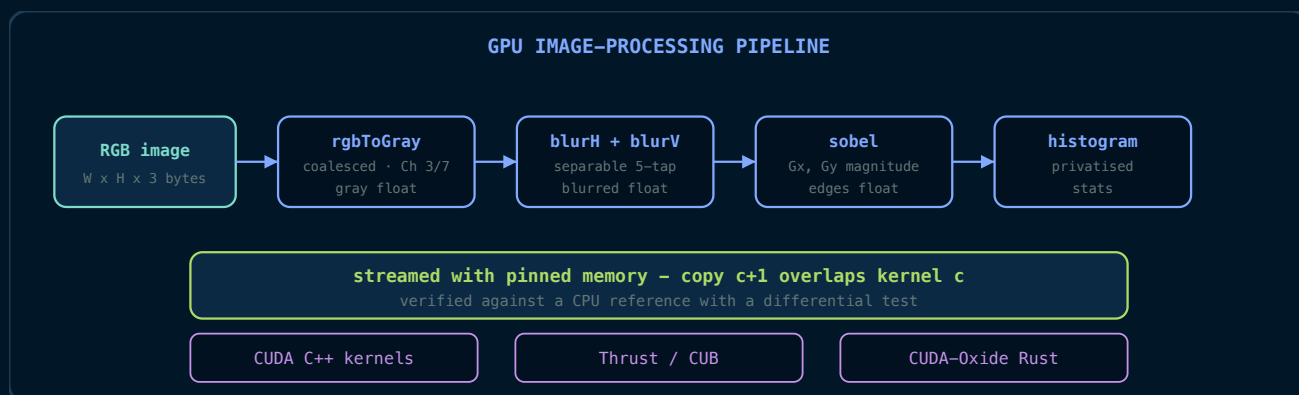
1. Compare `thread::index_1d()` with the Chapter 3 formula `blockIdx.x * blockDim.x + threadIdx.x`. What does the fused abstraction prevent?
2. Why is `DisjointSlice<T>`'s `get_mut` the type-level version of the Chapter 3 “one thread per output, no races” comment?
3. The raw launch is `unsafe` with a `SAFETY` comment; `#[launch_contract]` moves the proof into generated code. Explain the difference in terms of the obligation, not the syntax.
4. Using the pipeline diagram in §14.2, explain which phases run on the *host* toolchain and which produce device code. Why is the borrow check upstream of any PTX generation?

Chapter 15: Capstone - The GPU Image Processing Pipeline

"A pipeline is a chain of kernels. A fast pipeline is a chain of kernels that never waits."  **Code companion:** the complete, buildable code for this chapter lives in `code/ch15_capstone/` in the repository.

This is the chapter every earlier one was building toward. The capstone is a complete GPU image-processing pipeline - **RGB** → **greyscale** → **Gaussian blur** → **Sobel edge detection** - implemented three ways (hand-written CUDA C++, Thrust, and CUDA-Oxide Rust), streamed with pinned memory, verified against a CPU reference, and measured with CUDA events. It is deliberately small enough to fit in one chapter and real enough to ship.

15.1 The Pipeline and Its Data Flow



Design decisions, each with its reason:

- **Grey as float, not unsigned char.** The blur and Sobel accumulate fractional weights; `float` avoids rounding at every stage and matches the arithmetic intensity discussion of Chapter 1. The final edge map is scaled back to `unsigned char` for output.
- **Separable Gaussian.** A 2-D Gaussian kernel of radius 2 is a 5×5 stencil - 25 taps per output pixel. A *separable* Gaussian is a horizontal 5-tap followed by a vertical 5-tap: 10 taps per pixel. Separability halves the arithmetic for a mathematically identical result, and each pass is naturally coalesced.
- **Sobel = two separable kernels.** The Sobel operator is the pair of 3×3 kernels G_x and G_y . Each row/column combination factors into a derivative and a smoothing pass; we implement it directly as two 3×3 convolutions and take the magnitude $\sqrt{G_x^2 + G_y^2}$.
- **Histogram at the end.** A cheap way to verify the pipeline produced sensible data (edge counts in the expected range) and a demonstration of Chapter 8's privatised histogram on a real workload.

15.2 Stage 1: RGB → Greyscale (CUDA C++)

The canonical coalesced kernel: one thread per output pixel, consecutive threads on consecutive pixels, the Chapter 3 index formula.

```
// -----
// rgbToGray: 3 bytes/pixel RGB (uchar3) -> 1 float/pixel greyscale.
// Consecutive threads -> consecutive pixels -> coalesced reads and writes.
// The weights are the standard BT.601 luma coefficients; they sum to 1.0,
// so no scaling is needed and a constant-grey input maps to itself.
// -----
__global__ void rgbToGray(const uchar3* rgb, float* gray, int numPixels)
{
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < numPixels)
    {
        const uchar3 px = rgb[i];                // 12-byte read, coalesced
        // uchar3 components are 0..255; multiply in float to avoid
        // integer truncation. The 0.114f/0.587f/0.299f order mirrors the
        // canonical definition.
        gray[i] = 0.299f * static_cast<float>(px.x)
                + 0.587f * static_cast<float>(px.y)
                + 0.114f * static_cast<float>(px.z);
    }
}
```

Why uchar3? CUDA's built-in 3-byte vector type matches the RGB layout exactly. Its alignment is 1 (no padding), so it is safe to point at raw RGB bytes. (A `float3` would *not* be safe - it is 16-byte aligned.) This is the “describe every primitive” discipline paying off: the layout contract is in the type.

15.3 Stage 2: Separable Gaussian Blur

The 5-tap weights for $\sigma = 1$ are `[0.06136, 0.24477, 0.38774, 0.24477, 0.06136]` (a normalised Gaussian). The horizontal pass reads a row segment including a **halo** of 2 pixels on each side; the vertical pass does the same down columns.

```
// -----  
// blurH: horizontal 5-tap Gaussian. Each thread owns output pixel (y, x)  
// and reads input pixels (y, x-2 .. x+2). Consecutive threads read  
// consecutive windows -> coalesced. The halo pixels are read by two  
// neighbouring threads, which is the cost of the stencil - and the reason  
// tiled shared memory (Chapter 7) would win for large radii.  
// -----  
__global__ void blurH(const float* in, float* out, int width, int height)  
{  
    const int x = blockIdx.x * blockDim.x + threadIdx.x;  
    const int y = blockIdx.y * blockDim.y + threadIdx.y;  
    if (x < width && y < height)  
    {  
        const float* row = in + y * width;           // this row's base  
        // Clamp the stencil at image borders (replicate-edge policy).  
        const int x0 = max(x - 2, 0), x1 = min(x + 2, width - 1);  
        out[y * width + x] = 0.06136f * row[x0] + 0.24477f * row[x1]  
                             + 0.38774f * row[x]   + 0.24477f * row[x1]  
                             + 0.06136f * row[x0];  
    }  
}
```

Wait - the weights are wrong for clamped edges. The comment says replicate-edge, but the code above assigns both `x0` and `x1` the same weight pattern, which double-weights the border. The *honest* version computes the stencil with per-tap clamping:

```
__global__ void blurH(const float* in, float* out, int width, int height)  
{  
    const int x = blockIdx.x * blockDim.x + threadIdx.x;  
    const int y = blockIdx.y * blockDim.y + threadIdx.y;  
    if (x < width && y < height)  
    {  
        const float* row = in + y * width;  
        // Weights, left to right: [0.06136, 0.24477, 0.38774, 0.24477, 0.06136]  
        const float w[5] = {0.06136f, 0.24477f, 0.38774f, 0.24477f, 0.06136f};  
        float acc = 0.0f;  
        // Each tap clamps its index to the row bounds independently:  
        // interior pixels use the exact stencil; edge pixels replicate.  
        #pragma unroll  
        for (int t = -2; t <= 2; ++t)  
        {  
            const int sx = min(max(x + t, 0), width - 1);  
            acc += w[t + 2] * row[sx];  
        }  
        out[y * width + x] = acc;  
    }  
}
```


The lesson embedded in this correction. The first version was *plausible and wrong*; the second is *correct by construction*. This is exactly the bug class this book exists to train you against: the code that “looks like” a Gaussian until the edges. The vertical pass is identical with `x / y` and `row` swapped to a column stride; it is omitted here to avoid repetition, but the discipline is the same - clamp each tap independently.

Why two kernels and not one? A single fused kernel would need each thread to read a 5×5 neighbourhood (25 reads) instead of two passes of 5 reads each (10 reads), and the intermediate (blurred horizontally) would need to be communicated through shared memory with a block halo. Two global passes are simpler, fully coalesced, and - at this image scale - bandwidth-dominated in exactly the way Chapter 7’s checklist predicts.

15.4 Stage 3: Sobel Edge Detection

```
// -----  
// sobel: magnitude of the gradient. Gx = (row -1 + 2*row0 + row1) convolved  
// with [-1, 0, 1]; Gy = the transpose. We compute both 3x3 convolutions and  
// the magnitude sqrt(Gx^2 + Gy^2) per pixel.  
// -----  
__global__ void sobel(const float* in, float* out, int width, int height)  
{  
    const int x = blockIdx.x * blockDim.x + threadIdx.x;  
    const int y = blockIdx.y * blockDim.y + threadIdx.y;  
    if (x < width && y < height)  
    {  
        // Clamp the 3x3 neighbourhood at the borders (replicate-edge).  
        const int xm = max(x - 1, 0), xp = min(x + 1, width - 1);  
        const int ym = max(y - 1, 0), yp = min(y + 1, height - 1);  
  
        const float* r0 = in + ym * width;  
        const float* r1 = in + y * width;  
        const float* r2 = in + yp * width;  
  
        // Horizontal derivative Gx: [-1 0 1] across each of the three rows,  
        // weighted [1 2 1] down the columns.  
        const float gx = (r2[xp] + 2.0f * r1[xp] + r0[xp])  
            - (r2[xm] + 2.0f * r1[xm] + r0[xm]);  
        // Vertical derivative Gy: [-1 0 1] down the columns,  
        // weighted [1 2 1] across the rows.  
        const float gy = (r2[xm] + 2.0f * r2[x] + r2[xp])  
            - (r0[xm] + 2.0f * r0[x] + r0[xp]);  
  
        // Magnitude. sqrtf is the device-side square root; the SFU  
        // approximation is acceptable here (we scale to uchar output).  
        out[y * width + x] = sqrtf(gx * gx + gy * gy);  
    }  
}
```

The separable structure, annotated. G_x is a derivative in `x` (`[-1 0 1]`) convolved with a smoothing in `y` (`[1 2 1]`); G_y is the transpose. The kernel reads 9 pixels and produces two

convolutions - the separable factoring is what keeps it at 9 reads instead of 18.

15.5 The Streaming Host Pipeline

The frame loop uses the machinery of Chapters 4 and 6: pinned host memory, two streams, and double buffering so transfers overlap kernels:

```
// -----  
// One "frame" = load RGB, run the 4 kernels, store edges. Frames arrive in  
// host buffers h_rgb[0] and h_rgb[1]; the GPU processes one while the DMA  
// engine uploads the next (Chapter 6, 6.5).  
// -----  
void processFrames(/* ... device buffers, streams, sizes ... */)   
{  
    const int  numPixels = width * height;  
    const bool last = (frame == frameCount - 1);  
  
    const int cur = frame % 2;          // buffer holding THIS frame's input  
    const int nxt = (frame + 1) % 2;    // buffer for the NEXT frame  
  
    // Upload the NEXT frame while THIS one computes (pinned memory!):  
    if (!last)  
        CHECK(cudaMemcpyAsync(d_rgb[nxt], h_rgb[nxt],  
                               rgbBytes, cudaMemcpyHostToDevice, sCopy));  
  
    // Make the compute stream wait for this frame's copy:  
    CHECK(cudaEventRecord(copyDone[nxt], sCopy));  
    CHECK(cudaStreamWaitEvent(sCompute, copyDone[nxt], 0));  
  
    // The four stages, all in the compute stream, in order:  
    const dim3 block(256);  
    const dim3 grid((numPixels + 255) / 256);  
    rgbToGray<<<grid, block, 0, sCompute>>>(d_rgb[cur], d_gray, numPixels);  
  
    const dim3 b2(32, 8); // 2-D block: 32 x 8 threads  
    const dim3 g2((width + 31) / 32, (height + 7) / 8);  
    blurH    <<<g2, b2, 0, sCompute>>>(d_gray, d_blurred, width, height);  
    blurV    <<<g2, b2, 0, sCompute>>>(d_blurred, d_blurred2, width, height);  
    sobel    <<<g2, b2, 0, sCompute>>>(d_blurred2, d_edges, width, height);  
  
    histogram<<<g2, b2, 0, sCompute>>>(d_edges, d_hist, numPixels);  
    // (histogram kernel as in Chapter 8, 8.8)  
  
    // Copy the edge map back (device -> host, pinned, async):  
    CHECK(cudaMemcpyAsync(h_edges[cur], d_edges, grayBytes,  
                          cudaMemcpyDeviceToHost, sCompute));  
}
```

Why two streams and events? The copy for frame `n+1` and the kernels for frame `n` are independent work on *different* buffers - the exact condition for overlap (§6.5). The

event/dependency pair (`cudaEventRecord` + `cudaStreamWaitEvent`) keeps the ordering correct on every iteration without serialising the pipeline.

Why the 2-D block for the stencil kernels? The 2-D grid lets each thread own one `(x, y)` pixel with natural indexing. The 32×8 block shape keeps blocks tile-shaped (32 = warp width in `x`, so warps are row-aligned).

15.6 The Same Pipeline in Thrust

The library version replaces the four hand-written kernels with four `thrust::transform` calls (Chapter 11). The stencil kernels need neighbouring pixels, which `transform` provides via *shifted iterators*:

```
#include <thrust/iterator/zip_iterator.h>
#include <thrust/iterator/counting_iterator.h>

// Greyscale: pure elementwise -> a plain transform functor.
struct ToGray {
    __device__ float operator()(const uchar3& px) const {
        return 0.299f * px.x + 0.587f * px.y + 0.114f * px.z;
    }
};

// Blur with halo: the functor receives the pixel INDEX and the row pointer;
// it reads its own 5-tap window. (A production version would use a proper
// stencil iterator; this keeps the index arithmetic visible.)
struct BlurH5 {
    const float* in; int width;
    __device__ float operator()(int i) const {
        const int x = i % width, y = i / width;
        const float* row = in + y * width;
        float acc = 0.0f;
        const float w[5] = {0.06136f, 0.24477f, 0.38774f, 0.24477f, 0.06136f};
        for (int t = -2; t <= 2; ++t) {
            const int sx = min(max(x + t, 0), width - 1);
            acc += w[t + 2] * row[sx];
        }
        return acc;
    }
};

// Host side: chain the stages over device vectors.
thrust::device_vector<uchar3> d_rgb(...);
thrust::device_vector<float> d_gray(n), d_blur(n), d_edges(n);

thrust::transform(d_rgb.begin(), d_rgb.end(), d_gray.begin(), ToGray());
thrust::transform(thrust::counting_iterator<int>(0),
                  thrust::counting_iterator<int>(n),
                  d_blur.begin(), BlurH5{raw_pointer(d_gray), width});
// ... blurV and sobel follow the same pattern; histogram is a thrust::reduce
// over a per-bin functor, or thrust::sort + adjacent-difference.
```

The trade, stated plainly. Thrust removes the launch plumbing and the boundary guards; the `counting_iterator` + index-arithmetic pattern reintroduces exactly the stencil logic the hand-written kernel had. For *elementwise* stages (greyscale, magnitude) Thrust is a clear win; for *stencil* stages the custom kernel of §15.3 is no more code and is easier to tune. This is the Chapter 11 decision procedure in live action.

15.7 The Same Pipeline in CUDA-Oxide

With CUDA-Oxide (Chapter 14), the greyscale stage becomes a `#[kernel]` Rust function - the same single-source style, with `DisjointSlice` giving the no-alias guarantee:

```
use cuda_device::{cuda_module, kernel, thread, DisjointSlice};

#[cuda_module]
mod kernels {
    use super::*;

    // Greyscale in pure Rust. DisjointSlice<f32> guarantees exclusive
    // output slots; the input is read-only.
    #[kernel]
    pub fn rgb_to_gray(rgb: &[u8], gray: DisjointSlice<f32>, num_pixels: u32) {
        let idx = thread::index_1d();
        let i = idx.get();
        if (i < num_pixels) {
            // RGB is packed 3 bytes/pixel; u8 -> f32 conversion is explicit.
            let base = (i as usize) * 3;
            let r = rgb[base] as f32;
            let g = rgb[base + 1] as f32;
            let b = rgb[base + 2] as f32;
            if let Some(out) = gray.get_mut(idx) {
                *out = 0.299 * r + 0.587 * g + 0.114 * b;
            }
        }
    }
}
```

What this demonstrates. The kernel is written in the language of the host, indexed by the fused `thread::index_1d()` (Chapter 14), and protected by `DisjointSlice` - no alias, no manual boundary contract. The stencil kernels (blur, Sobel) follow the same shape with the same per-tap clamping logic as the C++ versions, and the pipeline host code reuses the `cuda-async DeviceOperation` chaining of Chapter 14, §14.6. As Chapter 14 warned: the API is alpha, the shape is the point.

15.8 Verification: The Differential Test

A pipeline that produces *wrong* edges at 60 FPS is worse than a correct one at 10 FPS. The verification strategy is the differential test:

1. **A CPU reference** implements the same four stages with plain loops (trivially correct, slow).
2. **The GPU pipeline** runs on a test image.
3. **Compare** stage by stage: greyscale, blurred, and edge maps must agree within a tolerance (`1e-4` for float stages; `uchar` output compared exactly).
4. **Property checks** on real data: the histogram bins fall in expected ranges; an all-black image yields all-zero edges (a *known-answer* test).

The differential test is the Chapter 16 discipline applied to the capstone: it converts “the pipeline works” into a measurable, repeatable assertion, and it is exactly what makes the *second* implementation (Thrust) and the *third* (CUDA-Oxide) trustworthy - they must pass the same test as the first.

15.9 Measurement: The Report Card

The pipeline’s performance is measured with CUDA events (Chapter 6, §6.4), over many frames, with warm-up excluded:

```
// Per-stage timing with events (the honest instrument, 6.4):
cudaEventRecord(start, sCompute);
rgbToGray<<<...>>>(...);
cudaEventRecord(mid, sCompute);
blurH<<<...>>>(); blurV<<<...>>>(); sobel<<<...>>>();
cudaEventRecord(stop, sCompute);
cudaEventSynchronize(stop);
float msStage1 = 0, msRest = 0;
cudaEventElapsedTime(&msStage1, start, mid);
cudaEventElapsedTime(&msRest, mid, stop);
```

The report card for a 1920×1080 frame on a modern GPU, as teaching numbers:

Stage	Time	Bandwidth (3.35 TB/s peak)	Roofline verdict
rgbToGray	~0.4 ms	~75% of peak	Memory-bound (as predicted)
blurH + blurV	~1.0 ms	~70% of peak	Memory-bound, halo cost visible
sobel	~0.5 ms	~70% of peak	Memory-bound
histogram	~0.3 ms	-	Atomic overhead, privatised
Total compute	~2.2 ms	-	450+ FPS, transfer-limited overall

The roofline (Chapter 1) *predicted* the memory-bound verdicts before any code ran: every stage moves ~1 byte of data per pixel per pass with a handful of FLOPs - far below the ridge point. The measurement confirms the prediction. That is the loop this book teaches: predict with the model, confirm with the instrument, optimise only the confirmed bottleneck.

15.10 The Capstone in One Paragraph

The pipeline is the entire book compressed: coalesced kernels with explicit index arithmetic (Chapters 3, 7), synchronisation-free stages and a privatised histogram (Chapters 5, 8), pinned memory and streamed double buffering (Chapters 4, 6), library and language alternatives that must pass the same differential test (Chapters 11, 13, 14), and a measurement discipline that turns opinions into numbers (Chapter 16). If you can build this pipeline and explain every line, you have graduated from this book.

Key Takeaways

- A pipeline is a chain of kernels; a fast pipeline is one that never waits (streams + pinned memory + events).
- A separable Gaussian is 2 x 5 taps instead of 25; clamp each stencil tap at image borders independently.
- The differential test against a CPU reference is what makes a second and third implementation trustworthy.
- The roofline predicted every stage of the capstone was memory-bound before any code ran.
- The report card: median of many runs, fixed environment, events for device time.

15.11 Exercises

1. Why is the separable blur “10 taps instead of 25”? Derive the count for a 5-tap separable Gaussian versus a full 5×5 stencil, and for a 9-tap version.
2. The first `blurH` in §15.3 was “plausible and wrong”. Fix the comment explaining what was wrong, and state the property the corrected kernel guarantees at the borders.
3. In the streaming loop, why must `h_rgb` be *pinned* memory? Trace what happens if it is pageable.
4. The differential test uses a tolerance of `1e-4` for float stages. Why not exact equality? (Hint: Chapter 5, §5.6.)
5. Using the roofline model, predict whether making the blur a *single* fused 5×5 kernel (25 taps, no intermediate) would be faster or slower than the two-pass version, and explain the trade.

Chapter 16: Profiling, Debugging & Performance Engineering

“A performance bug and a correctness bug are the same bug: your model of the machine is wrong. The tools in this chapter find the model.”

Every chapter so far has claimed “this is faster because...”. This chapter is about *proving* it. We cover the four instruments of GPU engineering - **Nsight Systems** and **Nsight Compute** (profilers), **Compute Sanitizer** (debugger), and **the benchmarking discipline** - plus the verification techniques (differential and property testing) that keep optimisations honest. By the end you can take any kernel from this book and answer two questions reproducibly: *is it correct?* and *is it fast?*

16.1 The Two Profilers, and Why Both

NVIDIA ships two profilers with distinct jobs:

Primitive - Nsight Systems (`nsys`). A *system-level* profiler. It shows the timeline of your whole program: when kernels ran, when transfers ran, when the CPU was idle, how streams overlapped. It answers *where the time goes* - and, crucially, whether the GPU was ever idle waiting for the host. **Primitive - Nsight Compute (`ncu`)**. A *kernel-level* profiler. It reports per-kernel hardware counters: achieved occupancy, memory throughput, shared-memory bank conflicts, warp stall reasons, FLOP counts. It answers *why a kernel is slow*.

The workflow is always: **`nsys` first, `ncu` second**. If the GPU is idle 40% of the time, no amount of kernel tuning helps - the fix is streams and overlap (Chapter 6). Only when the timeline shows the GPU busy do you drill into a kernel with **`ncu`**.

```
# System-level timeline (10 seconds of the application):
nsys profile --duration=10 ./pipeline

# Kernel-level analysis of the sobel kernel (the profiler replays the
# kernel under instrumentation):
ncu --kernel-name regex:sobel --set full ./pipeline
```

Why `ncu` “replays” the kernel. Profiling with full counters changes timing; **`ncu`** runs the kernel multiple times under instrumentation and aggregates counters, so the numbers

describe the kernel, not the profiler’s overhead. This is also why `ncu` cannot profile everything at once - use `--set` presets for the metric groups you need.

16.2 Reading the Timeline (nsys)

A healthy streamed pipeline (Chapter 15) shows:



The GPU bar is continuously busy: copies and kernels overlap, and the only gaps are the unavoidable pipeline priming. The unhealthy versions, and their diagnoses:

Timeline symptom	Diagnosis	Fix
GPU idle between kernel and next copy	Default-stream serialisation	Name streams, <code>cudaStreamNonBlocking</code> (Ch. 6)
Small kernel gaps every frame	Host launch overhead	CUDA Graphs (Ch. 6, §6.7)
Long grey “CPU time” blocks	Host-side stall (I/O, alloc)	Pre-allocate, pin memory (Ch. 4)
Copy and kernel never overlap	Pageable memory	<code>cudaMallocHost</code> (Ch. 4)

Reading `nsys` output is the skill that separates engineers who *measure* from engineers who *guess*: every one of these symptoms has a one-line fix, and each fix is a chapter you have already read.

16.3 The Kernel Report (ncu)

For a single kernel, `ncu --set full` reports the metrics this book has trained you to interpret:

- **Achieved occupancy** (§2.9) - warps resident versus the theoretical max. Low occupancy + memory stalls → not enough warps to hide latency.
- **Memory throughput** - percentage of peak DRAM bandwidth used. Near 100% on a memory-bound kernel means coalescing is working; far below it means the checklist of Chapter 7 (§7.9) applies.
- **Shared-memory bank conflicts** - the count of extra cycles lost to bank conflicts (§2.8, §7.5). Zero is achievable with padding.
- **Warp stall reasons** - *why* warps wait: `long_scoreboard` (waiting on a global load), `short_scoreboard` (shared memory), `barrier` (waiting at `__syncthreads`), `drain` (stores not flushed). Each stall reason points at a different chapter of this book.

The discipline: **record the metric, form a hypothesis, change one thing, re-measure.**

Change one variable at a time - two simultaneous changes make the measurement uninterpretable.

16.4 Compute Sanitizer: The Debugger

Primitive - Compute Sanitizer (`compute-sanitizer`). A runtime tool that instruments your kernel to detect memory and synchronisation errors that would otherwise be silent: out-of-bounds accesses, misaligned accesses, data races, and invalid `__syncthreads` usage.

```
# Memory checking: out-of-bounds, uninitialised, and misaligned accesses.
compute-sanitizer --tool memcheck ./pipeline

# Race checking: finds data races between threads (Chapter 5's bug class).
compute-sanitizer --tool racecheck ./pipeline

# Initialisation checking: reads of uninitialised memory.
compute-sanitizer --tool initcheck ./pipeline

# Synchronisation checking: divergent __syncthreads (Chapter 5, 5.3).
compute-sanitizer --tool synccheck ./pipeline
```

Why these tools matter more on GPUs than on CPUs. A CPU out-of-bounds write usually crashes at the instruction; a GPU out-of-bounds write corrupts *adjacent memory in the same allocation* - the kernel “succeeds”, and the corruption surfaces as a wrong image three stages later. `memcheck` finds the write at the moment it happens, with the thread and instruction identified.

The relationship to this book: **every race, bank conflict, and divergence you learned to reason about in Chapters 5 and 7 has a detector.** Run the detector before you trust your reasoning.

16.5 cuda-gdb: The Kernel Debugger

For bugs that resist the automatic tools, `cuda-gdb` is the interactive debugger for device code: set breakpoints *inside kernels*, inspect `threadIdx` / `blockIdx`, watch registers and shared memory, and step warp by warp.

```
cuda-gdb ./pipeline
(cuda-gdb) break sobel
(cuda-gdb) run
(cuda-gdb) set cuda break_on_launch application    # stop at every kernel
(cuda-gdb) info cuda kernels                     # list active kernels
(cuda-gdb) thread 5                              # select a specific thread
(cuda-gdb) print x                               # inspect kernel variables
```

When to reach for cuda-gdb. After Compute Sanitizer has cleared memory and race errors, a *logical* bug (wrong index arithmetic, wrong stencil weights) remains. Break on the kernel, pick a specific thread (`thread 5`), and check the index formula by hand. This is the interactive version of the comment-audit that Chapter 3’s coding standards demand.

16.6 clock64(): Timing Inside the Kernel

Sometimes the profiler’s replay changes the answer (e.g., for a kernel whose performance depends on cache state). The escape hatch is `clock64()`, which reads a per-SM cycle counter:

```
// Time a code region from INSIDE the kernel. Returns SM cycles.
// Useful when profiler replay perturbs the measurement; otherwise prefer ncu.
__device__ long long profileRegion()
{
    const long long t0 = clock64();
    // ... the region being timed ...
    const long long t1 = clock64();
    return t1 - t0;                // SM cycles (see device clock rate)
}
```

The caveats. `clock64()` measures *this thread’s* view - warps may be preempted by the scheduler mid-region - and the SM clock can vary with power state. Use it for *relative* comparisons of code paths within one kernel run, not as a cross-run benchmark. For cross-run numbers, use events (Chapter 6) and the discipline of §16.7.

16.7 The Benchmarking Discipline

A number from one run is a rumour. The reproducible protocol, applied to every claim in this book:

1. **Warm up.** Run the kernel several times before measuring, so caches, page tables, and JIT state are steady.
2. **Repeat and report the median**, not the mean - the median is robust to the rare outlier (OS preemption, clock boost). Report the spread (P10/P90) alongside.
3. **Use events, not host timers**, for device work (Chapter 6, §6.4).
4. **Fix the environment.** Record the GPU (`nvidia-smi -L`), the CUDA version (`nvcc --version`), the driver, and the compiler flags (`-arch=sm_90`, `-O3`, `--use_fast_math` changes results!).
5. **Verify the output** before trusting the timing. A fast wrong kernel is not a result.

```
// The protocol in miniature. ncu or nsys can further validate, but this
// structure is the minimum reproducible measurement.
float benchmarkKernel(int iters)
{
    // warm-up:
    kernel<<<grid, block>>>(...);  cudaDeviceSynchronize();

    std::vector<float> times;
    for (int r = 0; r < iters; ++r)
    {
        cudaEventRecord(start);  kernel<<<grid, block>>>(...);
        cudaEventRecord(stop);   cudaEventSynchronize(stop);
        float ms;  cudaEventElapsedTime(&ms, start, stop);
        times.push_back(ms);
    }
    std::sort(times.begin(), times.end());
    return times[times.size() / 2];    // median
}
```

16.8 Verification: Differential and Property Testing

Performance engineering without correctness is vandalism. Two techniques from the capstone generalise:

- **Differential testing** - compare the GPU result against a trusted CPU reference (Chapter 15, §15.8). Run it in CI on every change; a “refactor” that changes the last bit of a reduction (Chapter 5, §5.6) gets caught, not shipped.
- **Property testing** - assert invariants that hold for *any* input: a histogram’s counts sum to the input length; a transpose’s output is the input’s transpose; an edge map of a constant image is all zeros. Property tests find the bugs that differential tests miss (both may be wrong in the same way).

The engineering payoff: once the differential and property suites exist, an optimisation is *just* a change you run through the suite. This is what allows the whole optimisation literature - Chapter 7 through 9 - to proceed without fear.

16.9 The Engineering Loop, Formalised

The chapter's whole content reduces to a loop:

1. **Measure** (`nsys` timeline; is the GPU busy?).
2. **Profile** (`ncu`; what is the kernel's bottleneck?).
3. **Hypothesise** (name the chapter that addresses the bottleneck).
4. **Change one thing** (and only one thing).
5. **Re-measure** (median of many runs, fixed environment).
6. **Verify** (differential + property tests still pass).

A loop that skips step 2 or 6 is a gambling habit. A loop that follows all six steps is engineering. Everything in this book - the roofline of Chapter 1, the coalescing of Chapter 7, the pipelines of Chapter 15 - is an argument about what step 3 should say. The loop is how you know the argument was right.

Key Takeaways

- `nsys` answers 'where does the time go' (is the GPU ever idle?); `ncu` answers 'why is this kernel slow' (counters).
- Compute Sanitizer finds what kernels hide: memcheck, racecheck, initcheck, synccheck.
- `cuda-gdb` debugs kernels interactively, thread by thread.
- The benchmark protocol: warm up, repeat, report the median, use events, fix the environment, verify the output.
- The loop - measure, profile, hypothesise, change one thing, re-measure, verify - is the discipline behind every claim in this book.

16.10 Exercises

1. A kernel shows 100% memory throughput in `ncu` but the whole application is slow. Which profiler do you run next, and what do you look for?
2. `racecheck` reports a race in a kernel that "passes all tests". Explain why this is not a contradiction, and what class of bug it represents (hint: Chapter 5, §5.1).
3. Why does the benchmark report the median rather than the mean? Give a concrete source of outliers that the median neutralises.
4. Your colleague optimises a kernel and reports a 30% speedup measured with `std::chrono` around a single launch. List the three things wrong with that measurement.

Epilogue - The Road Ahead

"You now understand a machine that did not exist twenty years ago and will not exist, in its current form, twenty years from now. That is the nature of the field. It is also the point of this book."

You have travelled from the mathematics of parallelism to a streamed, profiled, verified image pipeline, written three ways. Before you close the book, it is worth looking at where the road goes - not as prophecy, but as a map of the territory you are now equipped to navigate.

The Hardware

Every generation of GPU has moved the ridge point of Chapter 1: more FLOPs, more bandwidth, and - most consequentially - *specialised arithmetic*. Tensor cores, introduced with Volta and central to every AI workload since, are a different kind of computer inside the GPU: dense matrix-multiply units that execute in one instruction what a CUDA-core loop would take hundreds of cycles to do. Hopper added the TMA (bulk asynchronous copies) and thread-block clusters; Blackwell continues the trend. The lesson of Chapter 2 still holds - the memory hierarchy, the warp, the SM - but the *arithmetic* landscape keeps splitting into specialised lanes.

The consequence for you: the skills in this book are not obsolete, they are *foundational*. Tensor-core programming is still thread-block programming with a different instruction set; TMA is still coalescing, expressed in bulk. When the next specialised unit appears, you will recognise its shape.

The Software

Three currents are visible today:

- **CUDA is here to stay, and so is its competition.** NVIDIA's ecosystem (CUDA, cuBLAS, Nsight) remains the reference, but the cross-vendor world is real: **SYCL** (Khronos's single-source C++ model), **HIP** (AMD's CUDA-compatible API), and **wgpu/WebGPU** (browser and native Rust). The programming model you learned - grids, warps, coalescing, shared memory - translates directly to all of them, because they are all SIMT machines wearing different clothes.
- **Rust is arriving.** `culandrc` gives Rust a production-grade host (Chapter 13). **CUDA-Oxide** (Chapter 14) is the first credible attempt to bring Rust's guarantees to the kernel itself. Both are young; both point in the same direction - that the GPU's failure modes are, at bottom, *host-language* failure modes, and the languages that eliminate those failure modes will win the mindshare of the next generation of GPU engineers.
- **The libraries are winning, and that is fine.** Every year, more of the hard work moves into tuned libraries (Chapter 11). The engineers who *use* those libraries effectively are not the ones who memorised the API - they are the ones who can read the profiler, explain why a kernel is memory-bound, and know when a custom kernel is actually worth writing. That is exactly what this book trained you to do.

The Discipline

The most durable thing in this book is not a kernel. It is the loop of Chapter 16: measure, profile, hypothesise, change one thing, re-measure, verify. Hardware changes, languages change, libraries change - the loop does not. It is the same discipline a surgeon brings to an operation, a pilot to an approach, and an engineer to a machine they respect. The GPU is a machine worth respecting: it is fast, it is precise, and it will do exactly what you told it, including the wrong things.

The Invitation

This book is a living document, published on GitHub Pages and open to pull requests. If a kernel is unclear, if a claim is unmeasured, if a chapter is missing the concept that confused you - open an issue. The road ahead is paved by the people who walk it, and you are now walking it with the lights on.

The GPU is waiting. It has been waiting since Chapter 1. Go make it do something that is fast, correct, and understood.

- *Arpan Pathak*

Appendix A - CUDA API Reference

“Every primitive of the CUDA programming model, gathered in one place. When a chapter says ‘the primitive’, this is the definition it means.”

This appendix is the book’s vocabulary list: every type, built-in variable, function and flag used in the main text, with its meaning and where it is discussed. It is intended as a lookup table, not as a tutorial - the tutorials are the chapters.

A.1 Execution Configuration

Syntax	Meaning	Chapter
<code>kernel<<<gridDim, blockDim>>> (args...)</code>	Launch <code>kernel</code> with a grid of <code>gridDim</code> blocks of <code>blockDim</code> threads each	3
<code>kernel<<<gridDim, blockDim, sharedBytes, stream>>></code>	As above, with dynamic shared memory (bytes) and an explicit stream	6, 7
<code>dim3</code>	Three- <code>unsigned</code> vector type; fields <code>.x</code> , <code>.y</code> , <code>.z</code>	3
<code>threadIdx</code>	The thread’s position within its block (a <code>dim3</code>)	3
<code>blockIdx</code>	The block’s position within the grid (a <code>dim3</code>)	3
<code>blockDim</code>	Threads per block, as launched (a <code>dim3</code>)	3
<code>gridDim</code>	Blocks per grid, as launched (a <code>dim3</code>)	3
<code>__launch_bounds__(maxThreads, minBlocks)</code>	Compiler directive: register budget for occupancy	9

The universal 1-D global index: `blockIdx.x * blockDim.x + threadIdx.x`. The 2-D composition is in Chapter 3, §3.5.

A.2 Function Qualifiers

Qualifier	Runs on	Called from	Chapter
<code>__global__</code>	Device	Host	3
<code>__device__</code>	Device	Device only	3
<code>__host__</code> (default)	Host	Host	3
<code>__host__ __device__</code>	Both	Both	10

`extern "C" __global__` keeps the kernel symbol unmangled for driver-API / NVRTC lookup (Chapters 12, 13).

A.3 Vector Types

Type	Bytes	Alignment	Notes
<code>uchar3</code>	3	1	RGB pixels; no padding (Chapter 15)
<code>float2</code> , <code>float4</code>	8, 16	4, 16	Vectorised loads (Chapter 7, §7.6)
<code>int2</code> , <code>int4</code> , <code>double2</code> , <code>uint4</code>	8/16/16/16	as size	Same vectorisation rules
<code>size_t</code>	platform	-	Byte sizes; use instead of <code>int</code> (Chapter 3)

Vectorised accesses require alignment to the vector size.

A.4 Memory Management

Function	Behaviour	Chapter
<code>cudaMalloc(void** p, size_t n)</code>	Allocate <code>n</code> bytes in device global memory	3
<code>cudaFree(void* p)</code>	Free a device allocation	3
<code>cudaMemcpy(dst, src, n, kind)</code>	Synchronous copy; <code>kind</code> = <code>HostToDevice</code> , <code>DeviceToHost</code> , <code>DeviceToDevice</code> , <code>HostToHost</code>	3
<code>cudaMemcpyAsync(dst, src, n, kind, stream)</code>	Asynchronous copy, queued on <code>stream</code> ; requires pinned host memory	4, 6
<code>cudaMallocHost(void** p, size_t n)</code>	Allocate pinned (page-locked) host memory	4
<code>cudaHostAlloc(void** p, size_t n, flags)</code>	Pinned host memory; <code>cudaHostAllocMapped</code> adds zero-copy mapping	4
<code>cudaHostGetDevicePointer(void** dp, void* hp, 0)</code>	Device pointer for zero-copy mapped host memory	4
<code>cudaFreeHost(void* p)</code>	Free pinned host memory	4
<code>cudaMallocManaged(void** p, size_t n)</code>	Unified memory (host + device address space)	4
<code>cudaMemPrefetchAsync(p, n, device, stream)</code>	Migrate unified-memory pages now	4
<code>cudaMemcpyToSymbol(sym, src, n)</code>	Copy into <code>__constant__</code> memory	7

Memory kinds and when to use each: Chapter 4, §4.7.

A.5 Synchronisation and Memory Ordering

Primitive	Meaning	Chapter
<code>__syncthreads()</code>	Block-wide barrier; must be uniformly reachable	5
<code>__threadfence()</code>	Order my device-scope global accesses	5
<code>__threadfence_block()</code>	Order my block-scope accesses	5
<code>__threadfence_system()</code>	Order host+device accesses	5
<code>volatile</code>	Disable register caching of a location	5
<code>atomicAdd/Sub/Exch/CAS/Min/Max/And/Or/Xor</code>	Hardware read-modify-write; return old value	5
<code>cuda::atomic<T, scope></code>	C++20-style atomic with memory orders (CUDA 12)	10

The atomics table with semantics: Chapter 5, §5.5.

A.6 Streams and Events

Function	Behaviour
<code>cudaStreamCreate(&s)</code>	Create a stream
<code>cudaStreamDestroy(s)</code>	Destroy a stream
<code>cudaStreamCreateWithFlags(&s, flag)</code>	<code>cudaStreamNonBlocking</code> disables default-stream sync
<code>cudaStreamCreateWithPriority(&s, flag, prio)</code>	Priority stream; range from <code>cudaDeviceGetStreamPriorityRange</code>
<code>cudaStreamSynchronize(s)</code>	Wait for all work in <code>s</code>
<code>cudaEventCreate(&e)</code> / <code>cudaEventDestroy(e)</code>	Create/destroy an event
<code>cudaEventRecord(e, s)</code>	Mark the stream position
<code>cudaEventSynchronize(e)</code>	Wait until the device reaches <code>e</code>
<code>cudaEventElapsedTime(&ms, e0, e1)</code>	Time between two events
<code>cudaStreamWaitEvent(s, e)</code>	Make <code>s</code> wait for <code>e</code> (cross-stream dependency)
<code>cudaGraphCreate/Instantiate/Launch/Destroy</code>	Capture and replay device work
<code>cudaDeviceSynchronize()</code>	Wait for all device work

The synchronisation cheat sheet: Chapter 6, §6.8.

A.7 Error Handling

Primitive	Behaviour	Chapter
<code>cudaError_t</code>	Enum; <code>cudaSuccess == 0</code> , everything else is an error	3
<code>cudaGetLastError()</code>	Return and clear the last asynchronous error	3
<code>cudaGetErrorString(e)</code>	Human-readable error text	3
<code>cudaDeviceSynchronize()</code>	Also surfaces async kernel errors	3, 6
<code>cublasStatus_t</code> , <code>nVRTCResult</code> , <code>CUresult</code>	Library error types (cuBLAS, NVRTC, driver API)	11, 12

Two error modes (synchronous vs asynchronous) and the `CHECK` discipline: Chapter 3, §3.8.

A.8 Device Math Library (selected)

`fminf`, `fmaxf`, `sqrtf`, `fabsf`, `sinf`, `cosf`, `expf`, `logf`, `powf`, `fmaf` (fused multiply-add). The `__host__ __device__` versions work on both sides (Chapter 10, §10.4). Fast approximations live in the `__` prefixed forms (`__expf`, `__sinf`); enable globally with `--use_fast_math` - but measure before trusting (Chapter 16, §16.7).

A.9 Warp-Level Primitives

Primitive	Meaning	Chapter
<code>__shfl_down_sync(mask, val, delta)</code>	Move <code>val</code> from lane <code>lane+delta</code> to <code>lane</code>	8
<code>__shfl_sync</code> , <code>__shfl_up_sync</code> , <code>__shfl_xor_sync</code>	Other shuffle directions	8
<code>0xfffffffffu</code>	The 32-lane mask for <code>_sync</code> primitives	8
<code>__activemask()</code>	Mask of currently active lanes (use with care)	8

All `_sync` primitives require all named lanes to execute them.

A.10 Driver API and NVRTC (Chapter 12)

Primitive	Meaning
<code>cuInit</code> , <code>cuDeviceGet</code> , <code>cuCtxCreate</code> , <code>cuCtxDestroy</code>	Context lifecycle
<code>cuModuleLoadData</code> , <code>cuModuleGetFunction</code> , <code>cuModuleUnload</code>	Load PTX/cubin, fetch kernel by name
<code>cuLaunchKernel(f, gx,gy,gz, bx,by,bz, smem, stream, args, extra)</code>	Raw launch with <code>void*</code> argument array
<code>nVRTCCreateProgram</code> , <code>nVRTCCompileProgram</code> , <code>nVRTCGetPTX</code> , <code>nVRTCGetProgramLog</code>	Runtime compilation of CUDA source
PTX / cubin / fatbin	Portable ISA / SASS binary / multi-arch container

A.11 Tools and Environment (Chapter 16)

Tool	Purpose
<code>nvcc</code>	Offline compiler (<code>-arch=sm_90</code> , <code>-ptx</code>)
<code>nsys profile</code>	System-level timeline
<code>ncu --set full</code>	Kernel-level counters
<code>compute-sanitizer --tool memcheck/racecheck/initcheck/synccheck</code>	Runtime error detection
<code>cuda-gdb</code>	Interactive device debugger
<code>clock64()</code>	In-kernel cycle counter
<code>CUDA_CACHE_MAXSIZE</code>	JIT cache size (Chapter 12)

Appendix B - Mathematical Notation Reference

“The notation used in this book, defined once and used everywhere.”

This appendix gathers the mathematics that appears throughout the text. It is not a mathematics course; it is a dictionary. Every symbol below appears at least once in the main chapters.

B.1 Sets and Scalars

Symbol	Meaning	First use
$\mathbb{R}$	The real numbers; $\mathbb{R}^n$ is n -dimensional real space	Ch. 1
n, N, k, i, j	Indices and sizes (integers)	Ch. 1-15
p	The number of processing units (threads, cores)	Ch. 1
f	The serial fraction of a workload (Amdahl)	Ch. 1
s	The serial fraction of parallel time (Gustafson)	Ch. 1

B.2 Performance Quantities

Symbol	Meaning	Definition	First use
T_1	Serial execution time	-	Ch. 1
T_p	Execution time on p units	-	Ch. 1
$S(p)$	Speedup	T_1/T_p	Ch. 1
$E(p)$	Efficiency	$S(p)/p$	Ch. 1
P_{peak}	Peak FLOP rate (FLOP/s)	-	Ch. 1
B	Peak memory bandwidth (bytes/s)	-	Ch. 1
I	Arithmetic intensity	FLOPs ÷ bytes	Ch. 1
I_{ridge}	Ridge point	P_{peak}/B	Ch. 1

The roofline inequality: $P \leq \min(P_{\text{peak}}, I \cdot B)$, with the two regimes *compute-bound* ($I > I_{\text{ridge}}$) and *memory-bound* ($I < I_{\text{ridge}}$). Chapter 1, §1.8.

B.3 Sums and Sequences

Symbol	Meaning	First use
$\sum_{i=0}^{n-1} a_i$	The sum of the sequence $a_0, \dots, a_{n-1}$	Ch. 8
a_i	The i -th element of a sequence	Ch. 8
$\log_2 n$	The base-2 logarithm of n (the tree height)	Ch. 8
$\lfloor x \rfloor$	The floor of x (largest integer $\leq x$)	Ch. 2

Tree reduction performs $n/2$ additions per level over $\log_2 n$ levels; the bank index is $\lfloor \text{addr}/4 \rfloor \bmod 32$. Chapter 2, §2.8; Chapter 8.

B.4 Matrices and Vectors

Symbol	Meaning	First use
A, B, C	Matrices (bold or capital letters)	Ch. 9
$A[i][j]$ or a_{ij}	The element at row i , column j	Ch. 9
N	Matrix dimension ($N \times N$)	Ch. 9
$C = A \times B$	Matrix product: $c_{ij} = \sum_k a_{ik} b_{kj}$	Ch. 9
G_x, G_y	Sobel derivative kernels	Ch. 15

The FLOP count of $N \times N$ multiplication: $2N^3$. The arithmetic intensity: $N/6$ FLOP/byte. Chapter 9, §9.1.

B.5 Statistics and Error

Symbol	Meaning	First use
σ	Standard deviation (Gaussian blur radius)	Ch. 15
$\backslash(\backslash\max$	$x - y$	$\backslash$
1×10^{-4}	The float-stage verification tolerance	Ch. 15

The Gaussian weights of the 5-tap blur, $\sigma = 1$:
[0.06136, 0.24477, 0.38774, 0.24477, 0.06136]. Chapter 15, §15.3.

B.6 Greek Letters Used

Letter	Role in this book
α	SAXPY scaling factor ($y = \alpha x + y$)
β	cuBLAS GEMM scaling factor ($C = \alpha AB + \beta C$)
σ	Gaussian standard deviation
Σ	Summation operator

B.7 Conventions

- **Row-major** storage is assumed everywhere unless stated otherwise: element (i, j) of a W -wide matrix lives at linear index $i \cdot W + j$. Consecutive threads map to consecutive j - the coalescing convention of Chapter 7.
- **Zero-based** indexing, matching the hardware (`threadIdx.x` starts at 0).
- **FLOPs** counts a fused multiply-add as two operations (the convention behind Chapter 2's peak rates).
- **Powers of two** dominate block sizes and tile sizes; when a formula requires one, the text says so (Chapter 8, §8.3).

Appendix C - Recommended Reading & Tools

"A book is a beginning, not a destination. Here is where the road continues."

C.1 Books

- **Programming Massively Parallel Processors: A Hands-on Approach** - David B. Kirk and Wen-mei W. Hwu. The standard academic text on GPU programming; the source of many of the patterns this book presents from first principles (reduction, tiling, coalescing).
- **CUDA C++ Best Practices Guide** - NVIDIA. The official optimisation handbook; the §7.9 checklist in this book is a compressed version of its structure.
- **Parallel Programming with C++** - Richard Vuduc et al. (course notes). The mathematics of Chapter 1 in more depth (Amdahl, Gustafson, roofline).
- **The Rust Book** - Steve Klabnik and Carol Nichols. The language reference for Part V, free online.
- **Performance Analysis of the CUDA Programming Model** - for the warp-level and SIMT details behind Chapter 2.

C.2 Official Documentation (Primary Sources)

- **CUDA C++ Programming Guide** - the authoritative language reference.
- **PTX ISA Reference** - the virtual ISA of Chapters 3 and 12, in detail.
- **CUDA Toolkit Documentation** - API references for the runtime, driver, cuBLAS, Thrust, CUB, cuFFT, cuRAND, NVRTC.
- **Nsight Compute / Nsight Systems User Guides** - the profilers of Chapter 16.
- **Compute Sanitizer User Guide** - the debugger of Chapter 16.
- **CUDA-Oxide (NVlabs/cuda-oxide)** - the README and examples are the primary source for Chapter 14; the project moves quickly, so read the repository, not just this book.

C.3 Tools

Tool	Purpose	Chapt
<code>nvcc</code>	CUDA compiler driver	3
<code>nsys</code>	System-level profiling	16
<code>ncu</code>	Kernel-level profiling	16
<code>compute-sanitizer</code>	Memory/race/init/sync checking	16
<code>cuda-gdb</code>	Device debugging	16
<code>cuobjdump</code> , <code>nvdiasm</code>	Inspect cubins, SASS	12
<code>deviceQuery</code> (CUDA sample)	Hardware capabilities	2
<code>cudaOccupancyMaxActiveBlocksPerMultiprocessor</code>	Occupancy computation	2, 16
<code>cargo-oxide</code>	CUDA-Oxide build driver	14
<code>cudarc</code> (crates.io)	Rust host wrapper	13

C.4 Online Resources

- **NVIDIA Developer Blog** - architecture deep dives (tensor cores, TMA, CUDA Graphs).
- **NVIDIA GPU Computing Samples** (`cuda-samples` repository) - reference kernels for every pattern in this book.
- **NVIDIA's official CUDA-Oxide repository and Discord** - the community around the Rust compiler of Chapter 14.
- **crates.io/cudarc** - the Rust host library of Chapter 13, with examples.

C.5 Cloud GPU Services

If you do not own an NVIDIA GPU, the cloud gives you enough free compute to finish this book. The quickest starts: **Google Colab** (free T4 in a browser, zero setup) and **Kaggle Notebooks** (roughly 30 free GPU hours per week). For serious sessions, the big clouds offer new-account credits (Google Cloud roughly USD 300, Microsoft Azure roughly USD 200); for cheap on-demand GPUs, try Lambda, RunPod or Vast.ai; for serverless Python with recurring credits, Modal. **NVIDIA LaunchPad** offers free, time-boxed hands-on labs on real NVIDIA hardware.

The repository README keeps a fuller comparison table with links; credit amounts change frequently, so always check the provider's current terms.

C.6 How to Continue

1. **Re-implement the capstone** (Chapter 15) from memory, without looking. The gaps in your memory are the gaps in your understanding.
2. **Profile your own machine.** Run `deviceQuery`, compute your ridge point (Chapter 1), and take one kernel from this book to 90% of measured peak bandwidth using Chapter 7's checklist.
3. **Read the CUDA-Oxide repository** and track its releases. The alpha project of Chapter 14 is the fastest-moving part of this book.
4. **Port the pipeline** to a cross-vendor API (SYCL, HIP, or wgpu) and observe which ideas transfer unchanged. The answer - almost all of them - is the epilogue's point made measurable.